

Tema-0. Presentación.

Tema-1: Búsqueda de soluciones en IA

Heurísticos y Metaheurísticos.

Búsqueda sistemática (global), local. Tipología Metaheurísticos.

Tema-2: Metaheurísticas Poblacionales (Evolutivas)

Algoritmos Genéticos

Algoritmos meméticos.

Búsqueda dispersa (Scatter Search).



**PRÁCTICAS
Aplicación
Metaheurísticas
(AG)**

Tema-3: Metaheurísticos de Mejora Iterativa (Búsqueda Local)

Búsqueda por Escalada. Multi-arranque. Búsqueda Tabú.

Enfriamiento Simulado. Búsqueda en Haz.

Tema-4: Metaheurísticas Constructivas. GRASP.

Tema-5: Inteligencia Social, Colectiva (Social Swarm Intelligence)

Algoritmo de las Hormigas (Ant Colony Optimization).

Colonia de Abejas (Bee Colony Optimization).

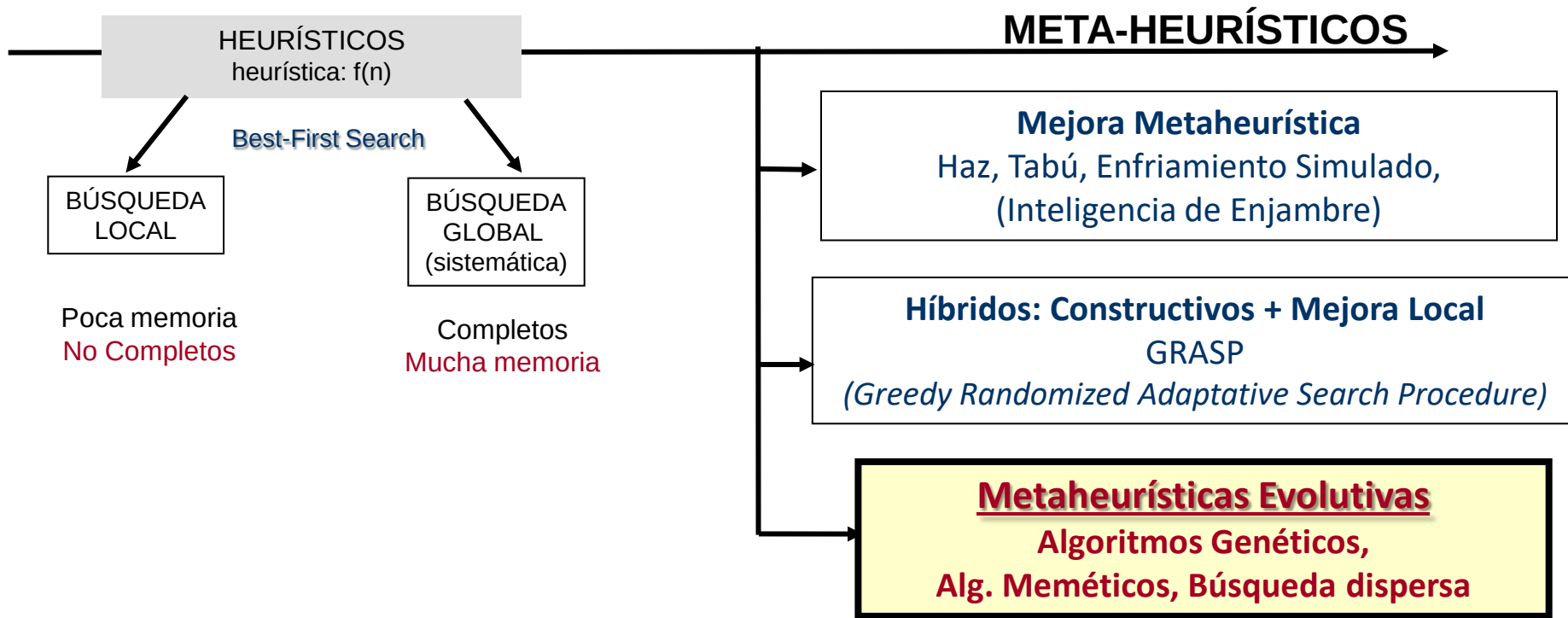
Enjambre de partículas (Particle Swarm Optimization). Variantes.

Tema-6: Búsqueda en Juegos Complejos

Bibliografía

(Artículos en Poliformat!)

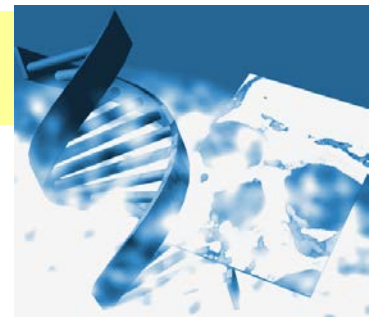
- **An Introduction to Metaheuristics for Optimization.** B. Chopard, M. Tomassini. Springer (2018). Cap. 8 & 9.
- **Inteligencia Artificial. Técnicas, métodos y aplicaciones (cap. 11).** Palma, Marín (eds). Mc Graw Hill (2008).
- **Monografía: Metaheurísticas.** Inteligencia Artificial, Vol 7, No 19 (ed. B. Melián, J.A. Moreno, J. Marcos) (2003). <http://journal.iberamia.org/index.php/ia/article/view/360/article%20%281%29.pdf>
- **Genetic Algorithms: Advances in Research & Applications**, J. Carson. Nova Sc. Pub. /2017)
- **Handbook of Evolutionay Computation**, T. Bäck, D.B. Fogel, Z. Michalewicz. Oxford University Press, 2007
- <https://geneticprogramming.com/>



Principales características

- Parten de un **conjunto de soluciones (población)** que se van reconstruyendo y mejorando.
- Mejoran la búsqueda local, sin la complejidad espacial/temporal de una búsqueda global.
- Existe una **Colaboración** y **Competición** entre soluciones (individuos) mediante las operaciones de **cruce** y **selección**. Tiene una componente **estocástica**.
- Puede haber un **Control Centralizado** y los individuos no son autónomos (Alg. Genéticos), o descentralizado y con autonomía (Alg. Meméticos).
- Característica **Any-Time**: Mejora sucesiva de las soluciones.
- Implementación sencilla. Dificultad: **Diseño Individuo** y **Ajuste**

Algoritmos Genéticos



1.- Conceptos Generales

2.- Estructura de un Algoritmo Genético

3.- Etapas de un Algoritmo Genético:

- Codificación en un AG
- Población Inicial
- Aptitud, fitness.
- Estrategia de Selección
- Cruce, Mutación, Reemplazo de la Población
- Condición de Parada.



4.- Algoritmos Genéticos Multiobjetivo

5.- Ejemplos

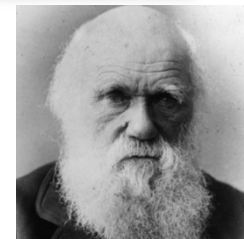


1) Conceptos generales.

Ideados por John Holland (1975) inspirándose en la evolución natural de los seres vivos.
“*Adaptation in Natural and Artificial Systems*”. University of Michigan Press, 1975

Idea subyacente:

- La población evoluciona de forma natural, mejorando su adaptación.
- En el transcurso de la **evolución** se generan **poblaciones sucesivas**, con información **genética** de los padres previos, y cuya adecuación al entorno va **mejorando** sucesivamente.



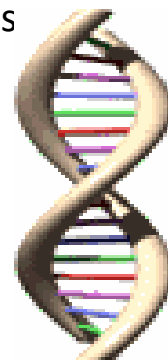
Los AG se inspiran en los procesos de Evolución Natural y Genética (métodos bioinspirados):

- Un AG evoluciona a partir de una población de **soluciones inicial**, intentando producir nuevas **generaciones de soluciones** que sean mejores que la anterior.
- El **proceso evolutivo** es guiado por **decisiones probabilísticas** (*componentes aleatorios*) basadas en la **adecuación** de los individuos (fitness). El fitness puede tener una componente heurística.
- Al final del proceso, se espera obtener un **buen individuo: buena solución** global al problema.
- Orientados hacia la resolución (**optimización**) de problemas combinatorios.
- Los AG son el método metaheurístico más aplicado y con más relevantes resultados prácticos
- AG: mejora colectiva de individuos con control centralizado

Población de Individuos ⇔ Conjunto de Soluciones

Evolución ⇔ Búsqueda

Mejor individuo ⇔ Solución Final



Computación Evolutiva: métodos metaheurísticos evolutivos (bio-inspirados).

Población de Individuos $\Leftrightarrow$ Conjunto de Soluciones
Evolución $\Leftrightarrow$ Búsqueda
Mejor individuo $\Leftrightarrow$ Solución Final

3) Mejora de Soluciones

1) Soluciones Iniciales



2) Nuevas Soluciones

Aplicaciones de los AG

- Diseño y configuración
- Optimización combinatoria en dominios reales
- Modelado e identificación de sistemas.
- Planificación y control
- Vida artificial
- Aprendizaje y minería de datos
- Internet y Sistemas de Recuperación de Información, etc....

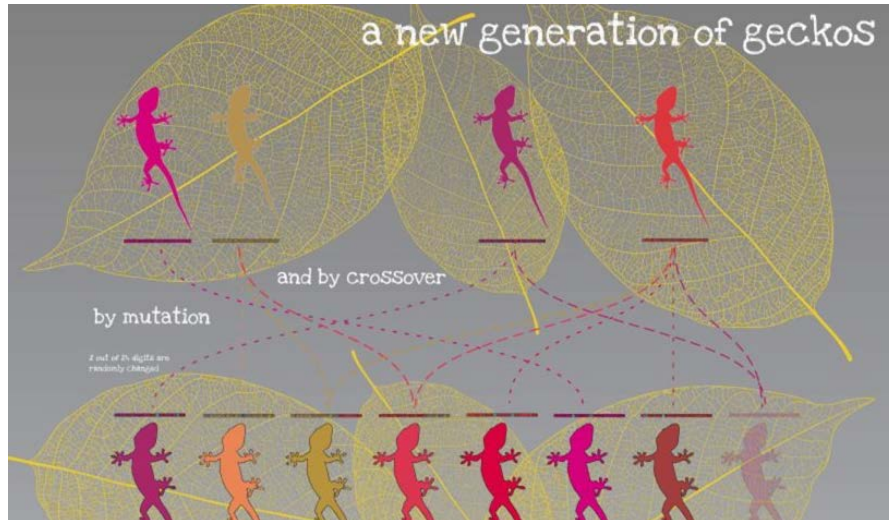


Winner of the Second Video Competition of
the IEEE Computational Intelligence Society



IEEE Computational Intelligence Society

Nature-Inspired Problem Solving



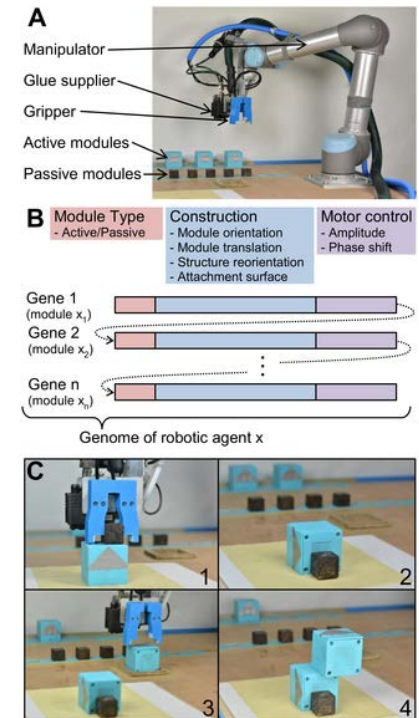
1st place

Team: Evolutionary Computation -
A Technology Inspired by Nature

El límite?...

Se está investigando el
proceso de evolución
en los robots,
construyendo un robot
'madre' que pueda
diseñar, construir y
evaluar sus propios
'hijos', y usar las
conclusiones para
mejorar la siguiente
generación.

Todo ello sin
intervención humana:



On the origin of (robot) species

Researchers have observed the process of evolution by natural selection at work in robots, by constructing a 'mother' robot that can design, build and test its own 'children', and then use the results to improve the performance of the next generation, without relying on computer simulation or human intervention.

Aprendizaje Neuronal mediante Algoritmos Genéticos

An Introduction to Meta-Heuristics, Qiangfu Zhao

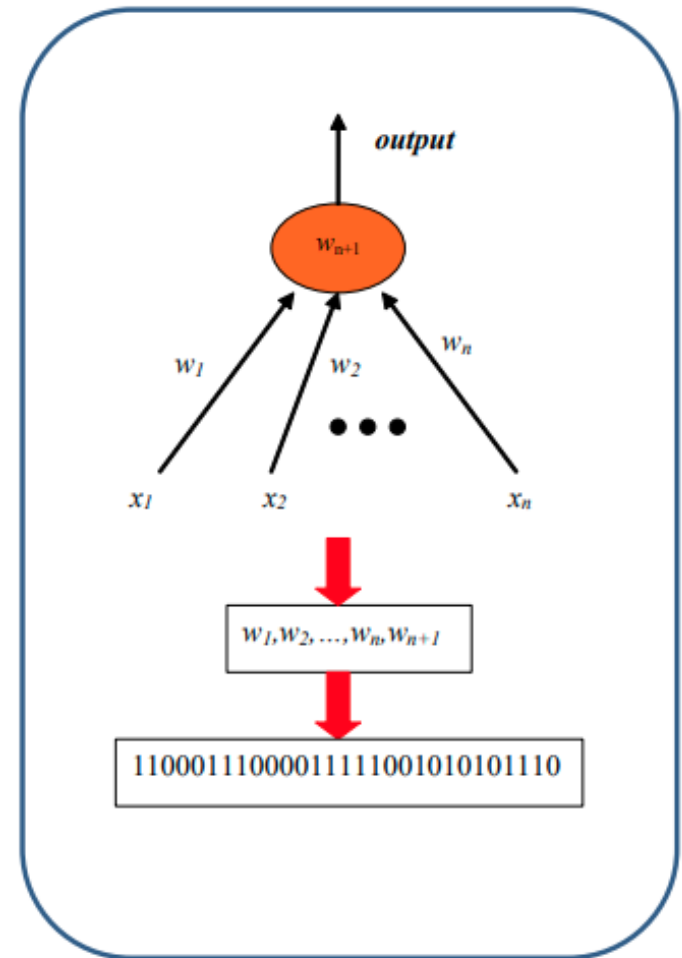
- A neuron is represented by its weights $\{w_1..w_n\}$ and the threshold $\{w_{n+1}\}$. The threshold is considered a special weight.
- The genotype of a neuron can be defined as the weight list $\{w_1, ..., w_{n+1}\}$. Each weight can be represented in a binary number.
- Reconstruction of the phenotype from the genotype:

B : number of bits per weight

b_{ik} : k -th bit for the i -th weight

$$y_i = \sum_{k=1}^B b_{ik} 2^{-k}, \quad w_i = a[y_i] + b$$

where a and b are the scaling and shifting factors.



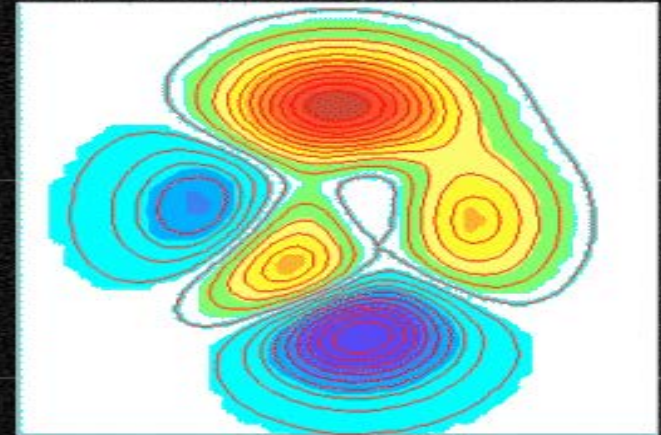
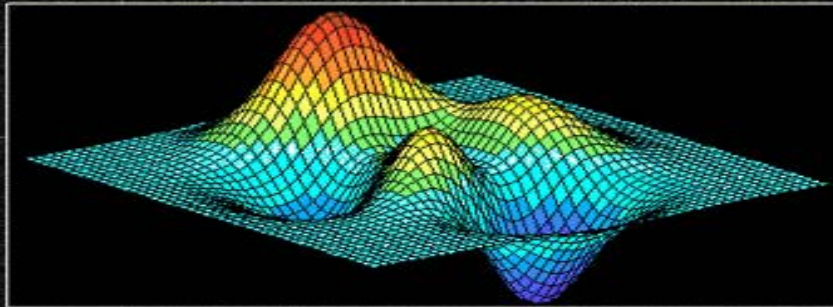
GA for deep learning (used for finding the “best structure” of a deep convolutional neural network)

- [“An introduction to optimizacion: Genetic Algorithm”. Qiangfu Zhao](#)
- **“Fast and Unsupervised Neural Architecture Evolution for Visual Representation Learning,”** Xue, Chen, Xie, Zhang, Gong and Doermann, in IEEE Comp. Intelligence Magazine, 16, 3, pp. 22-32, Aug. 2021.

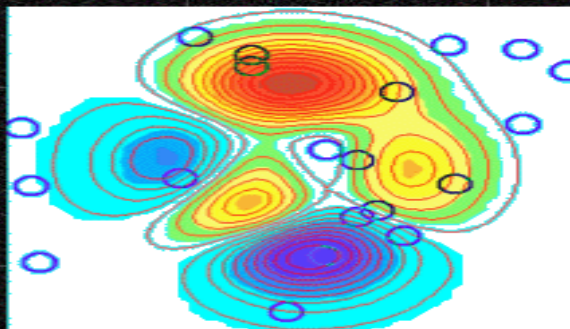
Un ejemplo de Algoritmo Genético

- Example: Find the max. of the “peaks” function
- $z = f(x, y) = 3 \cdot (1-x)^2 \cdot \exp(-(x^2) - (y+1)^2) - 10 \cdot (x/5 - x^3 - y^5) \cdot \exp(-x^2 - y^2) - 1/3 \cdot \exp(-(x+1)^2 - y^2)$.

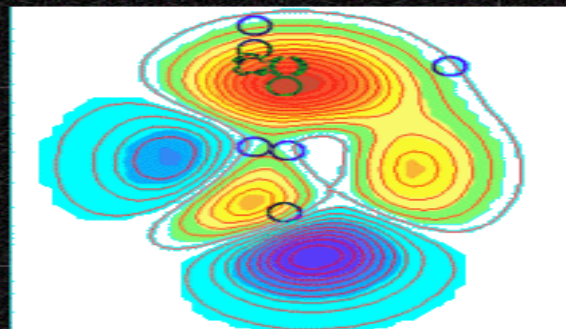
Ejemplo 1



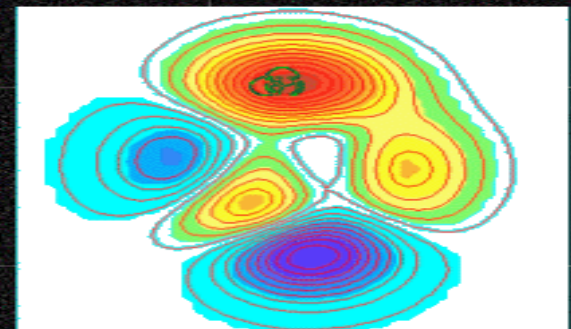
Source: Neuro-Fuzzy and Soft Computing, by J.-S. R. Jang, C.-T. Sun, E. Mizutani



Initial population

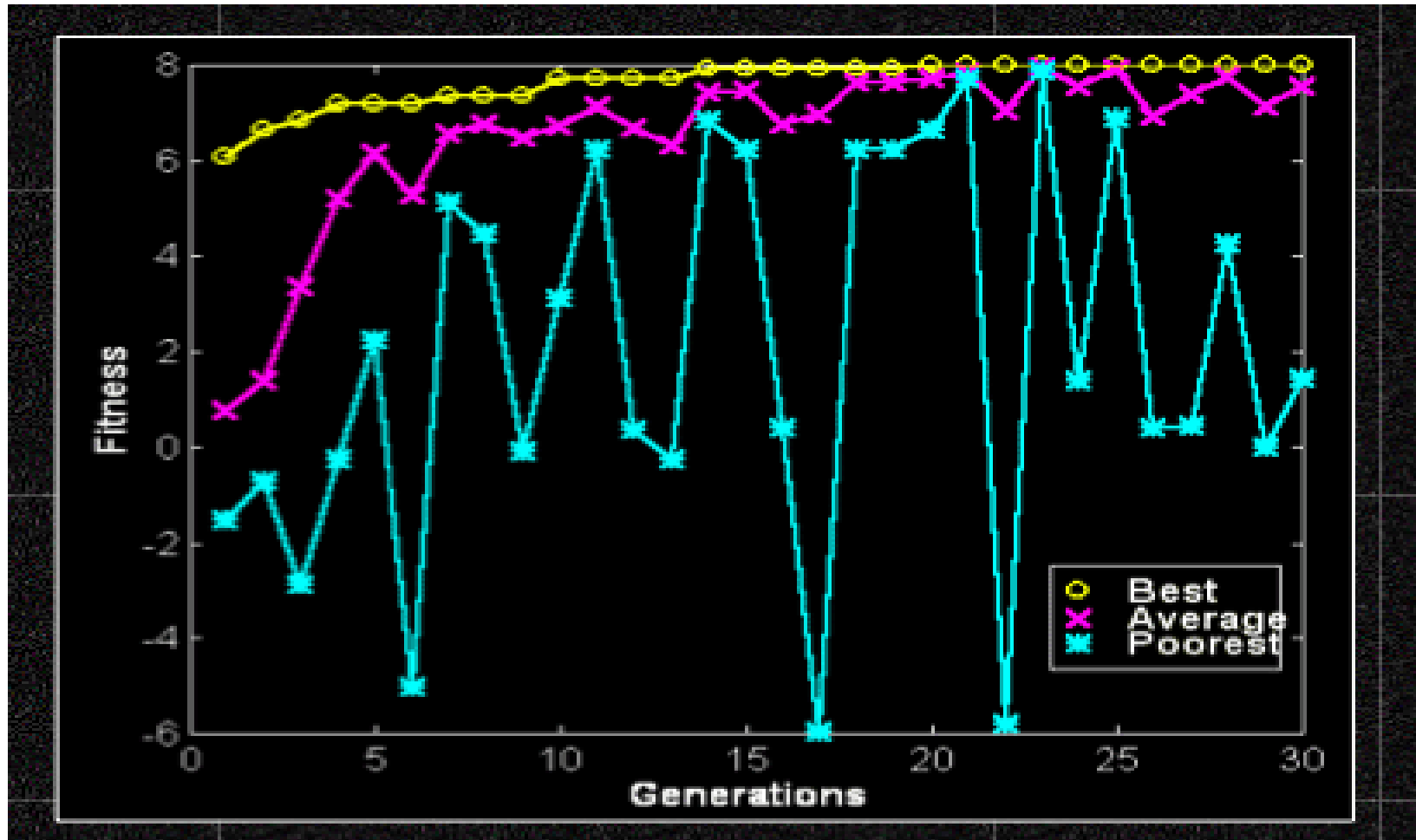


5th generation



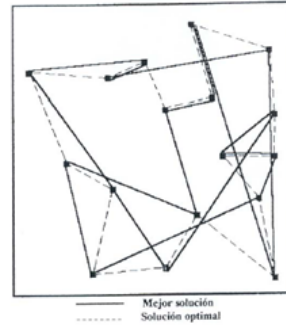
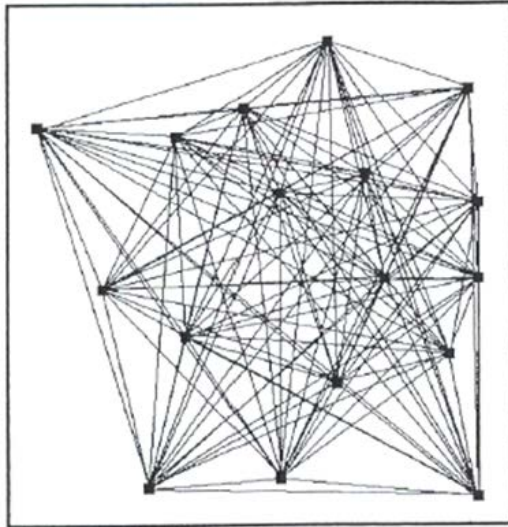
10th generation

La evolución de la adecuación (fitness)

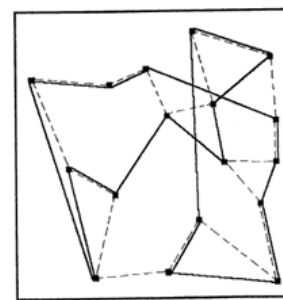


Ejemplo-1: Viajante de Comercio (Ciclo Hamiltoniano)

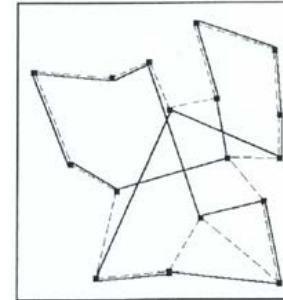
Generaciones de un AG →



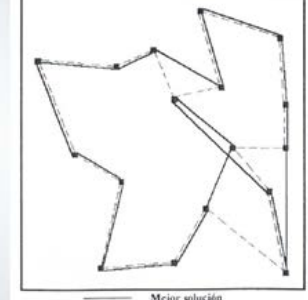
Iteración: 0 Costo: 403,7



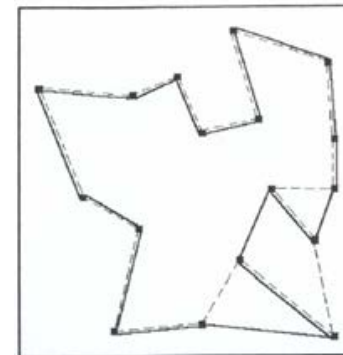
Iteración: 25 Costo: 303,86



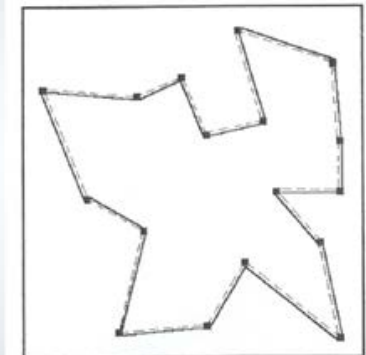
Iteración: 50 Costo: 293,6



Iteración: 100 Costo: 256,55



Iteración: 200 Costo: 231,4



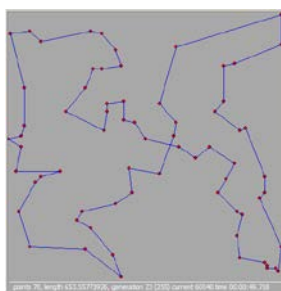
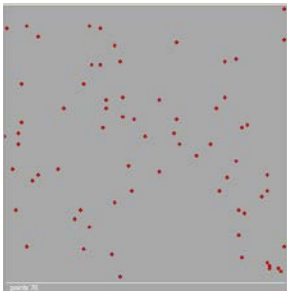
Iteración: 250 Solución
óptima: 226,64

Solución óptima: 226.64

17 ciudades,
 $17! = 3.5568743 \times 10^{14}$ Recorridos posibles
Con 1.000.000.000 recorridos/seg, 4 días de cómputo.

20 ciudades requerirían 77 años,

100 ciudades 10^{135} millones de años



**Aplicación TSP: logística, configuración
canales en sistemas de comunicación, etc,....**

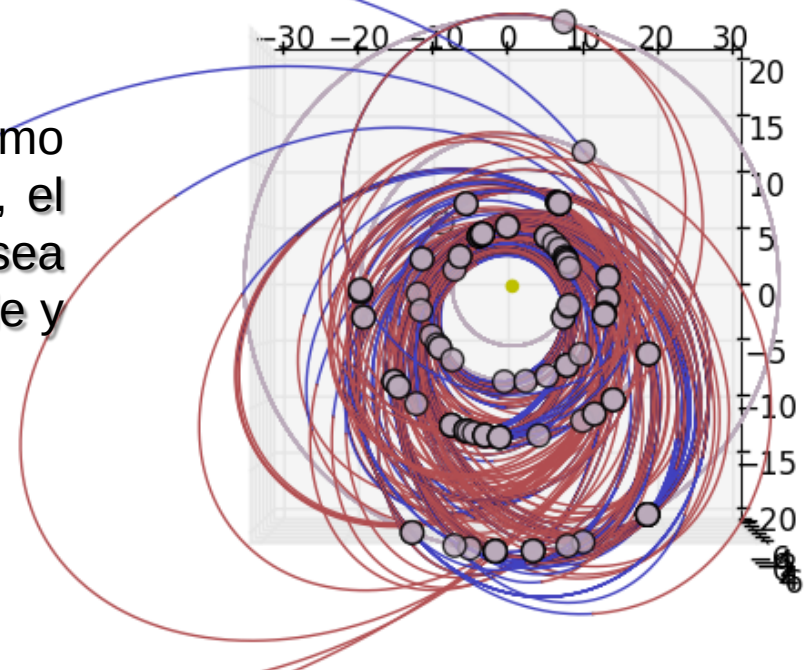
Ejemplo-2

http://www.esa.int/Our_Activities/Technology/ESA_team_s_computer-evolved_trek_round_Jupiter_wins_science_prize

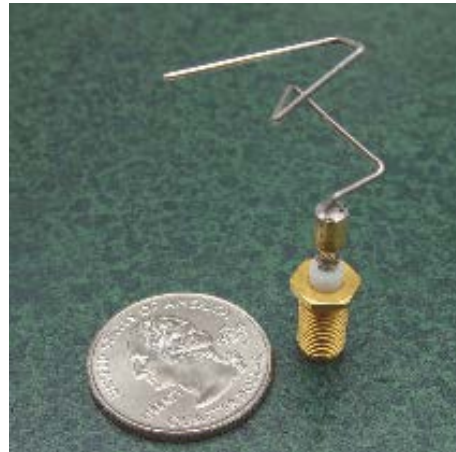
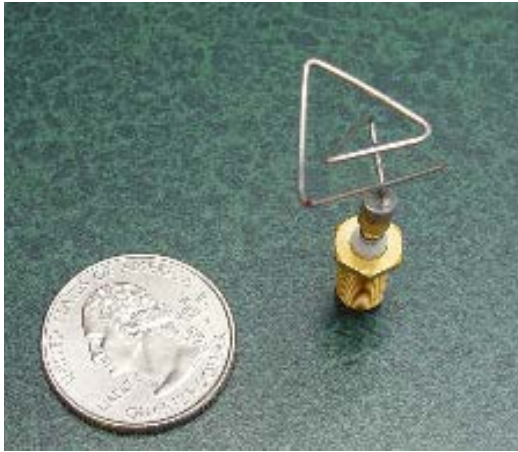
Planificación de la mejor ruta en torno a las principales lunas de Júpiter, logrando una eficacia más allá de los mejores esfuerzos humanos (julio'2013).

Descubiertas por Galileo, Júpiter tiene cuatro grandes lunas (Io, Europa, Calisto y Ganímedes), cada una con sus propias características y girando en complejas e interrelacionadas órbitas en un mar gigante de gas.

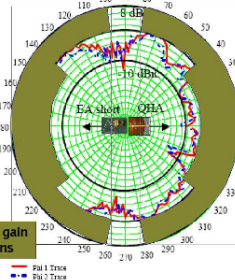
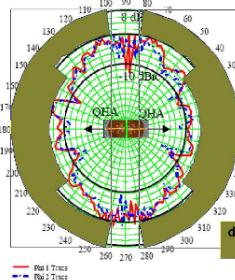
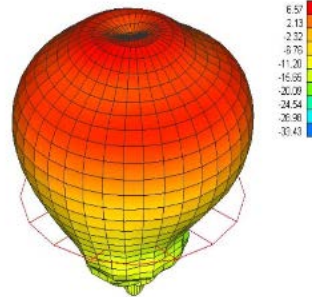
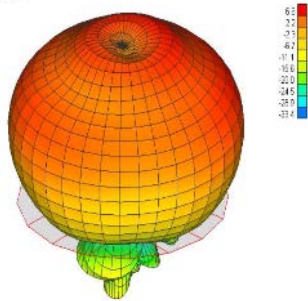
Describiendo los satélites galileanos como balones de fútbol, con 32 caras de cada uno, el reto fue fotografiar tantas caras como sea posible, utilizando el menor combustible posible y minimizando la radiación desde Júpiter.



Ejemplo-3: Diseño de antenas



Evolved antenna



Diseño antenas Banda-X para su aplicación en satélites artificiales (NASA)

Evolved *X-band antenna*:

NASA [ST5](#) spacecraft antenna.

This complicated shape was found by an evolutionary computer design program to create the best radiation pattern.

Computer-automated evolution of an X-band antenna for NASA's Space Technology 5 mission.

Hornby GS, Lohn JD, Linden DS.

Evol Computation no.19(1):1-23

https://en.wikipedia.org/wiki/Evolved_antenna

http://www.nasa.gov/mission_pages/st-5/main/index.html

Genetic algorithms applied to the scheduling of the hubble space telescope *

Jeffrey L. Sponsler (a software systems engineer on the staff)

 [Show more](#)

[https://doi.org/10.1016/S0736-5853\(89\)80015-2](https://doi.org/10.1016/S0736-5853(89)80015-2)

Applications of genetic algorithms from 2022

Comput Math Methods Med. 2018; 2018: 6154025.

Published online 2018 Jan 29. doi: [10.1155/2018/6154025](https://doi.org/10.1155/2018/6154025)

Prediction of Pathological Subjects Using Genetic Algorithms

Murat Sari and Can Tuna

An Improved Genetic Algorithm for Path-Planning of Unmanned Surface Vehicle

Junfeng Xin¹, Jiabao Zhong¹, Fengru Yang¹, Ying Cui¹ and Jinlu Sheng^{2,*}

¹ College of Electromechanical Engineering, Qingdao University of Science and Technology, Qingdao 266061, China; jfxin@163.com (J.X.); zhongjbchn@163.com (J.Z.); seawolf_yfr@163.com (F.Y.); cuiying@qust.edu.cn (Y.C.)

² Transport College, Chongqing Jiaotong University, Chongqing 400074, China

* Correspondence: seawolf_algorithm@163.com

Received: 9 April 2019; Accepted: 5 June 2019; Published: 11 June 2019



Genetic algorithms for computational materials discovery accelerated by machine learning

Paul C. Jennings, Steen Lysgaard, Jens Strabo Hummelshøj, Tejs Vegge & Thomas Bligaard

npj Computational Materials **5**, Article number: 46 (2019) | [Download Citation](#) 

Genetic algorithms for lens design: a review

December 2018

DOI: 10.1007/s12596-018-0497-3

License · [CC BY 4.0](#)

 Kaspar Höschel ·  Vasudevan Lakshminarayanan

Mathematical Problems in Engineering
Volume 2018, Article ID 9270802, 32 pages
<https://doi.org/10.1155/2018/9270802>

Review Article

Recent Research Trends in Genetic Algorithm Based Flexible Job Shop Scheduling Problems

Conceptos Principales de un AG

1. Una **representación** adecuada de las soluciones del problema (individuos). Típicamente binarias. **Cromosomas, genes, genotipo, fenotipo.**
2. Una forma de crear una **población de soluciones inicial** (individuos iniciales). A menudo, aleatoria.
3. Una **función de evaluación** capaz de medir la adecuación de cualquier solución (individuo) al problema. Hace el papel de 'entorno' para los procesos de selección y supervivencia. **Fitness.**
4. Un conjunto de **operadores evolutivos** para combinar las soluciones existentes con el objetivo de obtener nuevas soluciones (nuevos individuos): **selección, cruce, mutación y reemplazo** de individuos. Guían el proceso de la búsqueda.
5. Conjunto de **parámetros de entrada**: tamaño de la población, número de iteraciones (generaciones), probabilidades de selección, etc.

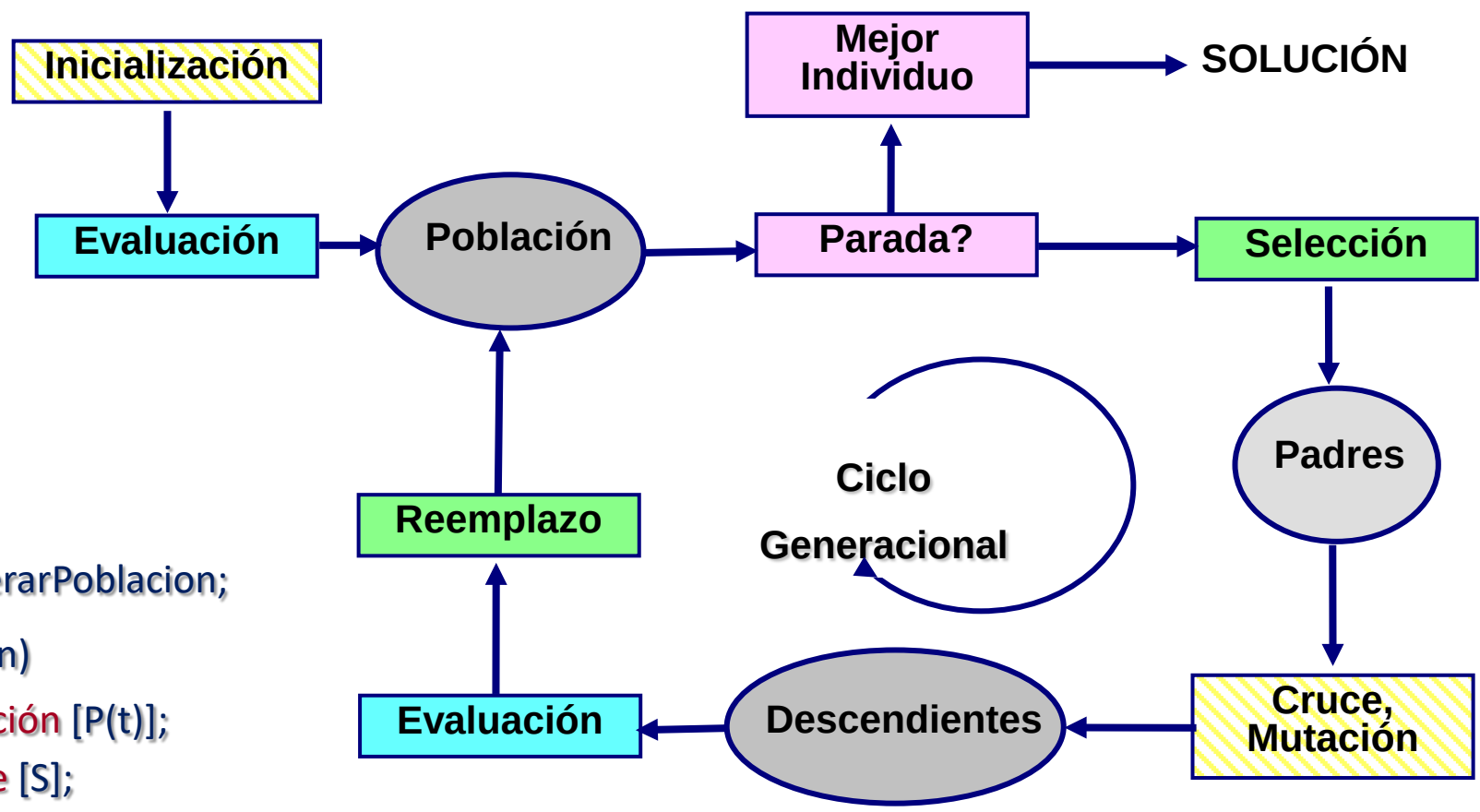
2.- Estructura general de un Algoritmo Genético

0. **Diseño. Codificación y Decodificación de los individuos (soluciones)**
1. **[Población Inicial]:** Generar población aleatoria de n cromosomas $\{x\}$ (*soluciones apropiadas al problema*)
2. **[Aptitud]:** Evaluar la aptitud o *fitness* $f(x)$ de cada cromosoma x en la población.
3. **[Nueva población]** Crear una nueva población repitiendo los pasos siguientes hasta completar la nueva población:
 1. **[Selección]** Seleccionar dos padres cromosomas de la población de acuerdo a su aptitud: Probabilidad de cruce (p_c).
 2. **[Cruce]** Combinar los genes de los dos padres para obtener hijos.
 3. **[Mutación]** Con una probabilidad de mutación (p_{mut}), mutar cada gen de los cromosomas hijo
 4. **[Reemplazo]** Añadir nuevos hijos y determinar nueva población.
4. **[Verificar]** Condición de parada:

[Parada], proporcionar como solución el **cromosoma con mejor valor de aptitud $f(x)$** .

[Ciclo] Ir al paso 2

Procesos de un Algoritmo Genético



```
t ← 0;  
Fin ← False;  
P(0) ← GenerarPoblacion;  
While not(Fin)  
  S ← Selección [P(t)];  
  Q ← Cruce [S];  
  Q' ← Mutación [Q];  
  P(t+1) ← Reemplazo [P(t), S, Q'];  
  Fin ← Convergencia [P(t+1)];  
  t ← t+1  
end_while
```

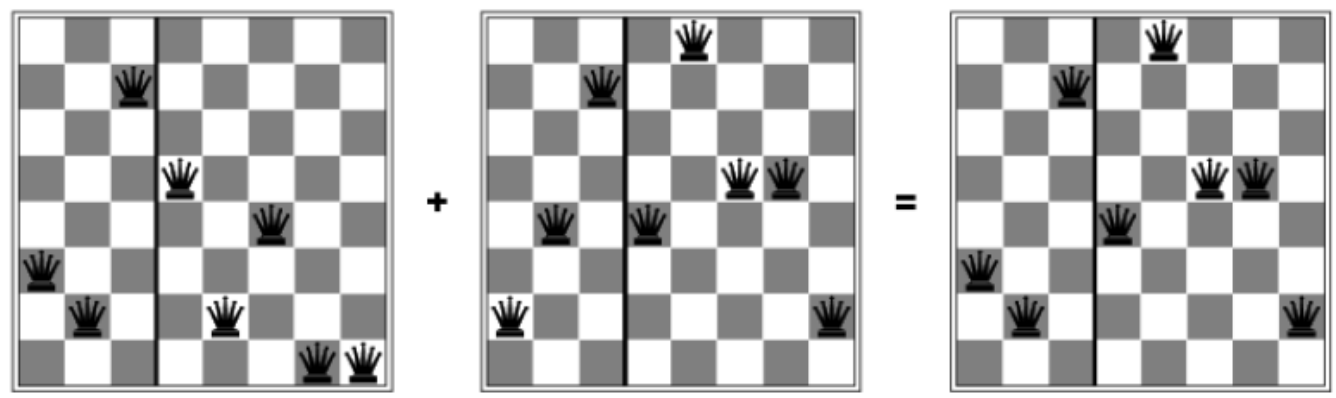
-  Principal coste CPU
-  Darwinismo, estocástico o determinista
-  Operadores Estocásticos
-  Puntos de control

Ejemplo. Procesos del AG para el problema de las n-reinas

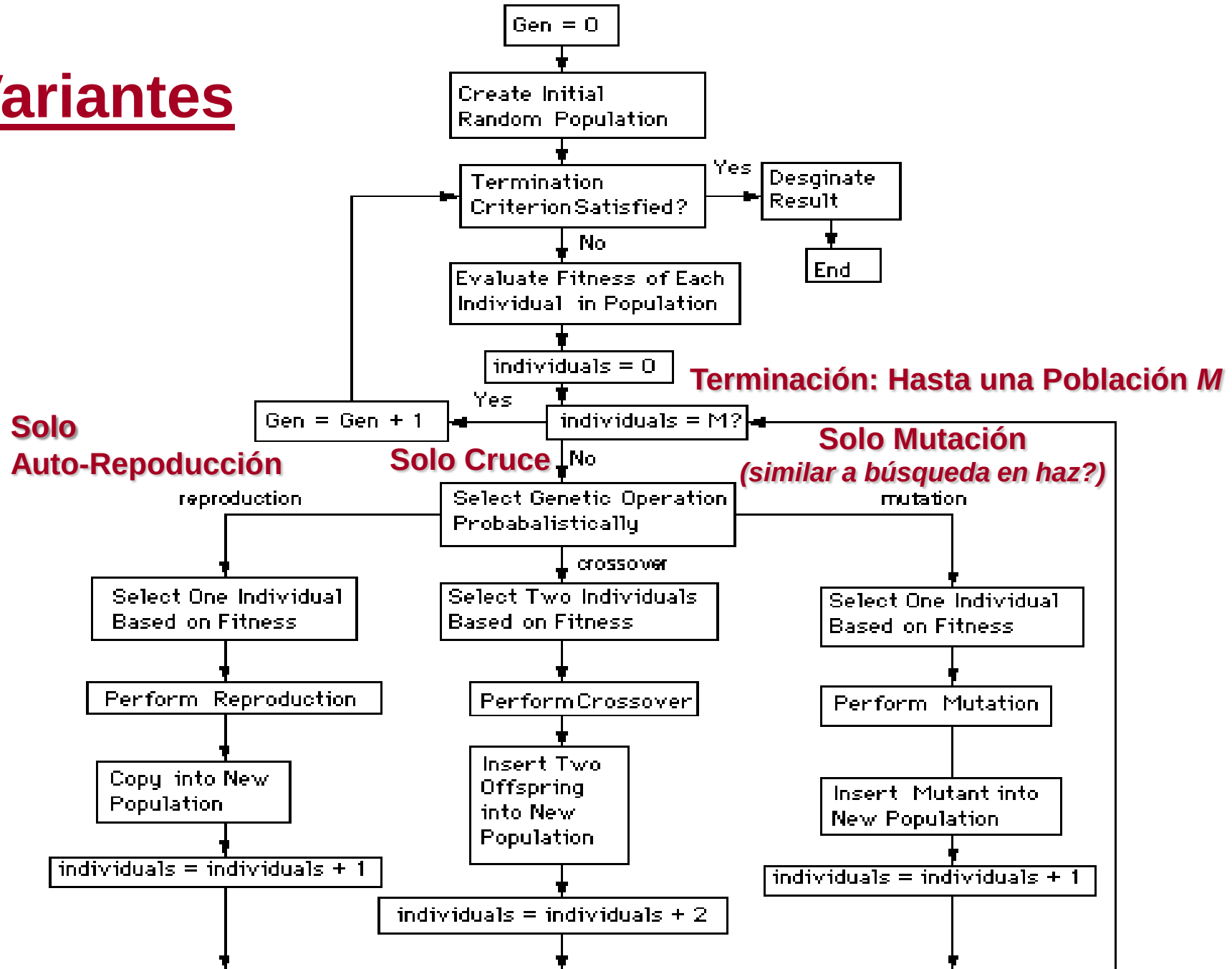


Función de evaluación (8 reinas) = número de pares de reinas no atacadas

28 para una solución



Variantes



Variante: Algoritmos Genéticos Paralelos

Los algoritmos genéticos pueden **implementarse en paralelo** de manera natural.

Hay diversos enfoques.

1) Enfoques de **grano grueso**: Dividen la población en **grupos de individuos** llamados **demes** que típicamente, comparten ciertos rasgos. Es la más típica.

- Cada deme es asignado a un nodo de computación y se aplica un AG específico.
- El cruce entre los demes ocurre con menor frecuencia que en el interior de ellos. La transferencia entre los demes se da por un proceso de migración.
- Una de las ventajas de este enfoque es que reduce el fenómeno de **crowding** que es común en versiones no paralelas de los algoritmos genéticos:

***Crowding:** Individuo muy aptos se reproducen muy frecuentemente, tal que individuos muy parecidos ocupan una fracción importante de la población. Esto reduce la diversidad de la población, haciendo la búsqueda mas lenta.*

2) Paralelización de **grano fino**: Asigna **un procesador a cada miembro** de la población. Las combinaciones se dan solo entre vecinos muy cercanos. Existen y diferentes tipos de vecindad.

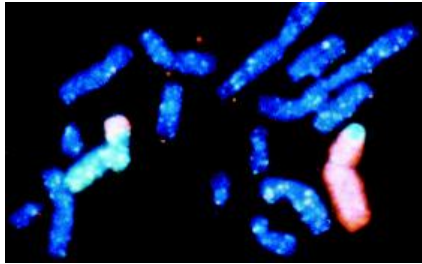
3.- Diseño y procesos de un Algoritmo Genético

1. Diseñar una **representación**: correspondencia entre **Genotipo** y **Fenotipo**
2. Decidir cómo **inicializar** una **población**
3. Diseñar una forma de **evaluar** un individuo (fitness)
4. Decidir cómo **seleccionar** los individuos para ser padres
5. Diseñar un operador de **cruce** adecuado
6. Diseñar un operador de **mutación** adecuado
7. Decidir cómo **reemplazar** a los individuos
8. Decidir la **condición de parada**

Individuo de la Población : Una solución del problema

Evolución Genética

Individuo $\equiv$ Cromosoma: cadena de ADN que se encuentra en el núcleo de las células. Los cromosomas son responsables de la transmisión de información genética.



Gen: sección de ADN que codifica una cierta función bioquímica definida. Supone la unidad de herencia.

Genotipo: Información genética de un organismo.

Fenotipo: cualquier característica observable (externa) de un organismo (dependiente del genotipo más la influencia del medio).

Algoritmo Genético

Individuo $\equiv$ Cromosoma: estructura de datos que contiene una **secuencia de parámetros** que permite definir (o formar) una **solución** al problema.

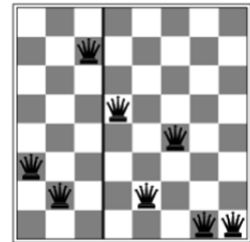
Un cromosoma representa el **Genotipo** (representación interna de la solución):

Genotipo:

3	2	7	5	2	4	1	1
---	---	---	---	---	---	---	---

Fenotipo:

representación externa de la solución



Gen: elementos del cromosoma que codifica el valor de un (o varios) parámetro, propiedad o aspecto del individuo.

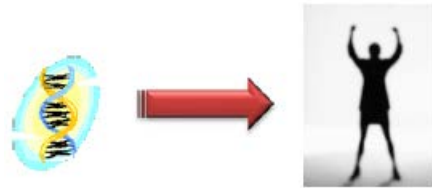
Alelo: Valor (Valores) de un gen

Es muy importante el diseño de los individuos (soluciones) !!!

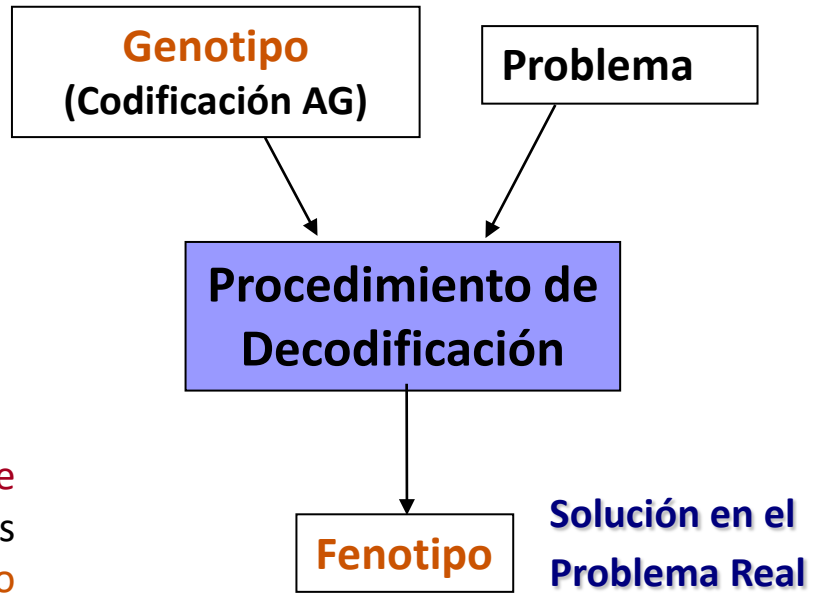
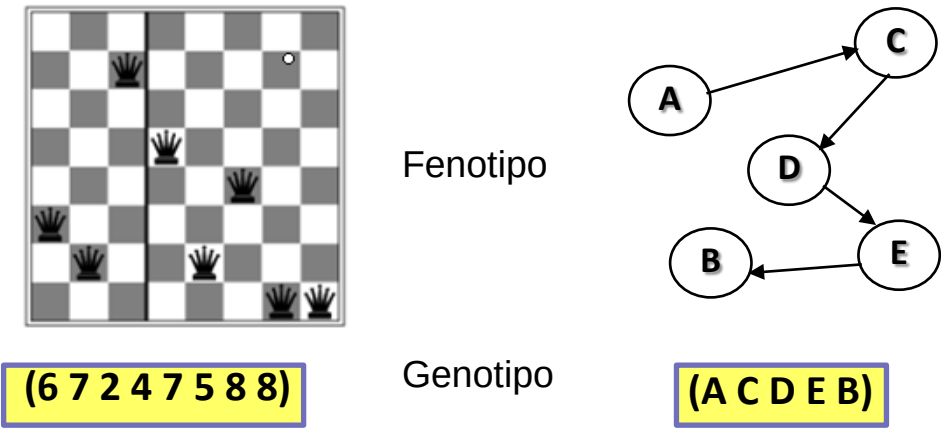
Codificación de Soluciones en un AG: Solución ⇔ Cromosoma

- La codificación de los **individuos** (o **solución**) como un cromosoma (**genotipo**) es fundamental para la aplicación de los AG y es lo primero que hay que diseñar.
- La **codificación** debe poder representar todos los **fenotipos** (soluciones) posibles.

Correspondencia entre Genotipo y Fenotipo



Algunas veces, la obtención del **Fenotipo** a partir del **Genotipo** es un proceso obvio.



En otras ocasiones, el **Genotipo** puede ser un **conjunto de parámetros** para algún algoritmo, que trabaja sobre los datos de un problema para obtener el **Fenotipo** correspondiente.

ESTRUCTURA GENERAL DE UN A.G. SIMPLE

1. **[Población Inicial]:** Generar población aleatoria de n cromosomas $\{x\}$ (*soluciones apropiadas al problema*)
2. **[Aptitud]:** Evaluar la aptitud o *fitness* $f(x)$ de cada cromosoma x en la población.
3. **[Nueva población]** Crear una nueva población repitiendo los pasos siguientes hasta completar la nueva población:
 1. **[Selección]** Seleccionar dos padres cromosomas de la población de acuerdo a su aptitud: Probabilidad de cruce (p_c).
 2. **[Cruce]** Combinar los genes de los dos padres para obtener hijos.
 3. **[Mutación]** Con una probabilidad de mutación (p_{mut}), mutar cada gen de los cromosomas hijo
 4. **[Reemplazo]** Añadir nuevos hijos y determinar nueva población.
4. **[Verificar]** Condición de parada $\rightarrow$ **STOP**, y proporcionar como solución el **cromosoma con mejor valor de aptitud $f(x)$.**
5. **[Ciclo]** Ir al paso 2

Población Inicial

- Método inicialización:

a) Se puede inicializar uniformemente de forma aleatoria en el espacio de búsqueda:

- **Representación binaria:** 0 ó 1 con probabilidad 0.5
- **Representación real:** uniforme sobre un intervalo dado (para valores acotados)
- **Representación de orden:** Soluciones (individuos) factibles aleatorias

b) Elegir la población inicial a partir de los resultados de una heurística previa

- Posible pérdida de la diversidad genética
- Posible preferencia no recuperable

- Tamaño de la población: Típicamente dependiente del problema.

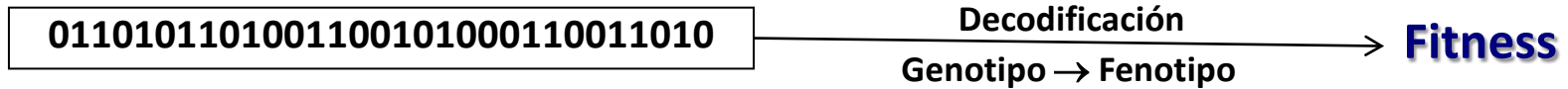
- *Hay propuestas teóricas que proponen que el tamaño óptimo para individuos de longitud L , con codificación binaria, debe crecer exponencialmente con L .*
- *Otros resultados empíricos sugieren un tamaño de **población entre L y $2 \cdot L$** .*

ESTRUCTURA GENERAL DE UN A.G. SIMPLE

1. **[Población Inicial]:** Generar población aleatoria de n cromosomas $\{x\}$ (*soluciones apropiadas al problema*)
2. **[Aptitud]:** Evaluar la aptitud o *fitness* $f(x)$ de cada cromosoma x en la población.
3. **[Nueva población]** Crear una nueva población repitiendo los pasos siguientes hasta completar la nueva población:
 1. **[Selección]** Seleccionar dos padres cromosomas de la población de acuerdo a su aptitud: Probabilidad de cruce (p_c).
 2. **[Cruce]** Combinar los genes de los dos padres para obtener hijos.
 3. **[Mutación]** Con una probabilidad de mutación (p_{mut}), mutar cada gen de los cromosomas hijo
 4. **[Reemplazo]** Añadir nuevos hijos y determinar nueva población.
4. **[Verificar]** Condición de parada $\rightarrow$ **STOP**, y proporcionar como solución el cromosoma con mejor valor de aptitud $f(x)$.
5. **[Ciclo]** Ir al paso 2

Aptitud

Evaluación de un individuo: **Función de Fitness o Aptitud**



- **Evalúa la calidad** de la solución o cromosoma
- Este es el **paso más costoso** en las aplicaciones reales
 - Requiere obtener el fenotipo (solución) del genotipo.
 - Puede ser una subrutina, una caja negra, o cualquier proceso externo
 - Se pueden utilizar **funciones aproximadas** para reducir el coste de evaluación
- **Manejo de restricciones:** *¿Qué hacer si un individuo incumple alguna restricción del problema?*
 - **Si es un problema de factibilidad:** Típicamente se penaliza el fitness de la solución (*ejemplo: 8-reinas*)
 - **Si no es un problema de factibilidad:** Típicamente, se repara la solución (*ejemplo: viajante*)

ESTRUCTURA GENERAL DE UN A.G. SIMPLE

1. **[Población Inicial]**: Generar población aleatoria de n cromosomas $\{x\}$ (*soluciones apropiadas al problema*)
2. **[Aptitud]**: Evaluar la aptitud o *fitness* $f(x)$ de cada cromosoma x en la población.
3. **[Nueva población]** Crear una nueva población repitiendo los pasos siguientes hasta completar la nueva población:
 1. **[Selección]** Seleccionar dos padres cromosomas de la población de acuerdo a su aptitud: Probabilidad de cruce (p_c).
 2. **[Cruce]** Combinar los genes de los dos padres para obtener hijos.
 3. **[Mutación]** Con una probabilidad de mutación (p_{mut}), mutar cada gen de los cromosomas hijo
 4. **[Reemplazo]** Añadir nuevos hijos y determinar nueva población.
4. **[Verificar]** Condición de parada $\rightarrow$ **STOP**, y proporcionar como solución el cromosoma con mejor valor de aptitud $f(x)$.
5. **[Ciclo]** Ir al paso 2

Estrategia de Selección

Selección de los elementos que se reproducen en cada ciclo generacional

- Debemos garantizar que los **mejores individuos** tengan una **mayor posibilidad de ser padres** (reproducirse) frente a los individuos menos buenos.
Mientras más apto sea un organismo (valor de aptitud más alto), más veces será seleccionado para reproducirse.
- Debemos ser cuidadosos para **dar una posibilidad de reproducirse a los individuos menos buenos**. Éstos pueden incluir material genético útil en el proceso de reproducción.
- La **PRESIÓN SELECTIVA** afecta el proceso de búsqueda
 - **Selección muy fuerte**: organismos de *alto fitness* que son subóptimos pueden dominar la población, disminuyendo la diversidad necesaria para progresar.
 - **Selección muy débil**: evolución muy lenta.
- El **numero de padres** que se seleccionan de la población (para cruzarse) es un parámetro de diseño (ver Reemplazo: *generacional, estacionario, gap generacional*).

Métodos de Selección

- **Selección elitista:** elección directa de los miembros más aptos de cada generación.
- **Selección proporcional:** los individuos más aptos tienen **más probabilidad** de ser seleccionados, pero no la certeza. Los peores individuos también pueden ser elegidos.
 - **Selección por rueda de ruleta:** Basado en el fitness del individuo y la media de grupo..
- **Selección jerárquica:** los individuos atraviesan **múltiples rondas de selección** en cada generación. Las evaluaciones de los primeros niveles son más rápidas y menos discriminatorias (hay muchos individuos), mientras que los que sobreviven hasta niveles más altos son evaluados más rigurosamente.
 - **Selección por torneo:** grupos de individuos compiten entre ellos, sucesivamente.
- **Selección por rango:** más que por el valor del fitness, la selección se basa en el rango (posición) de cada individuo. A cada individuo se le asigna un **ranking numérico** basado en su aptitud, y la selección se basa en este ranking, en lugar de las diferencias absolutas en aptitud.
- **Selección generacional:** La descendencia de una generación es la siguiente generación de padres. No se conservan individuos entre generaciones.
- **Selección multiobjetivo** (basado en frentes de Pareto, diversidad de soluciones, etc.): NSGA-II: (Non-dominated Sorting Genetic Algorithm II, Deb'2002)

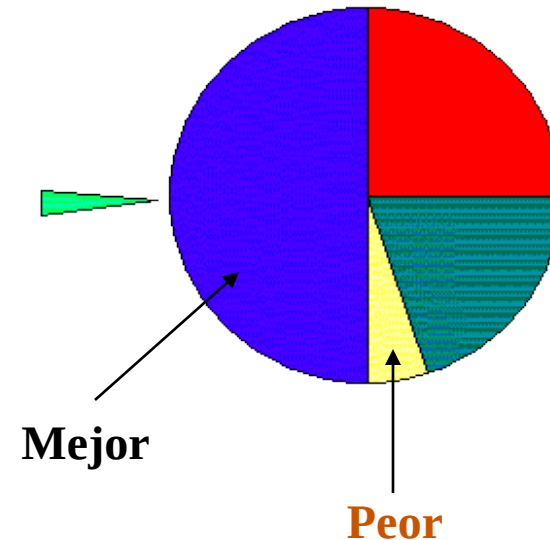
Selección Proporcional:

A cada individuo_i con fitness f_i , se le asigna una probabilidad de ser elegido basada en su f_i .

Selección por Rueda de Ruleta

A cada individuo de la población se le asigna una parte proporcional a su fitness f_i en una ruleta, tal que la suma de todos los porcentajes sea la unidad.

$$p_i = \frac{f_i}{\sum_{j=1}^N f_j}$$



Los mejores individuos (más adaptados) tienen:

- Más posibilidades de ser elegidos
- Más espacio en la ruleta

Ejemplo Selección por Rueda de Ruleta

SUPERINDIVIDUO

Es un cromosoma que tiene una **adaptación** muy alta en comparación con el resto de los cromosomas de la población.

Individuo

Adaptación

Prob. De Selección

Nº

f_i

$p_i = f_i / \sum f_i$

1

200

0.797

2

30

0.119

3

10

0.040

4

8

0.032

5

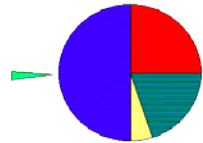
2

0.008

6

1

0.004

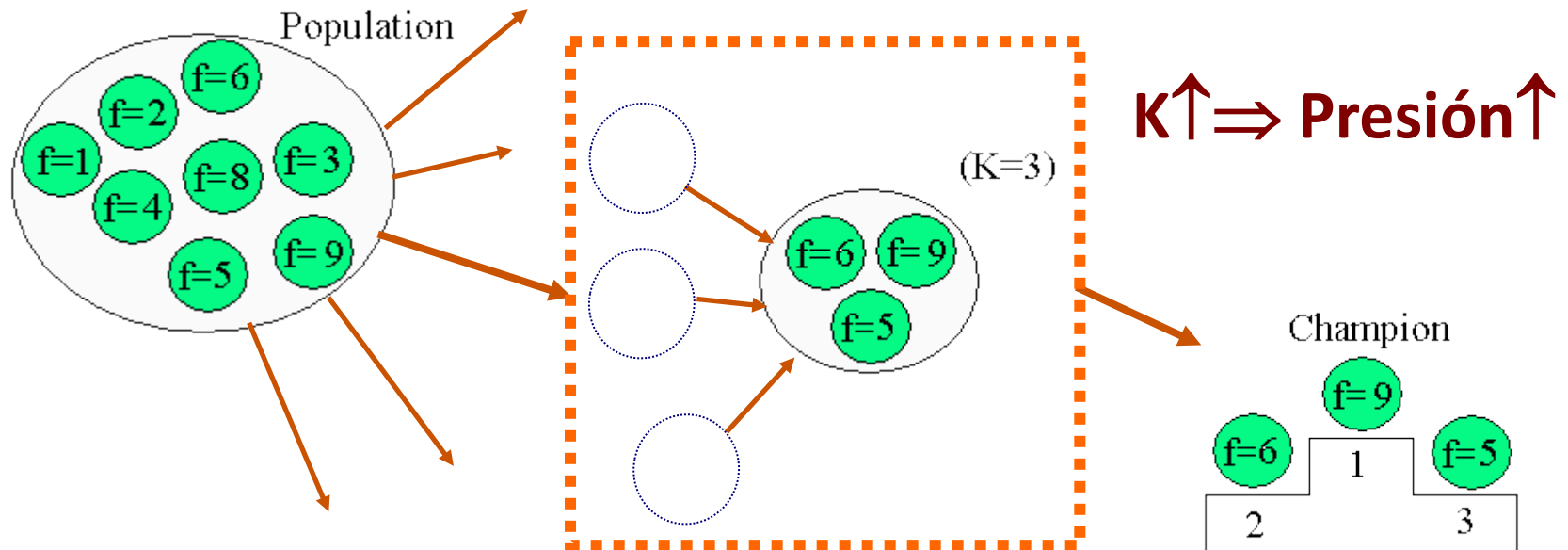


Inconvenientes:

- Peligro de **Convergencia Prematura** (*premature convergence*) a un sub-óptimo debido a que **individuos muy aptos dominan la población completa muy rápidamente**
- Baja **Presión Selectiva** (*selection pressure*) cuando los valores de aptitud son muy cercanos entre sí. Se produce **estancamiento en el proceso de búsqueda**
- **Modificación (windowing)**: Aplicar $f'(i) = f(i) - \beta(i)$, donde $\beta(i)$: peor fitness en cada generación

Selección por Torneo:

- Se establecen grupos de k individuos (k es el parámetro del tamaño del torneo)
- Se selecciona el mejor de cada grupo: método elitista o método proporcional
- Se realizan sucesivos torneos, hasta que se seleccionan el número deseado de padres.



Variando el número (K) de individuos que participan en cada torneo se modifica la presión de selección.

- Cuando participan *muchos individuos en cada torneo*, la presión de selección es *elevada* y los peores individuos apenas tienen oportunidades de reproducción.
- Cuando el tamaño del torneo es reducido, la presión de selección disminuye y los peores individuos tienen más oportunidades de ser seleccionados.

Selección por Rango:

- La población se ordena de acuerdo a sus valores de la función fitness. La posición de cada individuo en la lista ordenada representa su **rango: $r(x)$** .
- La **probabilidad** asignada a cada individuo depende de su **posición en el rango** y no de su valor objetivo
 - Solventa los problemas de la selección proporcional respecto a: **estancamiento** (cuando la presión selectiva es muy baja) o **convergencia prematura** (presión alta, súper-individuo)
 - El rango $r(x)$ se usa para ponderar los individuos, aplicando una función $h(x)$.

$$h(x) = \max - (\max - \min) * \frac{(r(x) - 1)}{n - 1}$$

$$P(x) = \frac{h(x)}{n}$$

Donde:

max+min=2

max ≥ min : max ∈ [1,2], min ∈ [0,1],

(max recomendado = 1.1)

↑max ⇒ Mayor Separación P(x)

max=1, min=1: Igual P(x) para todos

Ejemplo (max = 1.1 → min = 0.9):

<i>Fitness(x)</i>	<i>Ranking(x)</i>	<i>h(x)</i>	<i>P(x)</i>
5	2	1	0.33
2	3	0.9	0.30
19	1	1.1	0.37
Σ		3	1

**Aplicar Ruleta
u otra técnica
Proporcional**

<i>P(x)</i> <i>max=1</i>	<i>P(x)</i> <i>max=2</i>	<i>Propor</i> <i>cional</i>
0.33	0.33	0.19
0.33	0.00	0.08
0.33	0.67	0.73

Otros esquemas:

Ranking Lineal, Ranking exponencial, etc.

ESTRUCTURA GENERAL DE UN A.G. SIMPLE

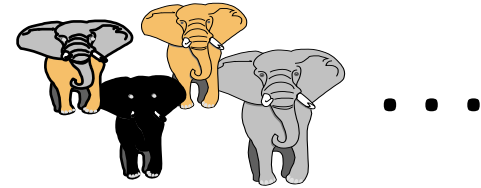
1. **[Población Inicial]:** Generar población aleatoria de n cromosomas $\{x\}$ (*soluciones apropiadas al problema*)
2. **[Aptitud]:** Evaluar la aptitud o *fitness* $f(x)$ de cada cromosoma x en la población.
3. **[Nueva población]** Crear una nueva población repitiendo los pasos siguientes hasta completar la nueva población:
 1. **[Selección]** Seleccionar dos padres cromosomas de la población de acuerdo a su aptitud: Probabilidad de cruce (p_c).
 2. **[Cruce]** Combinar los genes de los dos padres para obtener hijos.
 3. **[Mutación]** Con una probabilidad de mutación (p_{mut}), mutar cada gen de los cromosomas hijo
 4. **[Reemplazo]** Añadir nuevos hijos y determinar nueva población.
4. **[Verificar]** Condición de parada $\rightarrow$ **STOP**, y proporcionar como solución el cromosoma con mejor valor de aptitud $f(x)$.
5. **[Ciclo]** Ir al paso 2

Cruce

Naturaleza: proceso complejo que ocurre entre cromosomas homólogos. Los cromosomas se alinean, luego se fraccionan en ciertas partes y posteriormente intercambian fragmentos entre sí.

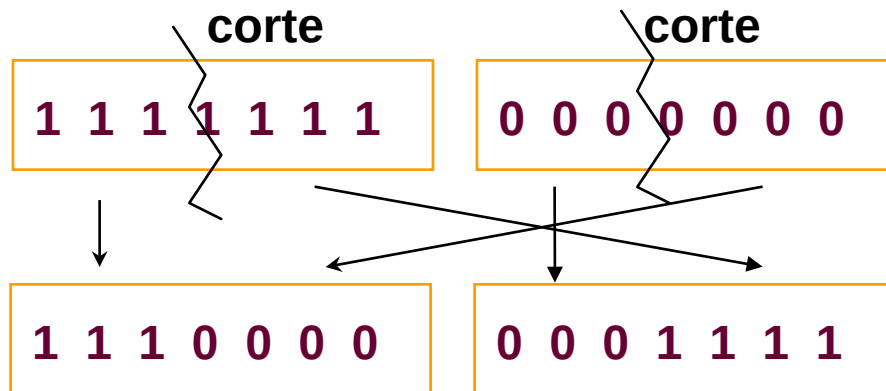
Computación Evolutiva: se simula la recombinación intercambiando segmentos de cadenas lineales de longitud fija (los cromosomas).

Población:

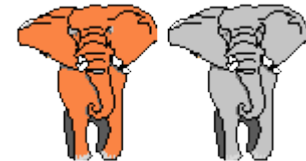


Cruce de n-puntos

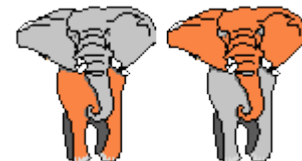
Cada cromosoma es cortado en **n puntos** y las porciones de cada cromosoma se recombinan (Ejemplo para **n=1**)



padres



hijos



Cruce de 2 puntos:

- Se seleccionan 2 puntos, y se intercambian los grupos de bits entre ellos

Padre1 = 10|1010|1010

Padre2 = 11|1000|1110

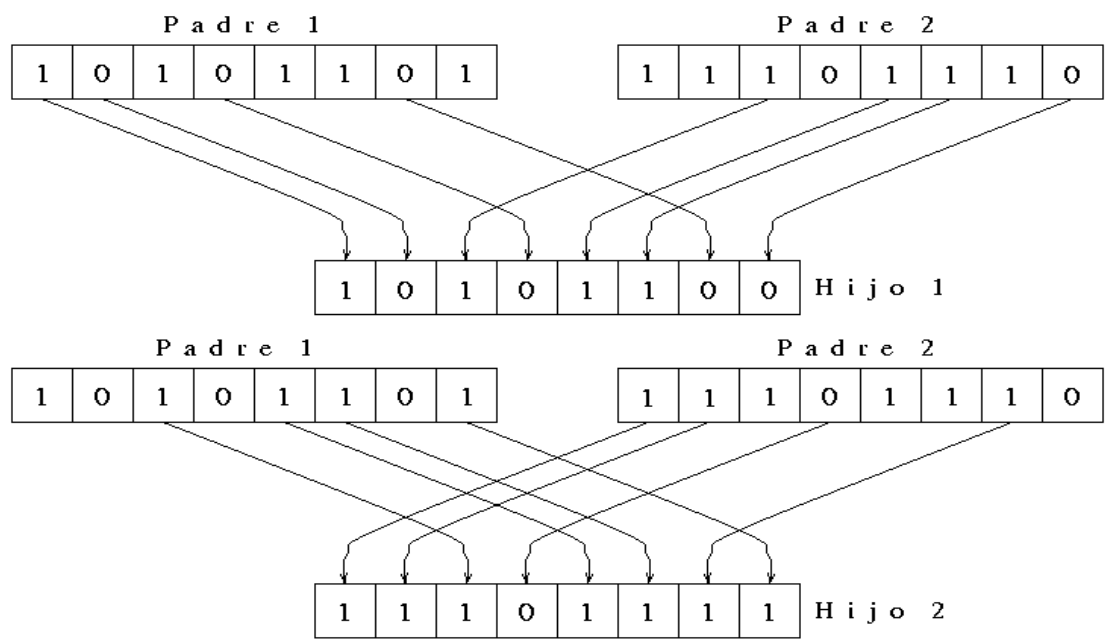


Hijo1 = 11|1010|1110

Hijo2 = 10|1000|1010

Cruce Uniforme:

- Se intercambian bits. Para cada bit del hijo se selecciona aleatoriamente de que padre viene



Recomendaciones Puntos de Cruce

- El cruce de n -puntos es un método más típico, si no existen restricciones adicionales.
- El cruce de un punto ($n=1$) presenta desventajas (poca flexibilidad, etc.).
- No existe consenso en torno al uso de valores para $n \geq 3$.
- Incrementar el valor de n se asocia con un mayor efecto destructivo del cruce.
- En general, la **recombinación de 2 puntos** y la **recombinación uniforme** son generalmente preferibles a la recombinación de 1 punto.
- Probablemente, la elección más acertada **dependa del problema a resolver**.

Existen otros diversos métodos de cruce.

Ejemplo: Cruce de Permutación (alelos acotados):

- Se selecciona un conjunto de genes consecutivos del Padre-1.
- Se trasladan al Hijo-1 y se marcan los genes en el Padre-2
- Comenzando en el lado derecho del cromosoma, trasladar los genes del padre-2 (que no estén ya) al hijo-1.
- Para el Hijo-2, repetir el proceso, invirtiendo previamente los padres.

Padre-1: 8 4 7 3 6 2 5 1 9 0

Padre-2: 0 ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ ~~6~~ 7 8 9

Hijo-1: 0 4 7 3 6 2 5 1 8 9

Padre-1: 0 9 1 5 2 6 3 7 4 8

Padre-2: 9 8 ~~7~~ ~~6~~ ~~5~~ 4 ~~3~~ ~~2~~ 1 0

Hijo-1: 9 8 4 5 2 6 3 7 1 0

ESTRUCTURA GENERAL DE UN A.G. SIMPLE

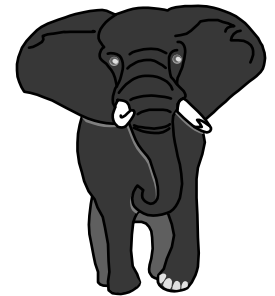
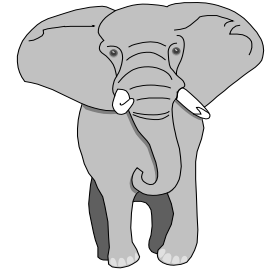
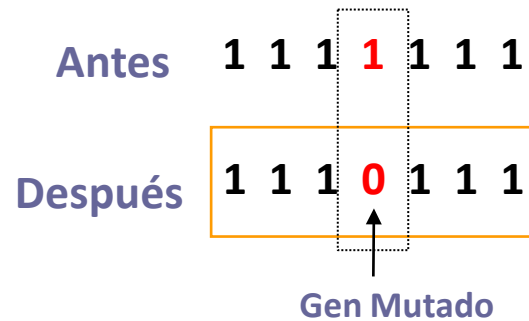
1. **[Población Inicial]:** Generar población aleatoria de n cromosomas $\{x\}$ (*soluciones apropiadas al problema*)
2. **[Aptitud]:** Evaluar la aptitud o *fitness* $f(x)$ de cada cromosoma x en la población.
3. **[Nueva población]** Crear una nueva población repitiendo los pasos siguientes hasta completar la nueva población:
 1. **[Selección]** Seleccionar dos padres cromosomas de la población de acuerdo a su aptitud: Probabilidad de cruce (p_c).
 2. **[Cruce]** Combinar los genes de los dos padres para obtener hijos.
 3. **[Mutación]** Con una probabilidad de mutación (p_{mut}), mutar cada gen de los cromosomas hijo
 4. **[Reemplazo]** Añadir nuevos hijos y determinar nueva población.
4. **[Verificar]** Condición de parada $\rightarrow$ **STOP**, y proporcionar como solución el cromosoma con mejor valor de aptitud $f(x)$.
5. **[Ciclo]** Ir al paso 2

Mutación: Introduce diversidad en la búsqueda

Mutación bit-a-bit

La Mutación se aplica con probabilidad p_{mut} para cada gen de cada cromosoma.

La probabilidad de mutar (P_{mut}) en general es baja, aprox. en el rango $\{0.001, 0.05\}$.



Mutación por Inserción:

Se selecciona un valor en forma aleatoria y se le inserta en una posición arbitraria.

P = 9 4 2 1 5 7 6 10 3 8

P' = 9 6 4 2 1 5 7 10 3 8

Ejemplo:

elegimos la posición 7 y movemos ese valor a la posición 2

Mutación por Intercambio Recíproco:

Se seleccionan dos puntos al azar y se intercambian estos valores de posición.

P = 9 4 2 1 5 7 6 10 3 8

^ ←-----> ^

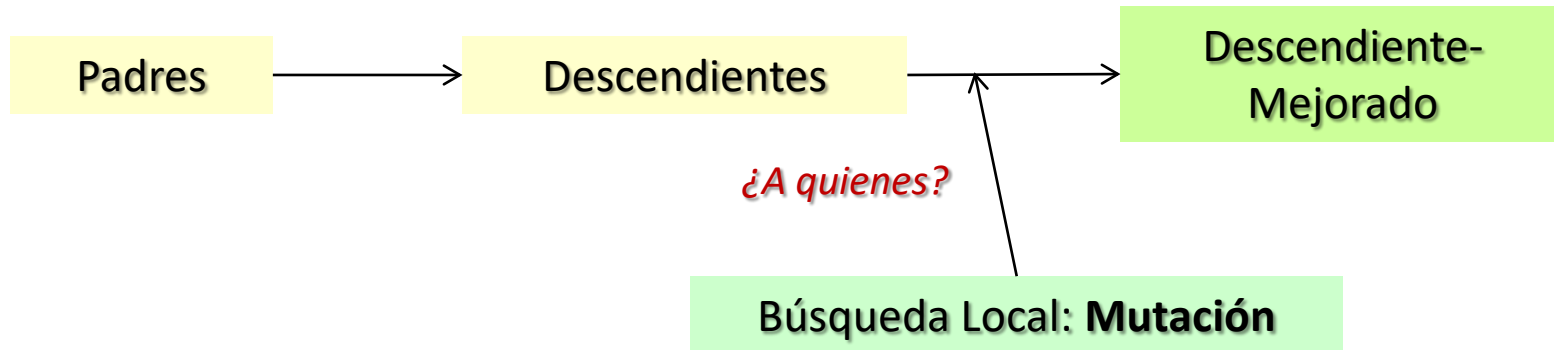
P' = 9 10 2 1 5 7 6 4 3 8

Otros métodos de mutación:

- Inversión (de un grupo de genes),
- Scramble (reordenación aleatoria de un subconjunto) , etc.

Mutación: ¿Búsqueda Local?

- Estudios recientes **dan** importancia a la mutación : un algoritmo basado en mutación- selección, puede ser un muy poderoso algoritmo de búsqueda.
- En la mutación se puede producir un proceso de Búsqueda Local, Adaptación, o Aprendizaje sobre el individuo.
- Este proceso más complejo da lugar a los **AG Híbridos** o, posteriormente, a los Algoritmos Meméticos.



Ejemplo de Algoritmo Genético (Problema del Viajero)

Encontrar una ruta a través de una serie de ciudades que visitando todas las ciudades se minimice la distancia recorrida.

Genotipo: *Lista de las ciudades a visitar*

1) London 3) Dunedin 5) Beijing 7) Tokyo
2) Venice 4) Singapore 6) Phoenix 8) Victoria

Cromosomas (*orden en el que se recorren las ciudades*)

CityList1 (3 5 7 2 1 6 4 8)

CityList2 (2 5 7 6 8 3 1 4)

Cruce (2-puntos):

Cromosoma-i (3 5 **7 2 1 6** 4 8)

Cromosoma-j (**5 8** 7 6 8 3 **1 4**)

Hijo k (**5 8 7 2 1 6** **3** 4) ←

reparación

Los individuos válidos deben visitar todas las ciudades:

- La operación de cruce debe obtener individuos válidos,
- Los individuos no válidos son eliminados, reparados, o se les asocia un valor de fitness = $-\infty$

Mutación (intercambio):

Antes: (5 8 **7** 2 1 **6** 3 4)

Después: (5 8 **6** 2 1 **7** 3 4)

ESTRUCTURA GENERAL DE UN A.G. SIMPLE

1. **[Población Inicial]:** Generar población aleatoria de n cromosomas $\{x\}$ (*soluciones apropiadas al problema*)
2. **[Aptitud]:** Evaluar la aptitud o *fitness* $f(x)$ de cada cromosoma x en la población.
3. **[Nueva población]** Crear una nueva población repitiendo los pasos siguientes hasta completar la nueva población:
 1. **[Selección]** Seleccionar dos padres cromosomas de la población de acuerdo a su aptitud: Probabilidad de cruce (p_c).
 2. **[Cruce]** Combinar los genes de los dos padres para obtener hijos.
 3. **[Mutación]** Con una probabilidad de mutación (p_{mut}), mutar cada gen de los cromosomas hijo
 4. **[Reemplazo]** Añadir nuevos hijos y determinar nueva población.
4. **[Verificar]** Condición de parada $\rightarrow$ **STOP**, y proporcionar como solución el cromosoma con mejor valor de aptitud $f(x)$.
5. **[Ciclo]** Ir al paso 2

Reemplazo de la Población (*cambio generacional*)

Establece el criterio de los supervivientes en cada iteración / sustitución generacional.

□ **Generacional:**

- **Toda la población es reemplazada en cada generación:** cada individuo sobrevive solo una generación y todos los hijos reemplazan a todos los padres.

□ **Estado – Estacionario (Steady–State AG). *Típico.***

- **En cada generación se generan pocos hijos (1, 2, 3,...) y solo pocos individuos (los peores) de la población son reemplazados.**

□ **Gap Generacional (*Generacional Elitista*)**

- **Entre los dos esquemas anteriores: se define un porcentaje (gap) de los individuos que serán reemplazados cada generación, por evaluación del fitness:**

Proporción = 1 (para generacional), $= 1/|población|$ para SSAG

□ **Reemplazo catastrofista:** Cada cierto tiempo se eliminan una gran cantidad de individuos:

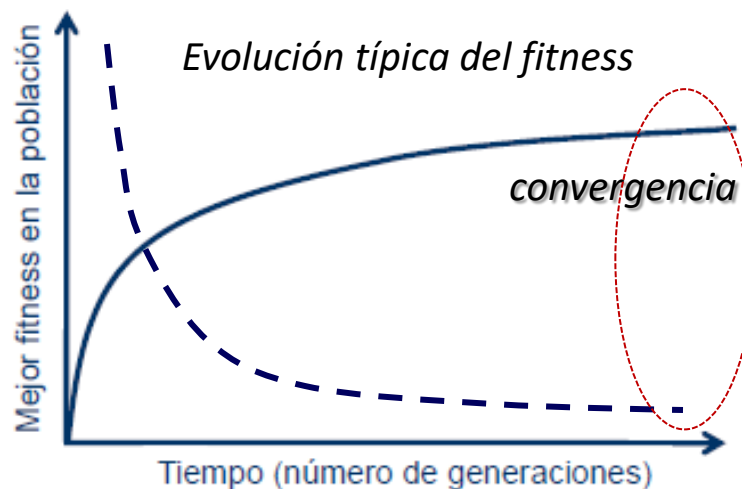
- **Empaquetado:** Solo se mantiene un individuo de los que tienen igual fitness.
- **Juicio final:** Se destruye toda la población excepto el más avanzado. El resto de población se genera aleatoriamente.

ESTRUCTURA GENERAL DE UN A.G. SIMPLE

1. **[Población Inicial]**: Generar población aleatoria de n cromosomas $\{x\}$ (*soluciones apropiadas al problema*)
2. **[Aptitud]**: Evaluar la aptitud o *fitness* $f(x)$ de cada cromosoma x en la población.
3. **[Nueva población]** Crear una nueva población repitiendo los pasos siguientes hasta completar la nueva población:
 1. **[Selección]** Seleccionar dos padres cromosomas de la población de acuerdo a su aptitud: Probabilidad de cruce (p_c).
 2. **[Cruce]** Combinar los genes de los dos padres para obtener hijos.
 3. **[Mutación]** Con una probabilidad de mutación (p_{mut}), mutar cada gen de los cromosomas hijo
 4. **[Reemplazo]** Añadir nuevos hijos y determinar nueva población.
4. **[Verificar]** Condición de parada $\rightarrow$ **STOP**, y proporcionar como solución el cromosoma con mejor valor de aptitud $f(x)$.
5. **[Ciclo]** Ir al paso 2

Condición de Parada

- Número fijo de **generaciones** o **soluciones** generadas
- Tras un **tiempo de cómputo** fijo
- Considerando medidas de **diversidad en la población** (*convergencia*)
- Cuando se alcance el **óptimo** (si es conocido), o un determinado valor de fitness.
- Tras **varias generaciones sin ninguna mejora**



Ejemplo-1. Aplicación A. Genéticos

Resolver el siguiente problema de “cifras y letras”:



Solución:

$$((10 \times 100) + 8) / (9 - 5) = 252$$

Codificación: Genes del individuo



?

Diseñar un algoritmo genético para encontrar la secuencia de operaciones matemáticas (+, -, *, /) para, a partir de los números dados, aproximarse tanto como sea posible a 252

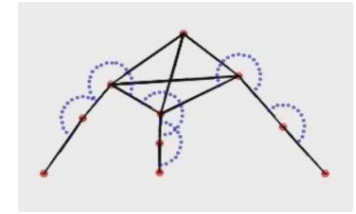
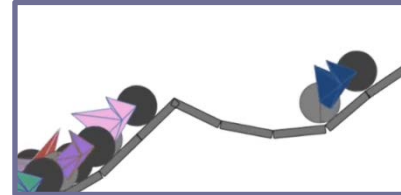
- **Utilidad relativa de las herramientas genéricas para desarrollar AG concretos:**
 - La codificación/decodificación/fitness suelen ser muy dependientes del problema (suponer un individuo como una secuencia de bits sin estructura no suele bastar).
 - Las operaciones (cruce, mutación, etc.) son dependientes de la codificación de los individuos, etc.
 - El proceso 'genérico' del AG es muy simple.
- **Entornos libres (o demos)**
 - Opt4J (<http://opt4j.sourceforge.net/>). Formulación sencilla utilizando librerías implementadas en Java.
 - HeuristicLab <http://dev.heuristiclab.com/> Entorno generalista; original método de programación.
 - PARADISEO <http://paradiseo.gforge.inria.fr/> API de programación en C++
 - <http://www.rubicite.com/Tutorials/GeneticAlgorithms.aspx> ,
 - http://people.sc.fsu.edu/~jburkardt/cpp_src/simple_ga/simple_ga.html
 - <http://www.aridolan.com/ga/gaa/gaa.html>
 - <http://www.geneticprogramming.com/GPpages/software.html>
- **Entornos comerciales:** MATLAB (*Toolbox libres sobre AG*). MATLAB & Global Optimization Toolbox
- **Librerías que codifican código generalista en los AG:**
 - En entornos generales: Matlab, Mathematica, GoldenBerry-GA, etc.
 - En **java** (JGAP, JCLEC, ECJ, Opt4J, OSGiLiath), **C++** (Open BEAGLE, MALLBA, Evolving Objects-EO, GALIB), **Python** (DEAP) etc.

Estas utilidades pueden ayudar, pero la verdadera complejidad es el diseño de los individuos y semántica de las operaciones (cruce, mutación) en cada problema concreto.

Conclusiones sobre Algoritmos Genéticos

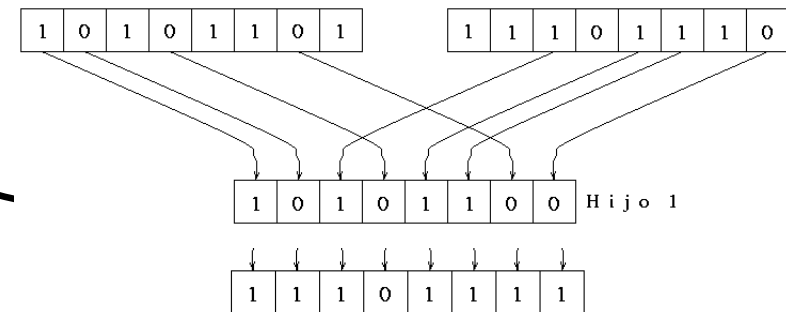
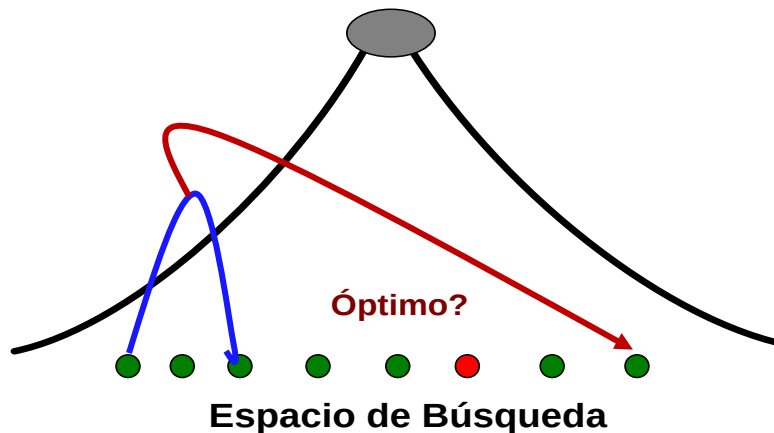
Los algoritmos genéticos constituyen una de las metaheurísticas más aplicadas.

- Existen muy diversas aplicaciones reales
- Existe una amplia base de trabajo en el área.



La metaheurística, fundamentalmente, se basa en:

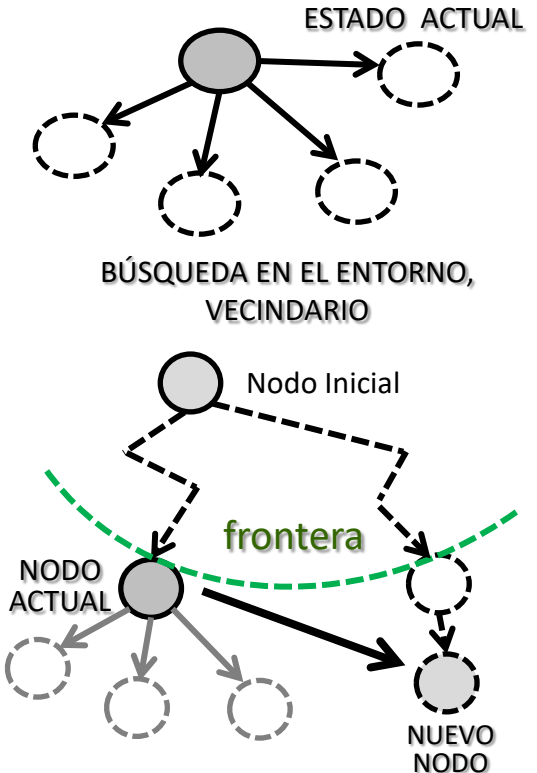
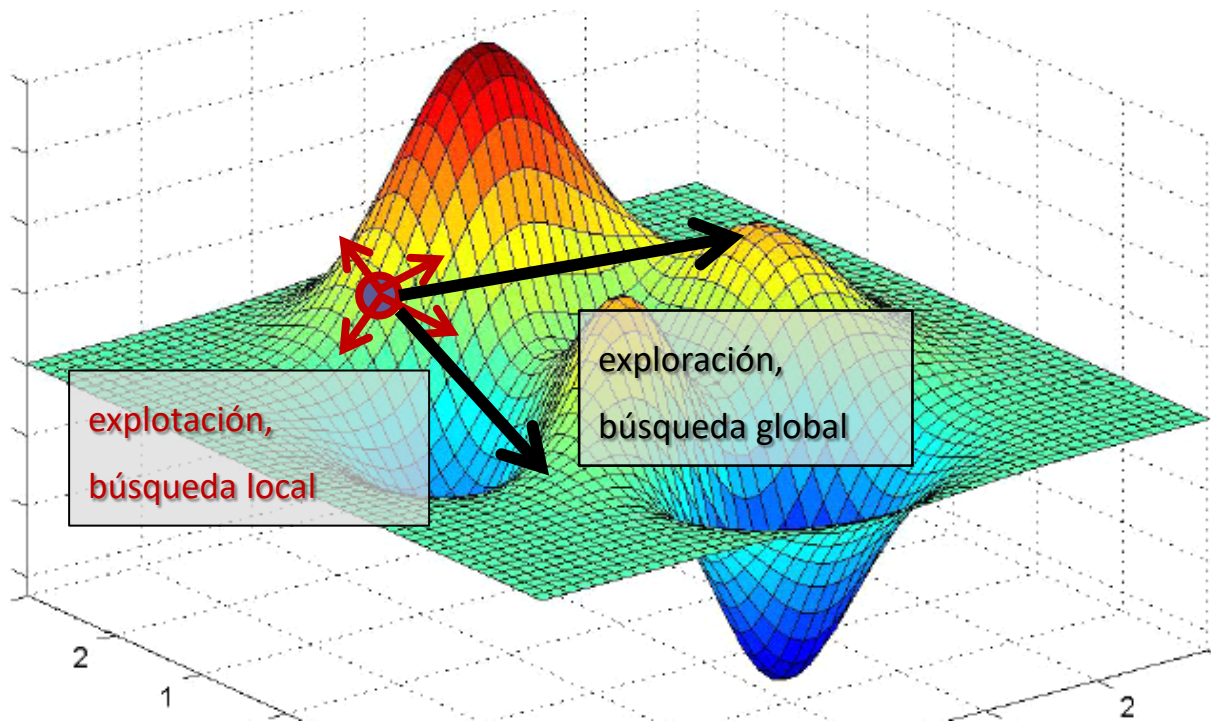
- 1) Propagar CARACTERÍSTICAS de buenas soluciones (**selección/cruce**) a nuevas soluciones $\approx$ **mejora local**
- 2) Cierta **exploración** de nuevas soluciones variantes (**mutación**)



Explotación vs Exploración

En el proceso de búsqueda de buenas soluciones, existe un equilibrio entre los conceptos de Exploración vs Explotación:

- Buscar mejores soluciones alrededor de la solución actual (**explotación, búsqueda local**),
- Buscar mejores soluciones en zonas lejanas desconocidas (**exploración, búsqueda global**),



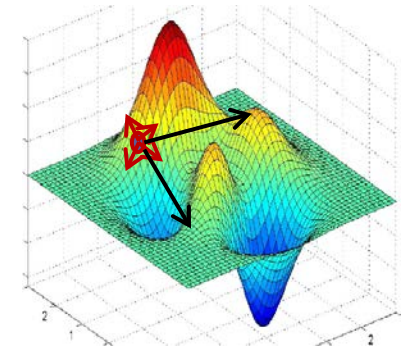
≈ **Teoría de la Selección r/K** , donde las fuerzas evolutivas tienen dos estrategias diferentes:

- **Estrategia r :** Muchos descendientes, con baja probabilidad de supervivencia.
- **Estrategia K :** Se invierten grandes recursos en pocos descendientes, con alta probabilidad de supervivencia.

Exploración vs Explotación en Algoritmos Evolutivos

La aplicabilidad de los AG proviene en gran parte en su capacidad de combinar un proceso de búsqueda global (exploración) con un proceso de búsqueda local (explotación).

- La **exploración** está relacionada con mayor diversidad de la población, a fin de explorar un amplio espacio de estados **no priorizando un alto fitness**.
- La **explotación** se incrementa focalización en individuos de **alto fitness**, priorizando su importancia.



La exploración / explotación en AG es controlada por los parámetros de las operaciones:

- Mutación: **Introduce innovación en la búsqueda, diversificación.**
 - Salto a una zona de búsqueda distante/local.
 - Mayor mutación implica **mayor exploración** (búsqueda global). Menor mutación implica mayor explotación.
- Cruce: **Recombinación de soluciones existentes, memoria, intensificación, explotación.**
 - Relacionado con la exploración/explotación:
mayor nº puntos de cruce $\Rightarrow$ mayor exploración. Menor nº puntos $\Rightarrow$ mayor explotación.
- Selección (fitness): **Dirige la búsqueda hacia zonas prometedoras del espacio de búsqueda.**
 - Relacionado con la explotación (presión selectiva). A mayor presión selectiva **mayor explotación** (menor presión, mayor exploración).

Algunas discusiones:

Cruce vs Mutación

- Depende del problema pero, en general, mejor utilizar ambos operadores (Se podría utilizar solo mutación, pero no solo cruce).
- El cruce permite explorar áreas lejanas de los padres (combinando lo mejor de ellos).
- La mutación explora áreas cercanas al individuo que muta

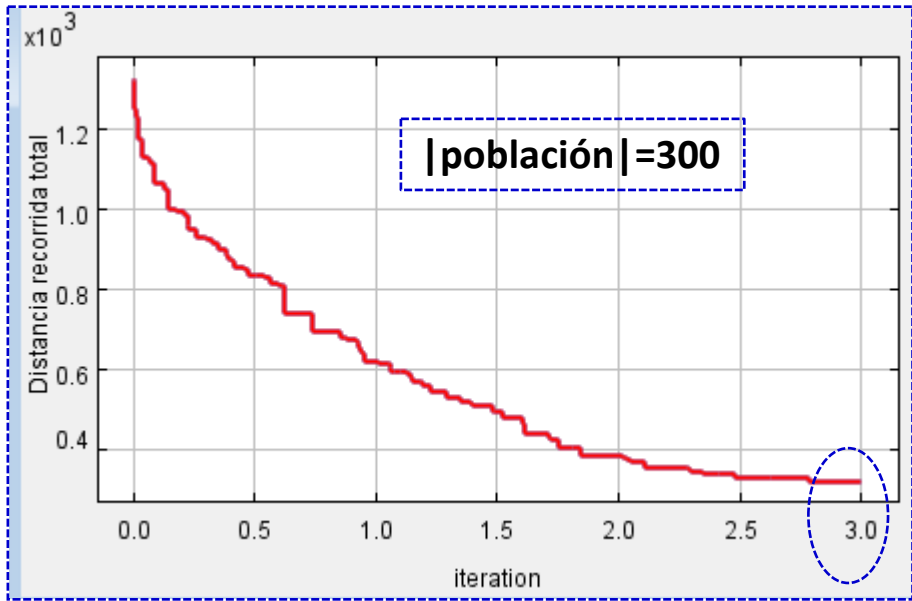
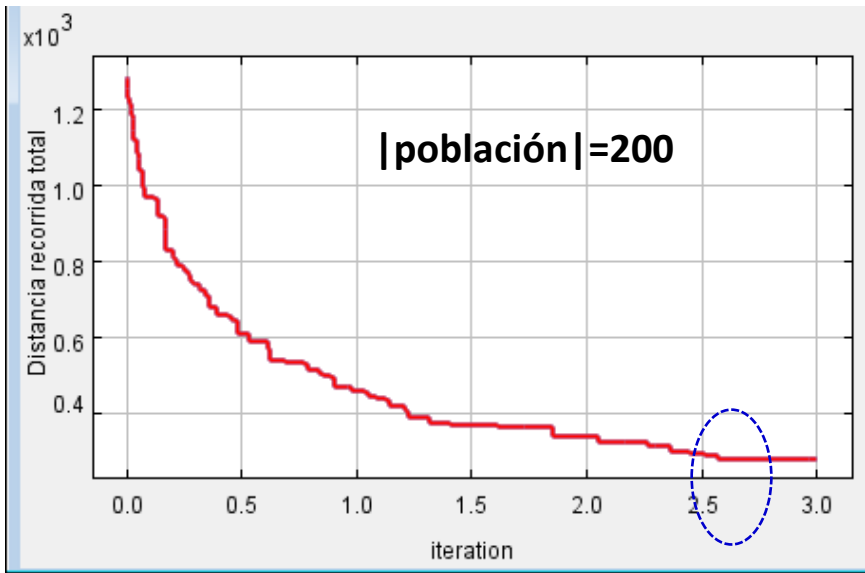
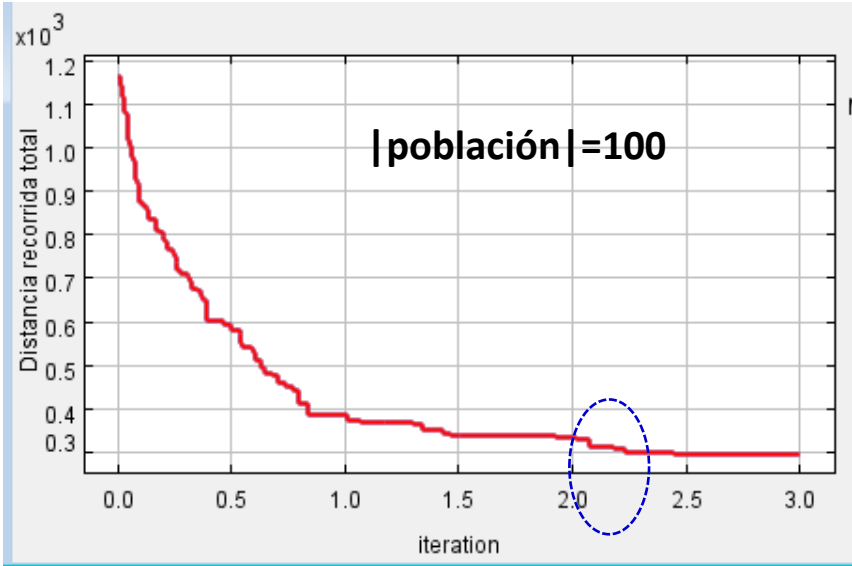
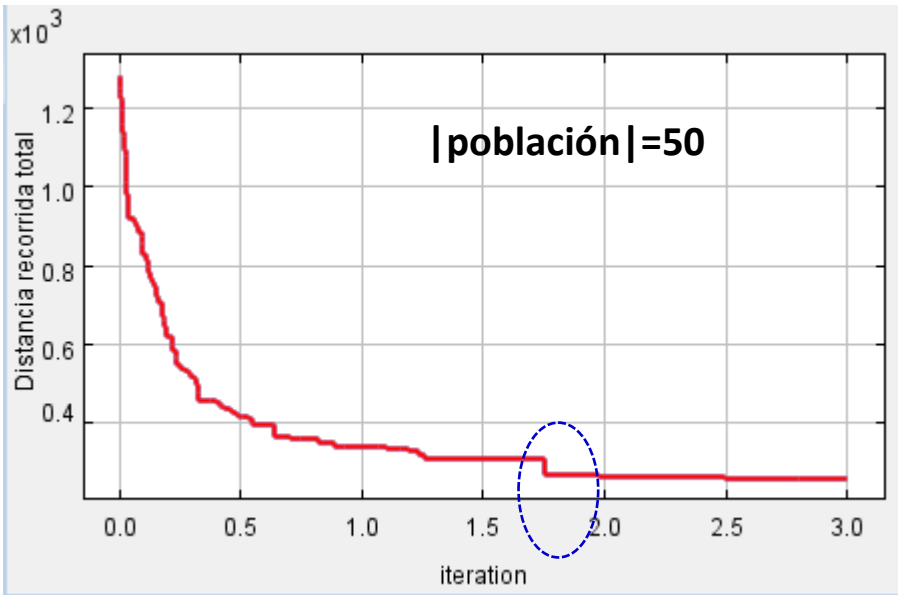
Selección

- Es necesaria la selección por la limitación de los recursos (*similar naturaleza!*).
- Se debe garantizar la probabilidad de combinación de todos los individuos. Pero siempre debe haber un cierto elitismo.

Reemplazo

- Típicamente, la selección se aplica para seleccionar padres.... ¿Y para el reemplazo (*no siempre los peores*)? $\Rightarrow$ Reemplazo Proporcional
- Reemplazo basado en **Fitness** **VS** basado en la **Antigüedad**

Convergencia en un AG vs |población| (iteraciones x 1000)



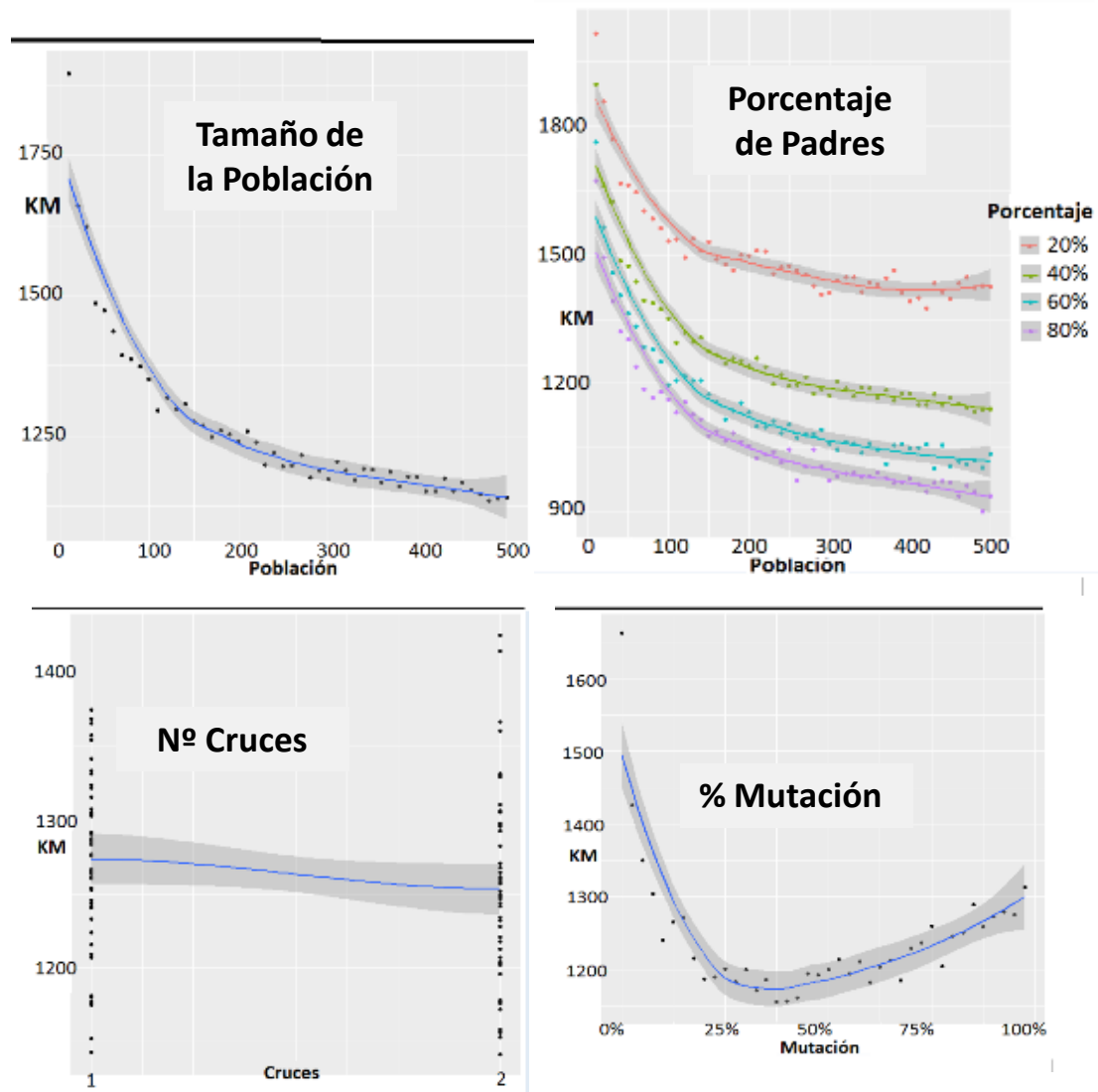
Parametrización de los AG

- ✓ Codificación Individuos.
- ✓ Población: Nº individuos
- ✓ Población Inicial: Generación
- ✓ Método de Selección
- ✓ Método de Cruce
- ✓ Mutación: Probabilidad y Método
- ✓ Reemplazo

Diversos estudios analíticos / experimentales

Cuál es la mejor parametrización ante un problema?

Cuál es la dependencia de la parametrización?



Genetic algorithm parameter optimisation using Taguchi method for a flexible manufacturing system scheduling problem.

Int. Journal of Production Research, 2015. Vol. 53, No. 3,

Problemas de Optimización: Mono / Multi Objetivo

Un problema de optimización **mono-objetivo** se puede formular como obtener una solución al problema, *valor de las variables* $X = \{x_1, x_2, \dots, x_n\}$, tal que:

$$\min/\max [f_{\text{objetivo}}(X)],$$

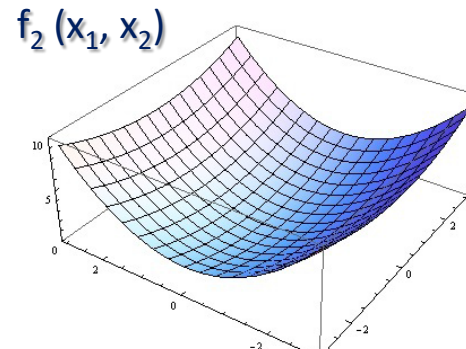
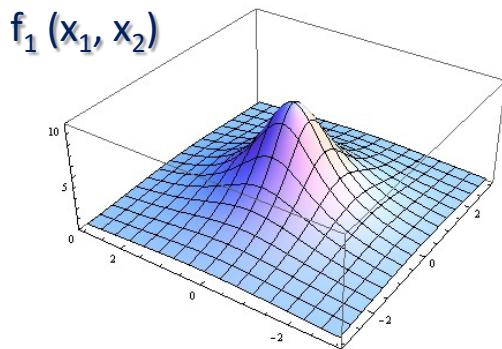
y tal que se cumpla el conjunto de restricciones del problema.

Ejemplos: *Problema del Viajante, donde $(x_1, x_2, \dots, x_n)$ representaría la secuencia de ciudades, Problema de la Mochila, donde $(x_1, x_2, \dots, x_n)$ representaría la inclusión del objeto, Planificación clásica de acciones, etc.*

Un problema de optimización **multi-objetivo** se puede formular como la maximización/minimización de una función vectorial:

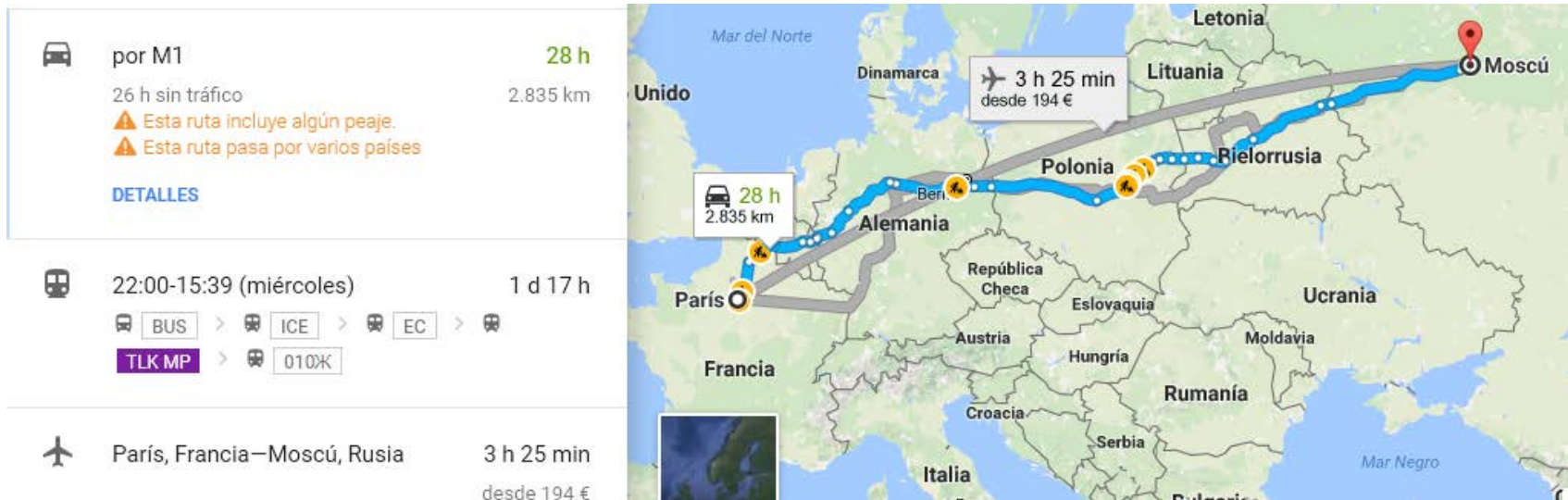
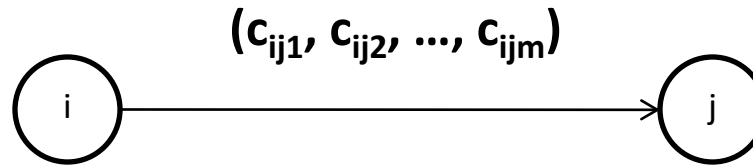
$$\min/\max [f_1(X), f_2(X), \dots, f_m(X)],$$

tal que se quiere maximizar/minimizar *simultáneamente* un conjunto de funciones



Ejemplo: Viajante Multi-Objetivo

Cada arco c_{ij} entre dos ciudades (i, j) tiene un vector de costes, donde cada componente representa el coste de atravesar el arco según un cierto objetivo (*tiempo, distancia, consumo, etc.*).



- Normalmente no existe una solución que optimice de forma simultánea todos los objetivos.
- Habitualmente existen conflictos, tal que la mejora de un objetivo da lugar al empeoramiento de otro.

Principales opciones para resolver problemas multiobjetivo:

a) Combinación ponderada de objetivos (pesos w_i): Se convierte en problema mono-objetivo

$$\text{Min } \sum_{i=1,m} w_i f_i(X)$$

Ejemplo (viajante multiobjetivo):

$$\text{Min } (w_1 \text{ Coste } (x_1, x_2, \dots, x_n) + w_2 \text{ Distancia } (x_1, x_2, \dots, x_n) + w_3 \text{ Tiempo } (x_1, x_2, \dots, x_n))$$

Ventajas: Sencillez, Conversión a problema mono-objetivo

Inconvenientes: Diferencia en valor de los objetivos (requiere normalización),
No realiza una verdadera optimización multiobjetivo,
Solo ofrece una solución.

Devuelve: Una única solución

b) Métodos basados en la Eficiencia de Pareto: $\min/\max [f_1(X), f_2(X), \dots, f_m(X)]$,

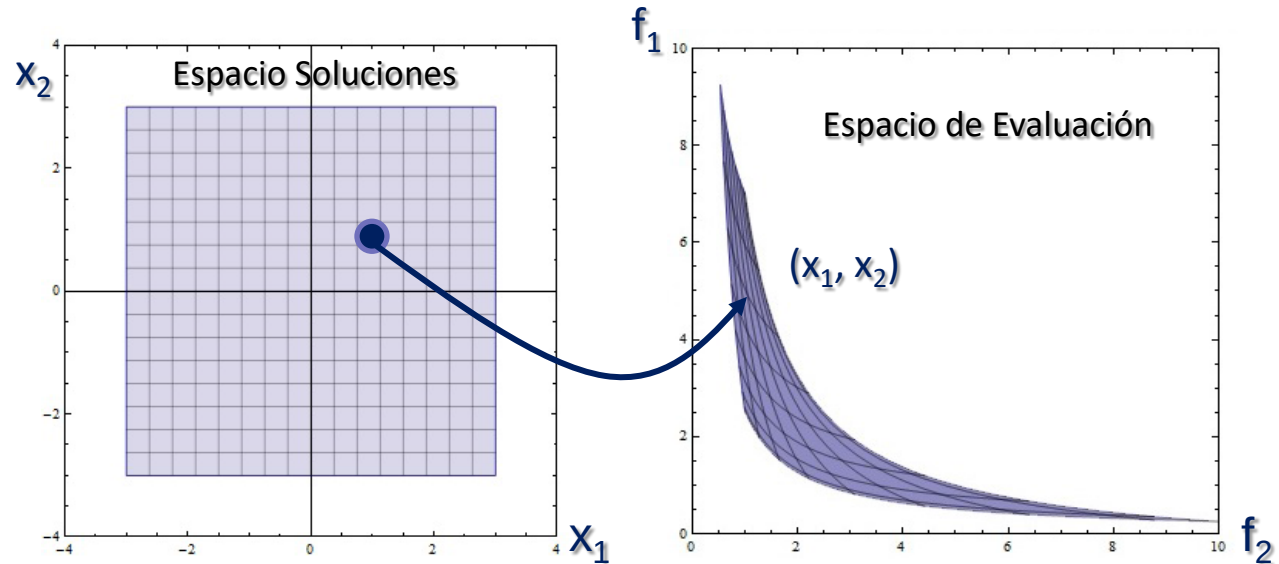
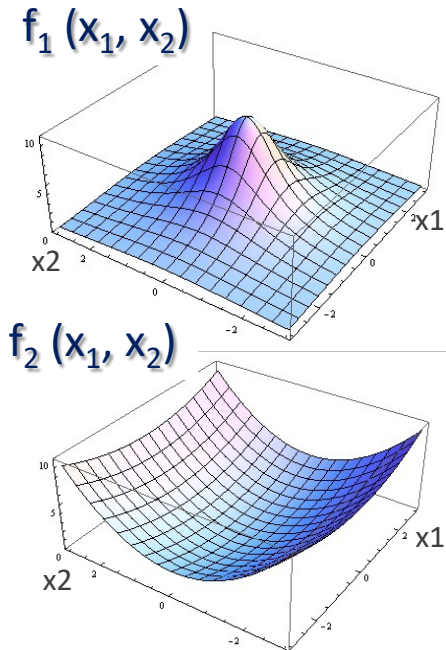
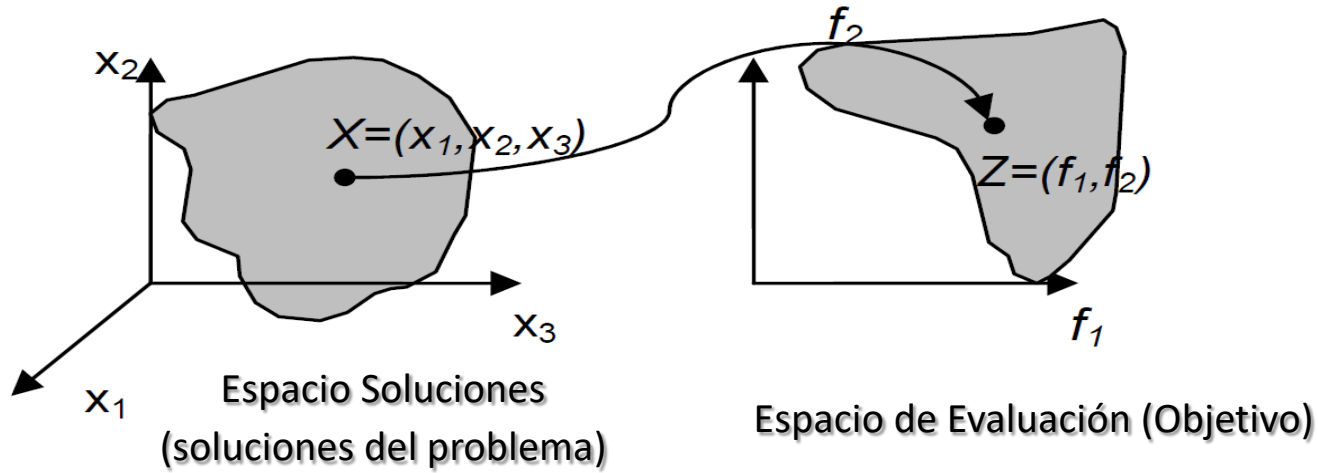
Ventajas: Trata realmente un problema multi-objetivo

Inconvenientes: Mayor complejidad.

Devuelve: Conjunto de soluciones (frontera de Pareto, o puntos cercanos a ella)

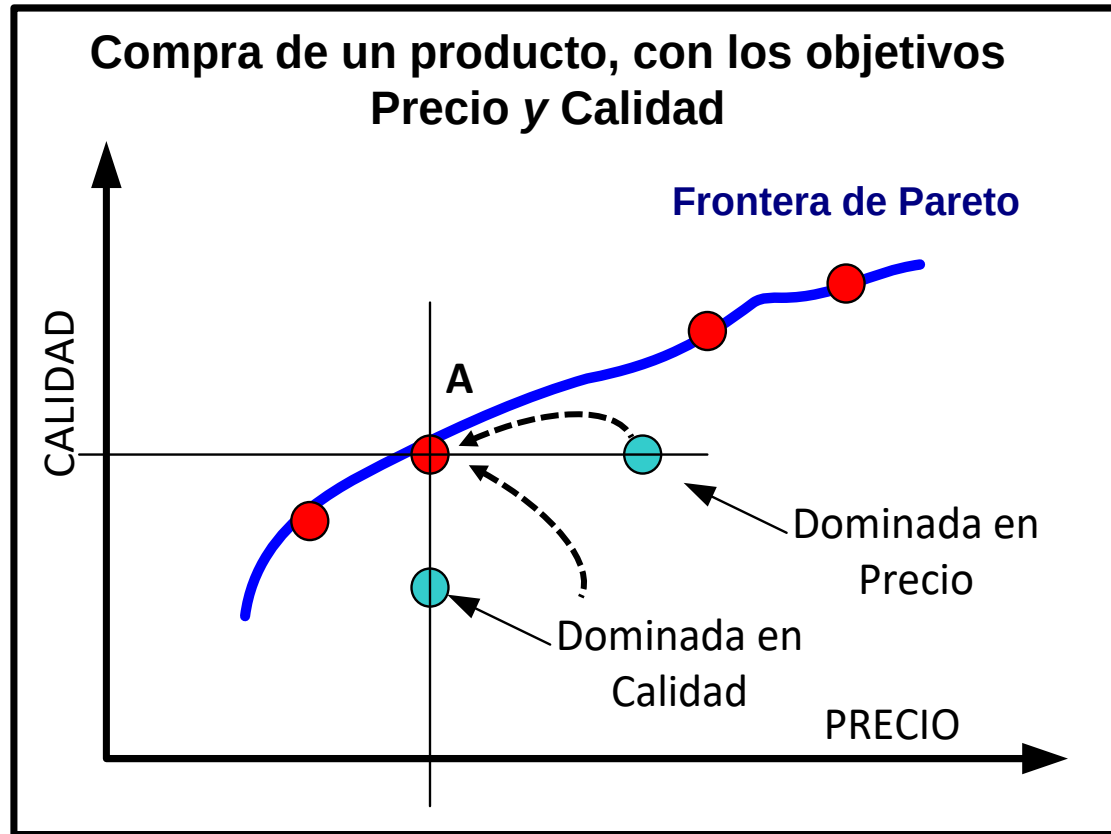
Espacio de Soluciones y Espacio de Evaluación

- El Espacio de Soluciones representa todas las combinaciones posibles en el valor de las variables.
- El Espacio de Evaluación (o Espacio Objetivo) representa el valor de los objetivos del problema



Frente (o Frontera) de Pareto (soluciones de la optimización multiobjetivo)

En problemas multi-objetivo, la mejora de un objetivo habitualmente empeora otro.



Dominancia de Soluciones: Una solución X_1 domina a otra solución X_2 , si:

- X_1 es mejor que X_2 en al menos un objetivo, y
- X_1 no es peor que X_2 en todos los objetivos

$\Rightarrow$ Es preferible X_1 a X_2

Aplicándolo iterativamente se obtiene el **Conjunto de Soluciones Dominantes:**

Conjunto de soluciones dominantes (soluciones no dominadas) $\Rightarrow$ Frente de Pareto

- **Conjunto de soluciones de la Frontera de Pareto:**

- Soluciones que no pueden mejorar uno de los objetivos sin empeorar algún otro.
- Estas soluciones determinan las mejores alternativas para la solución del problema.

Formalizando:

Una solución $X_i = (v_{i1}, v_{i2}, \dots, v_{in})$ **domina** a otra solución $X_j = (v_{j1}, v_{j2}, \dots, v_{jn})$, si y solo si:

$$\exists f_p \in \{f_1, f_2, \dots, f_m\}, f_p(X_i) < f_p(X_j) \quad \text{y} \quad \forall f_k \in \{f_1, f_2, \dots, f_m\}, f_k(X_i) \leq f_k(X_j)$$

Una solución X^* es **Pareto-óptima** si y solo si no existe otra solución X' , que la domine: **Conjunto de soluciones dominantes**

En caso de maximización, el cambio es trivial

Dominancia de Soluciones: Una solución $X1$ domina a otra solución $X2$, si:

- $X1$ es mejor que $X2$ en al menos un objetivo, y
- $X1$ no es peor que $X2$ en todos los objetivos

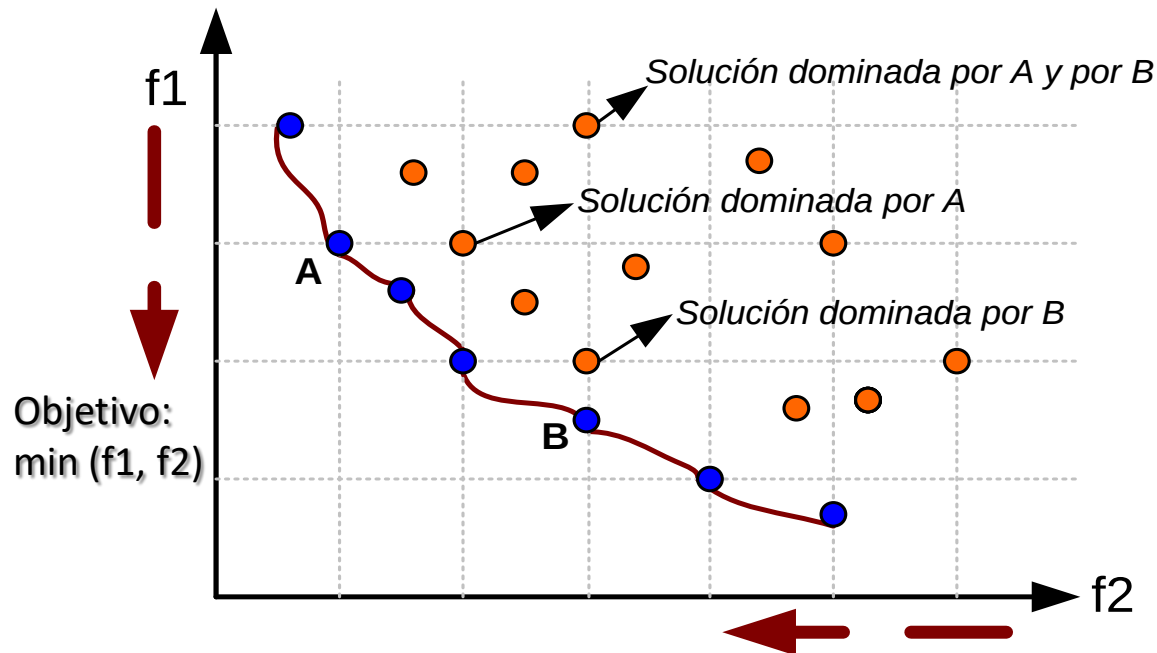
$\Rightarrow$ Es preferible $X1$ a $X2$

Aplicándolo iterativamente se obtiene el **Conjunto de Soluciones Dominantes:**

Conjunto de soluciones dominantes (soluciones no dominadas) $\Rightarrow$ Frente de Pareto

• **Conjunto de soluciones de la Frontera de Pareto:**

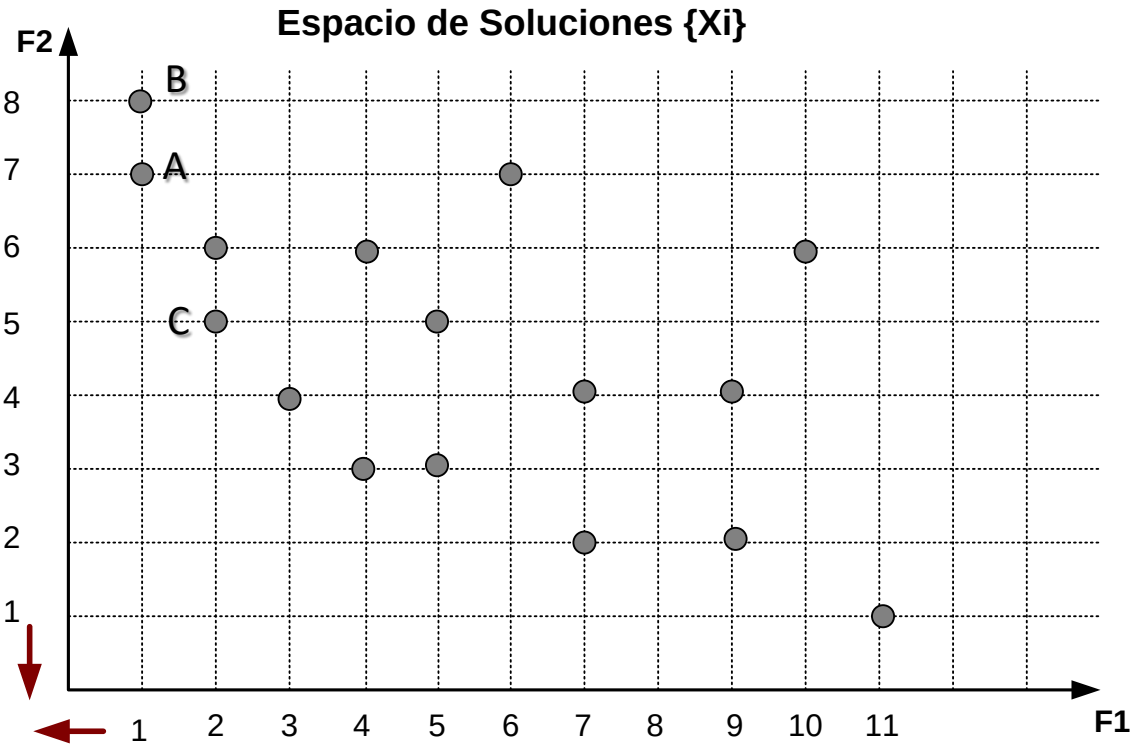
- Soluciones que no pueden mejorar uno de los objetivos sin empeorar algún otro.
- Estas soluciones determinan las mejores alternativas para la solución del problema.



- Un resolutor multiobjetivo debe ofrecer el Conjunto de Soluciones de la Frontera de Pareto (o soluciones más cercanas al frente).
- El frente puede tener una, muchas, o incluso infinitas.
- Se prefiere que las soluciones estén bien distribuidas.

Calculo de la Frontera de Pareto (se desea minimizar F1 y F2)

Se ordenan las soluciones según las funciones fitness objetivo (F1, F2,)



	F1	F2
A	1	7
	1	8
B	2	5
	2	6
C	3	4
	4	3
	4	6
	5	3
	5	5
	6	7
	7	2
	7	4
	9	2
	9	4
	10	6
	11	1

Se puede seguir cualquier orden (F1,F2 / F2,F1, ...)

- 1) Para cada valor de F1, se determina el par que minora F2.
- 2) Siguiendo el orden creciente de F1, se seleccionan los pares de valores que vayan minorando F2

F1	F2
1	7
1	8
2	5
2	6
3	4
4	3
4	6
5	3
5	5
6	7
7	2
7	4
9	2
9	4
10	6
11	1

Primer valor de F1

No minora F2 (obvio, repite valor F1)

Minora valor de F2 (**baja a 5**)

No minora F2 (obvio, repite valor F1)

Minora valor de F2 (**baja a 4**)

Minora valor de F2 (**baja a 3**)

No minora F2 (obvio, repite valor F1)

No minora F2

No minora F2 (obvio, repite valor F1)

No minora F2

Minora valor de F2 (**baja a 2**)

No minora F2 (obvio, repite valor F1)

No minora F2

No minora F2 (obvio, repite valor F1)

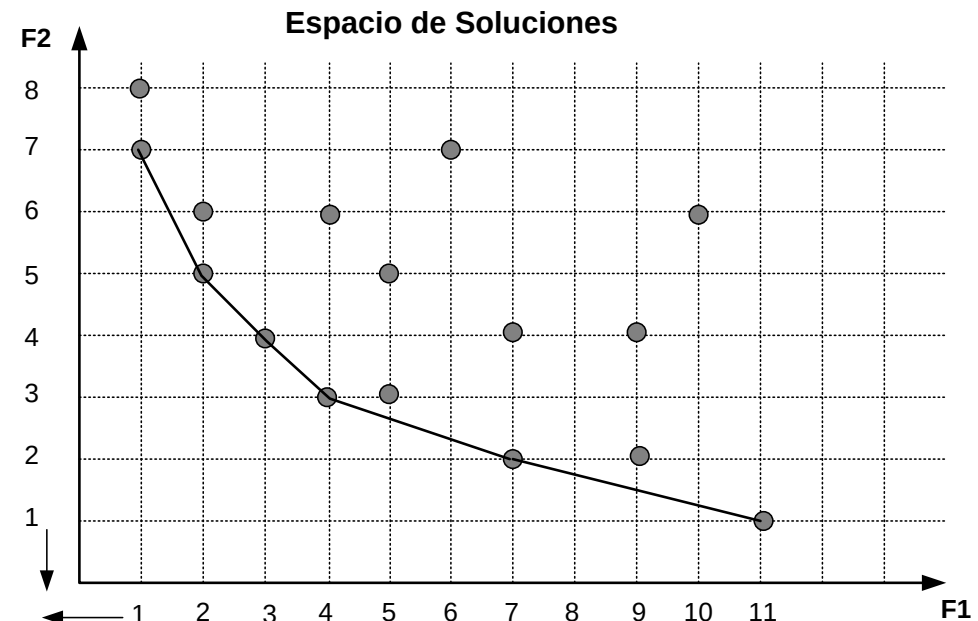
No minora F2

Minora valor de F2 (**baja a 1**)

La frontera de Pareto es:

**Conjunto de
Soluciones
Pareto-óptimas**

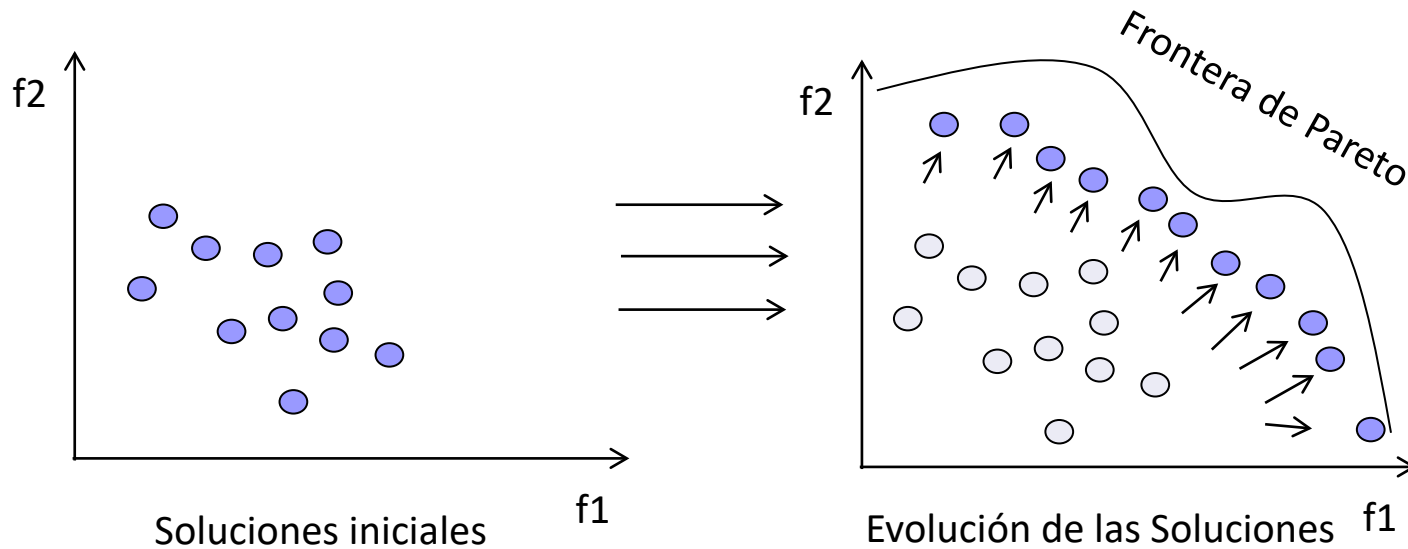
F1	F2
1	7
2	5
3	4
4	3
7	2
11	1



Una optimización multi-objetivo devuelve un conjunto de soluciones

El objetivo de una optimización multiobjetivo es:

- Obtener soluciones lo más cercanas a la Frontera de Pareto
- Obtener soluciones diversificadas

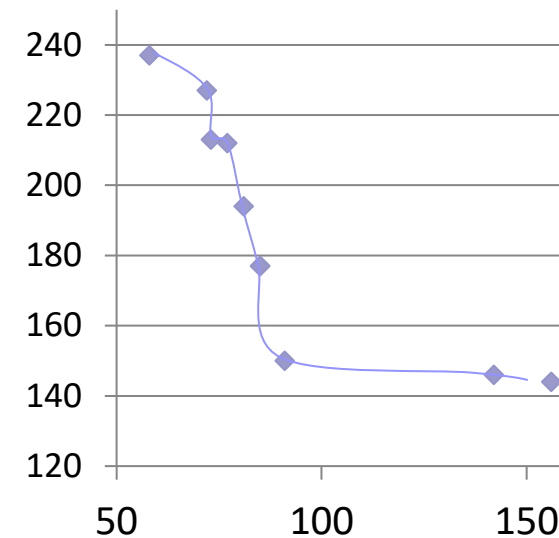


Ejemplo Opt4J (Viajante multiobjetivo)

Cada arista entre un par de nodos i,j tiene dos componentes: $(Coste_{ij}, Distancia_{ij})$

Problema multiobjetivo: $\min [Coste(X), Distancia(X)]$ recorrida

Population Monitor				
#	Individual	State	Coste total (M...	Distancia reco...
1	7 6 5 0 4 9 3 1 ...	evaluated	156.0	144.0
2	7 3 0 9 4 1 2 8 ...	evaluated	91.0	150.0
3	7 6 5 9 4 0 3 1 ...	evaluated	142.0	146.0
4	5 7 3 1 2 8 9 4 ...	evaluated	81.0	194.0
5	7 3 0 9 4 2 8 6...	evaluated	77.0	212.0
6	9 4 0 1 5 6 7 3 ...	evaluated	72.0	227.0
7	2 8 9 4 0 6 1 5 ...	evaluated	58.0	237.0
8	9 4 0 3 1 5 6 7 ...	evaluated	85.0	177.0
9	9 4 0 6 3 1 5 7...	evaluated	73.0	213.0
10	9 4 0 3 1 5 6 7...	evaluated	85.0	177.0
11	5 7 3 1 2 8 9 4...	evaluated	81.0	194.0
12	7 3 0 9 4 2 8 6 ...	evaluated	77.0	212.0
13	7 3 0 9 4 1 2 8...	evaluated	91.0	150.0
14	1 2 8 7 6 5 9 4 ...	evaluated	142.0	146.0
15	1 2 8 7 6 5 0 4 ...	evaluated	156.0	144.0
16	1 2 8 7 6 5 9 4 ...	evaluated	142.0	146.0
17	1 2 8 7 6 5 0 4 ...	evaluated	156.0	144.0
18	1 2 8 7 6 5 0 4 ...	evaluated	156.0	144.0
19	1 2 8 7 6 5 9 4 ...	evaluated	142.0	146.0
20	3 1 2 8 7 6 5 0 ...	evaluated	156.0	144.0
21	1 2 8 7 6 5 0 4 ...	evaluated	156.0	144.0
22	1 2 8 7 6 5 9 4 ...	evaluated	142.0	146.0
23	1 2 8 7 6 5 0 4 ...	evaluated	156.0	144.0
24	1 2 8 7 6 5 0 4 ...	evaluated	156.0	144.0



Se debe decidir una solución de la frontera de Pareto.

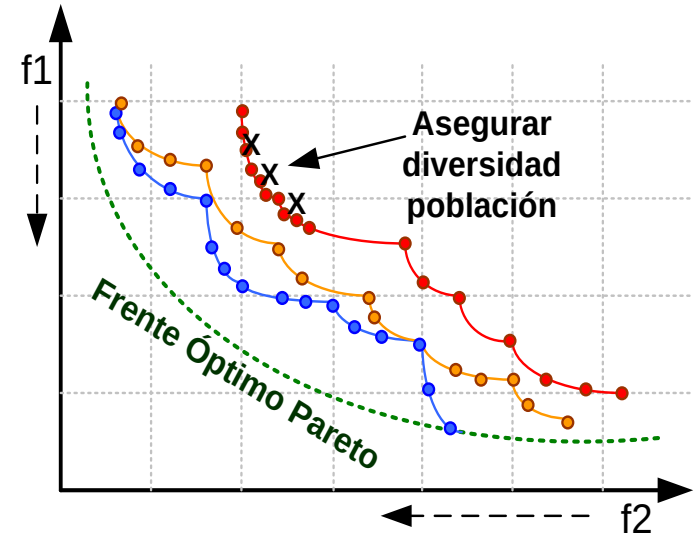
Algoritmos Genéticos para Problemas Multi-Objetivo

(MOEA Multi-Objective Evolutionary Algorithms)

El objetivo es obtener tantas soluciones Pareto-óptimas como sea posible y bien distribuidas.

Fundamentos del método elitista de selección:

- Se intenta **priorizar las soluciones que sean dominantes** en cada sucesiva generación del AG (*no tienen por qué ser 'todavía' soluciones del frente óptimo de Pareto*)
- *También debe mantenerse la diversidad de la población!*



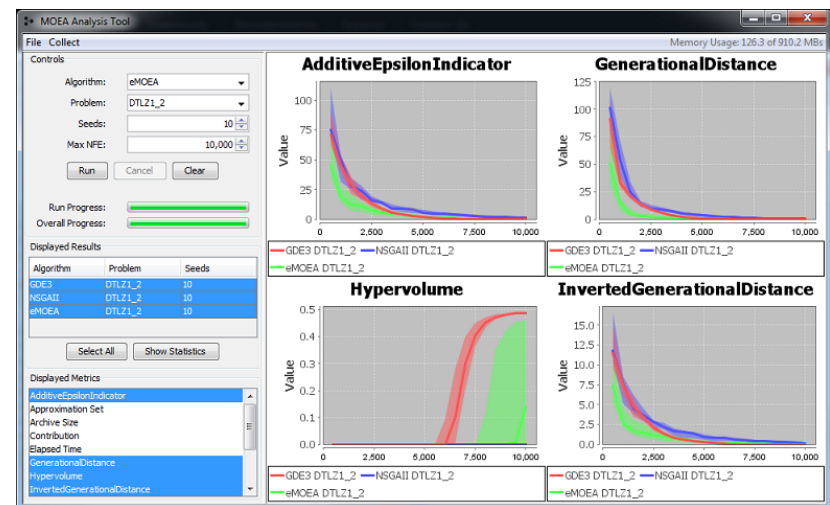
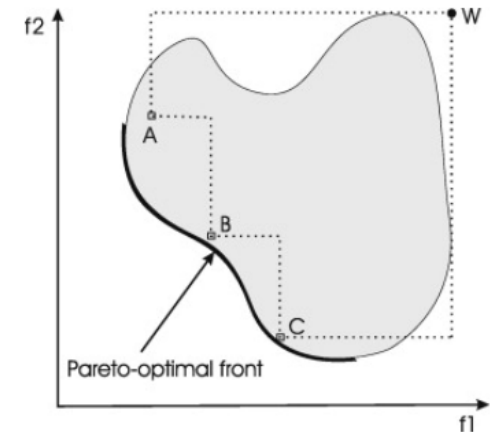
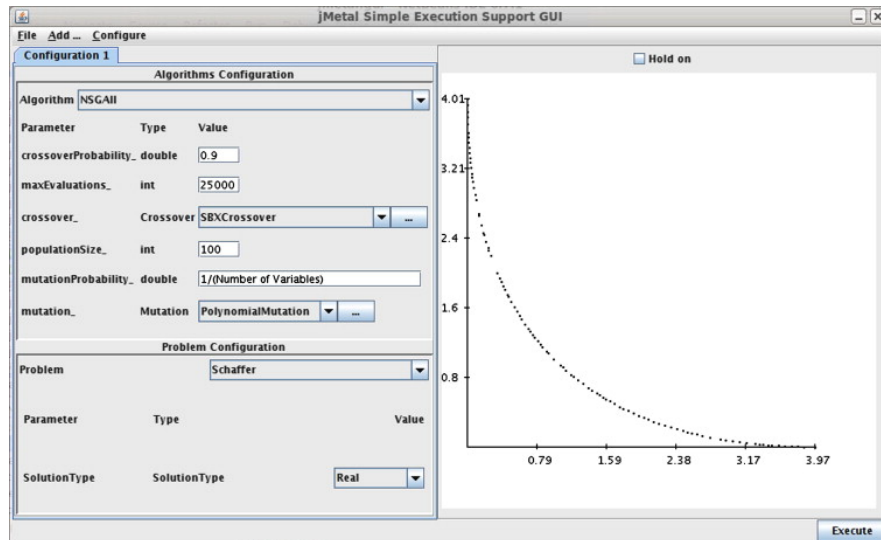
Proceso de selección:

- Las soluciones de la población se agrupan en diferentes *frentes*, típicamente según:
 - *nº de soluciones que las dominan (Rank de dominancia)*, o
 - *nº de soluciones que dominan (Count de dominancia)*.
- Mediante un método elitista, se seleccionan como padres las soluciones de los frentes (de mejor a peor), generando la nueva población hasta alcanzar el tamaño máximo.
- Para mantener la diversidad, se priorizan las *soluciones más dispersas* (que tengan mayor distancia respecto a otras soluciones del frente: *crowding distance sorting*).
- *Diversas variantes de ordenación: Rank (NSGA, NSGA-II), Count (MOGA, NPGA), Mixto (SPEA, SPEA2), etc.*
- El más conocido es NSGA (*Elitist Non-Dominated Sorting Genetic Algorithm*), con diversas variantes.

Los procesos de cruce y mutación son similares a un AG normal.

Algunas librerías y entornos para AG Multiobjetivo

- *Opt4J* (Java): <http://opt4j.sourceforge.net/>
- Project-Platypus/**Platypus** (Python) <https://github.com/Project-Platypus/Platypus>
- *jMetal* (Java): <http://jmetal.github.io/jMetal/>
- *MOEAFramework* (Java): <http://moeaframework.org/>



RESUMEN: Hay muchas guías o recomendaciones para realizar un AG:

- **Es muy importante la codificación genética:**

Debe capturar la información relevante de la solución y hacerla fácilmente identificable.

- **Tamaño de la población:**

Debe de ser suficiente para garantizar la diversidad de las soluciones.

- **Condición de terminación:**

Lo más habitual es que la condición de terminación sea la convergencia del algoritmo genético o un número prefijado de generaciones.

- **Lenguaje de Representación:**

Mejor utilizar alfabetos con ***baja cardinalidad*** (es decir, con pocas letras) como el binario.

- **Función de Aptitud:**

Debe hacer una presión adecuada y reflejar la calidad de la solución en el problema.

- ***No fiarse de la autoridad central (operadores que ‘controlan’ a toda la población):***

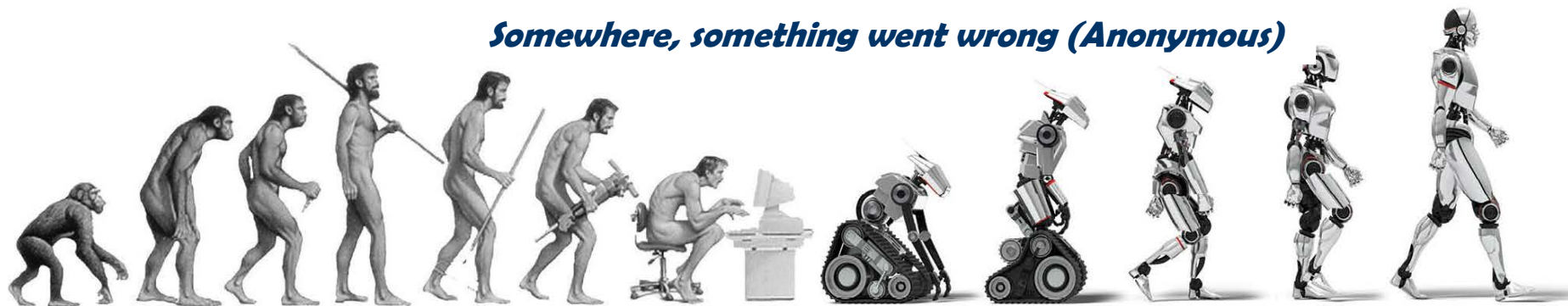
La Naturaleza actúa de forma distribuida y segmentada, por tanto, se debe de minimizar la necesidad de operadores que "vean" a toda la población.

Por ejemplo, en vez de comparar el fitness de un individuo con todos los demás, se puede comparar sólo con los *vecinos*, es decir, aquellos que estén, de alguna forma, situados cerca de él.

→ Más parecido a la naturaleza, donde los recursos son compartidos por los individuos locales.

- **Detractores (falta de criterio o razón subyacente para la optimización) y Defensores (proceso evolutivo, aplicaciones con éxito).**
 - Es suficiente la información genética para capturar y progresar en los aspectos relevantes de la solución?
 - Es suficiente una evolución basada solo en los genes?
 - O más bien, los individuos evolucionan por aprendizaje, comunicación y competición entre ellos, por 'entes' de información no exclusivamente genéticos?
- **Muchas aplicaciones!**

Acústica, Ingeniería aeroespacial, Astronomía y astrofísica, Química, Ingeniería eléctrica, Mercados financieros, Juegos, Geofísica, Ingeniería de materiales, Militares, Biología molecular, Reconocimiento de patrones y minería de datos, Robótica, Diseño de rutas y horarios, etc.
- **Diversa información disponible en web:** www.geneticprogramming.com/



Ejemplo-2: Scheduling

Cuatro lectores (L1, L2, L3, L4) desean leer tres periódicos (P1, P2, P3) en ese mismo orden.

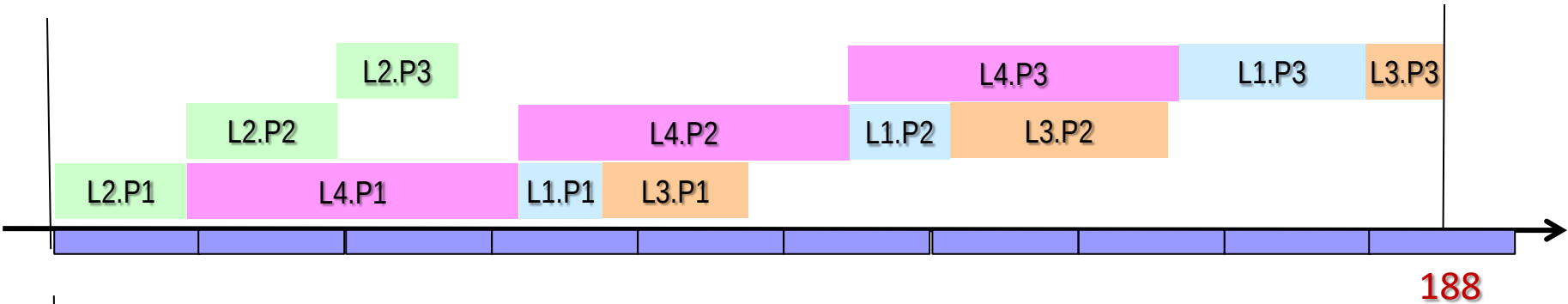
Todos pueden empezar a partir de un mismo tiempo, cada uno requiere un tiempo de lectura y no pueden compartir los periódicos mientras lo leen.

Objetivo: Asignación de periódicos a lectores, tal que se minimice el tiempo total en el que todos los lectores han leído todos los periódicos

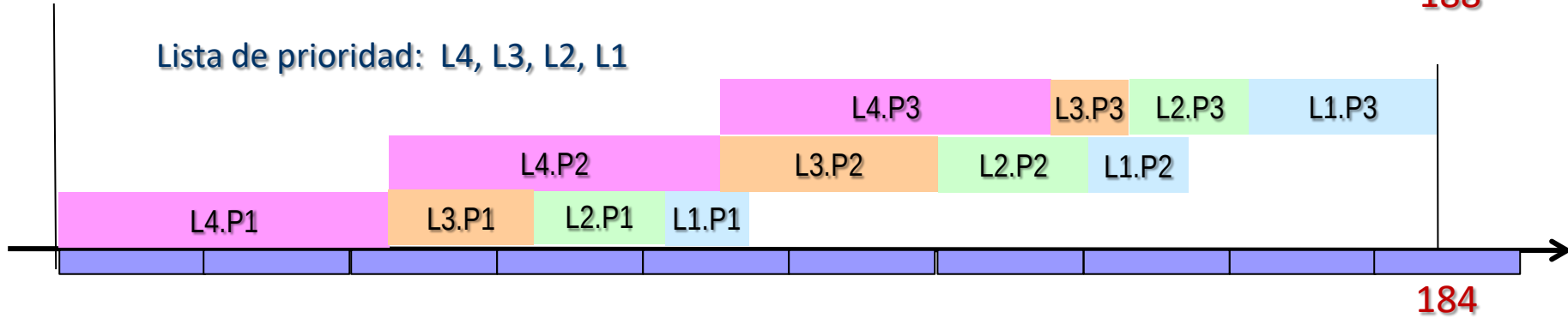
	P1	P2	P3
L1	10	15	25
L2	18	20	15
L3	20	30	10
L4	45	45	44

Codificación de un individuo?

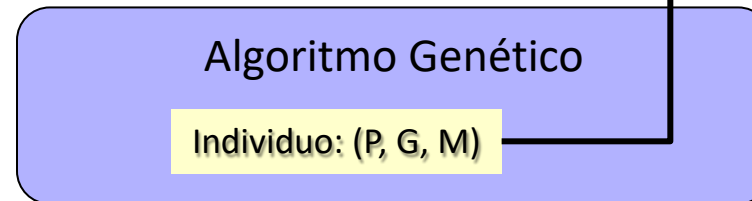
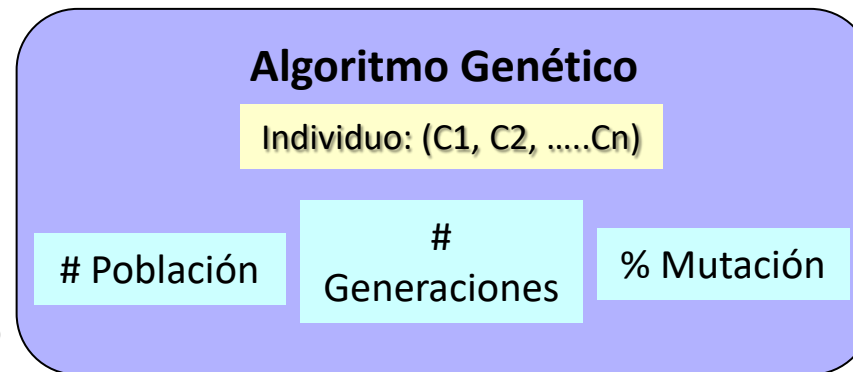
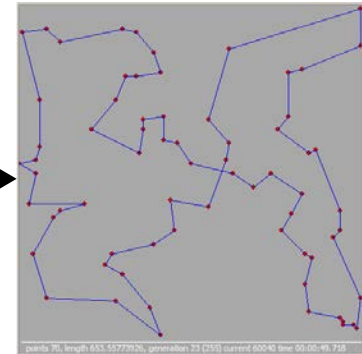
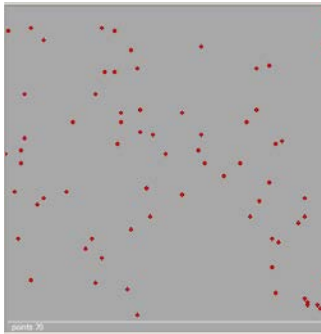
Lista de prioridad: L2, L4, L1, L3



Lista de prioridad: L4, L3, L2, L1



Ejemplo-3: *Genético sobre Genético*



fitness

**Objetivo:
Mejor (P, G, M)**

- Un AG es un proceso estocástico (no garantía obtención de buenas soluciones).
- Ante un tipo de problemas, hay que establecer sus parámetros, en función de sus características, tamaño, etc.
- Esto se hace mediante un análisis razonado del problema, esperando encontrar los parámetros que darán buenas soluciones (sin garantía).
- Se puede plantear un proceso de optimización que obtenga los mejores parámetros ante similar clase de problemas. No garantiza un buen funcionamiento futuro, pero es una alternativa al punto anterior.

Ejemplo-4: lista de espera de barcos para el atraque en una terminal de contenedores:



Cada Barco se caracteriza por:

- Tiempo de llegada previsto (a una lista de espera)
- Eslora (longitud de muelle a asignar).
- Movimientos (import/export) de contenedores. Implica su tiempo de servicio.
- Prioridad

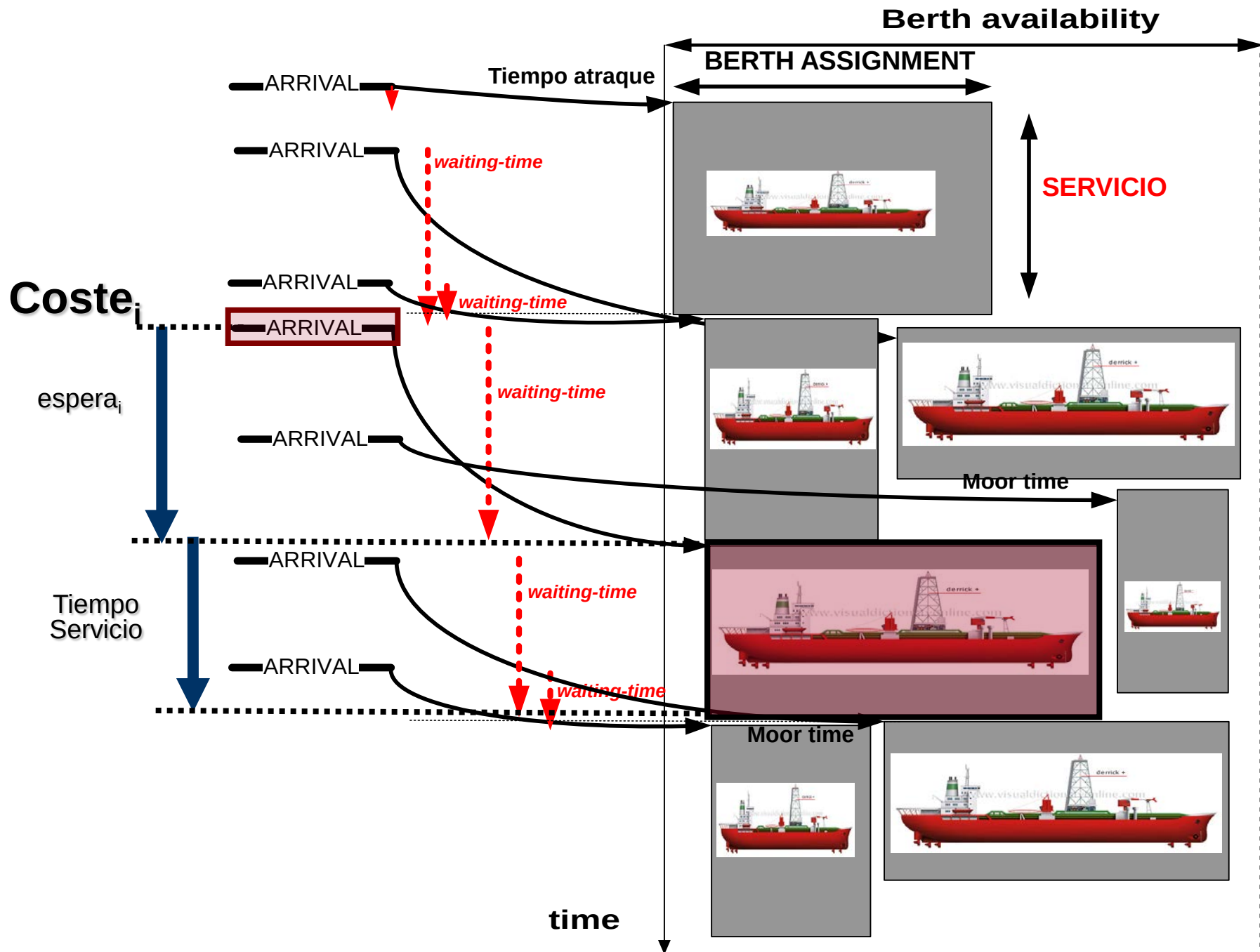
El muelle de atraque tiene un longitud determinada (90 metros)

Objetivo: Asignar tiempo y posición de atraque, tal que se minimice el tiempo total ponderado de espera.

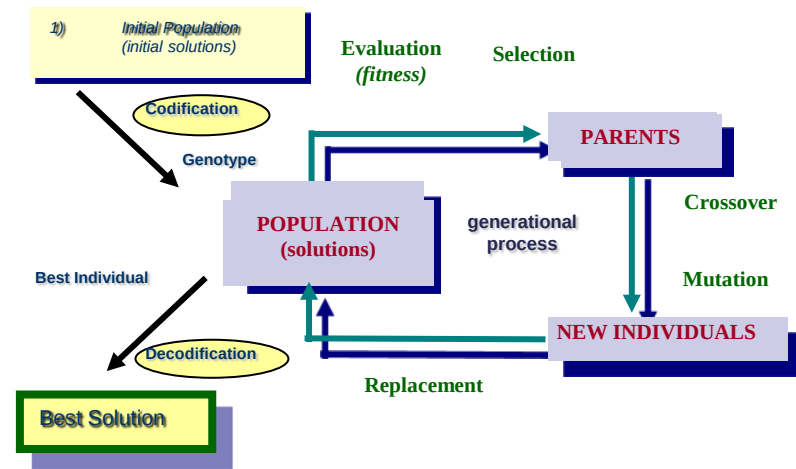
$$\sum_i (Prioridad * Tiempo-espera) = Función Fitness$$

Tiempo de Espera = Tiempo de Atraque – Tiempo de Llegada

Hay otros condicionamientos: Calado y profundidad del muelle, distancia de seguridad, asignación de grúas y duración del servicio, preferencia del lugar de atraque, etc....



Diseño del AG



Codificación: Un gen es un barco (Barco_i)

Un cromosoma (individuo) es una secuencia de genes

Ejemplo de individuos:

$\text{Indiv}_i: (1\ 2\ 3\ 4\ 5) \Rightarrow$ regla de despacho para el ataque

$\text{Indiv}_j: (3\ 1\ 2\ 5\ 4) \Rightarrow$ regla de despacho para el ataque

El esquema de descodificación aplica la prioridad de ataque conforme al orden en la secuencia de genes del individuo.

Descodificación:

El orden de los barcos en el cromosoma se utiliza como una **regla de despacho**.

- Los genes se visitan de izquierda a derecha
- Para cada gen (i), el barco B_i se programa para el tiempo más próximo posible, de forma que ninguna de las restricciones se viola.
- Si hay varias posiciones disponibles, se atraca lo más cerca de uno de los extremos libres

Ejemplo (20 barcos):

Datos

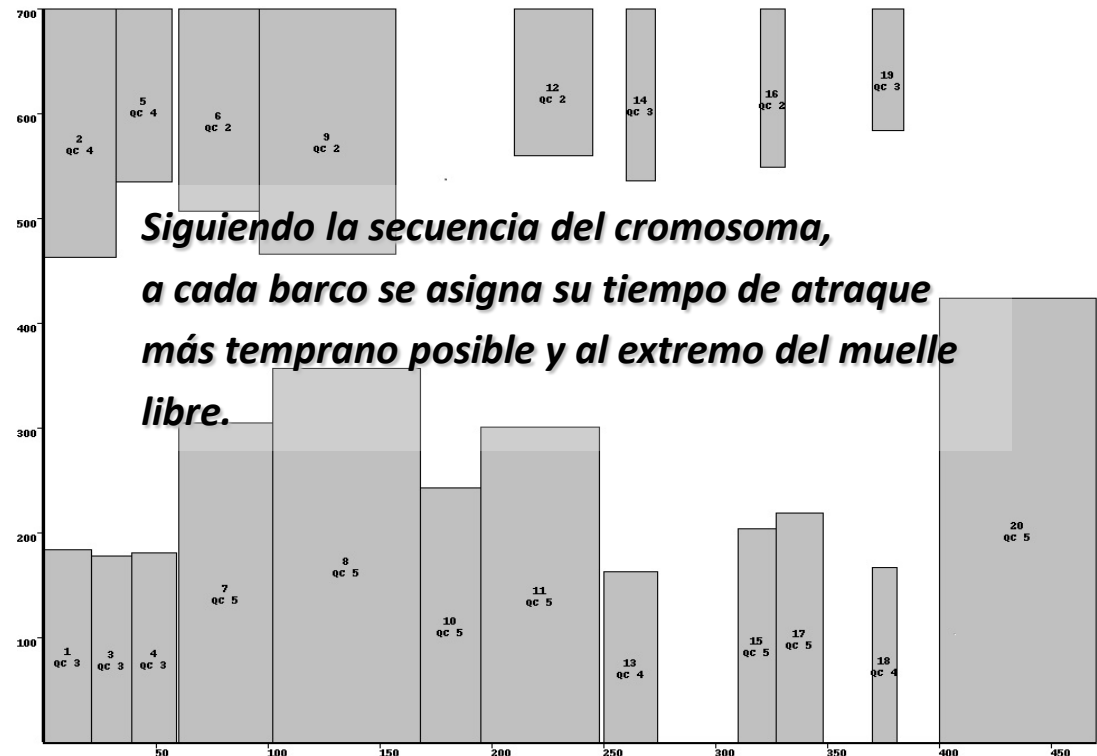
V (i)	t (i)	pr (i)	l(i)	mov(i)
1	0	8.0	468	564
2	50	5.0	207	349
3	110	6.0	178	659
4	140	7.0	205	709
5	170	8.0	209	869
6	180	5.0	294	215
7	180	4.0	271	187
8	190	5.0	249	209
9	200	7.0	479	392
10	230	9.0	353	890
11	240	6.0	369	365
12	260	9.0	423	691
13	280	7.0	205	775
14	290	3.0	195	190
15	290	5.0	203	244
16	310	7.0	264	666
17	320	7.0	430	295
18	360	5.0	255	253
19	390	7.0	363	458
20	410	8.0	391	626

Ejemplo de un cromosoma (individuo):

(1, 3, 17, 5, 7, 2, 16, 4, 6, 11, 15, 18, 13, 20, 19, 12, 14)

Descodificación:

Regla de Despacho

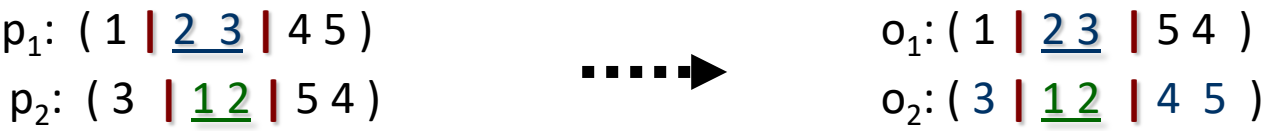


Población Inicial: 200 individuos (Generada Aleatoriamente) $\equiv$ 200 secuencias.

Selección: Por torneo de un paso (2:1). Todos los cromosomas se agrupan en pares, seleccionando el mejor por fitness. Cada uno de estos pares será cruzado o no de acuerdo a una probabilidad de cruce ($P_c=0,8$) para obtener dos hijos.

Operador de Cruce: Cruce aleatorio de dos puntos.

Ejemplo: (" | " representa puntos de cruce): Los hijos son:



Se toman las subsecuencias entre las posiciones 2 y 3 de cada padre y se rellenan las posiciones restantes con los genes que representan los restantes barcos tomados en el orden que tienen en el otro padre (*sin repetición: evita reparción*)

Mutación: Cada hijo (o padre si no ha sido seleccionado para cruce), es mutado de acuerdo a una probabilidad ($P_{mut}=0,1$).

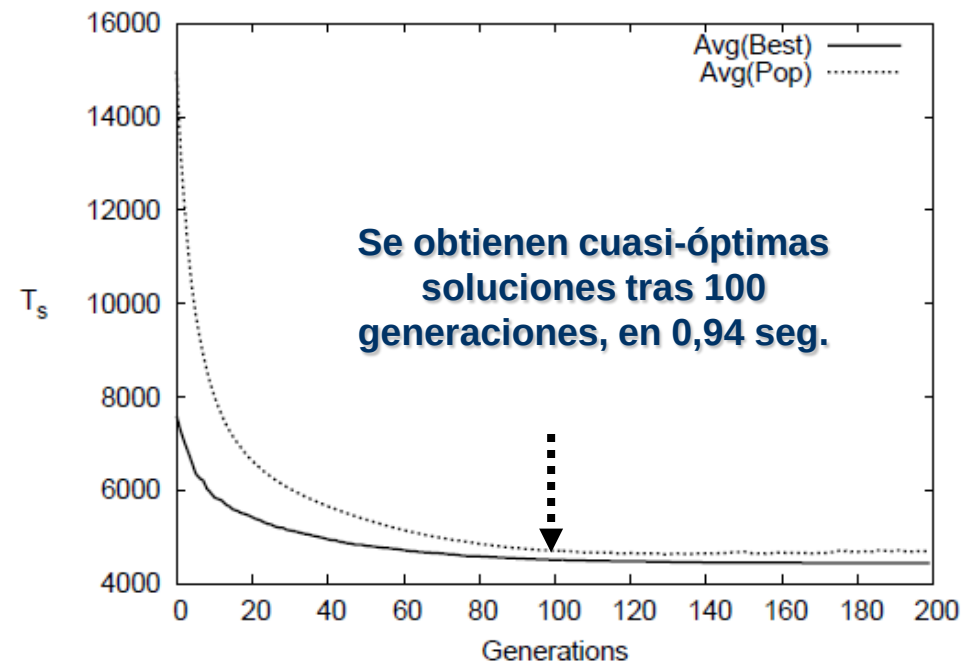
Se modifica aleatoriamente el orden de una subsecuencia del cromosoma,



Reemplazo: Por torneo (4:2). Se seleccionan dos individuos, por su mejor fitness entre cada par padres y sus dos hijos.

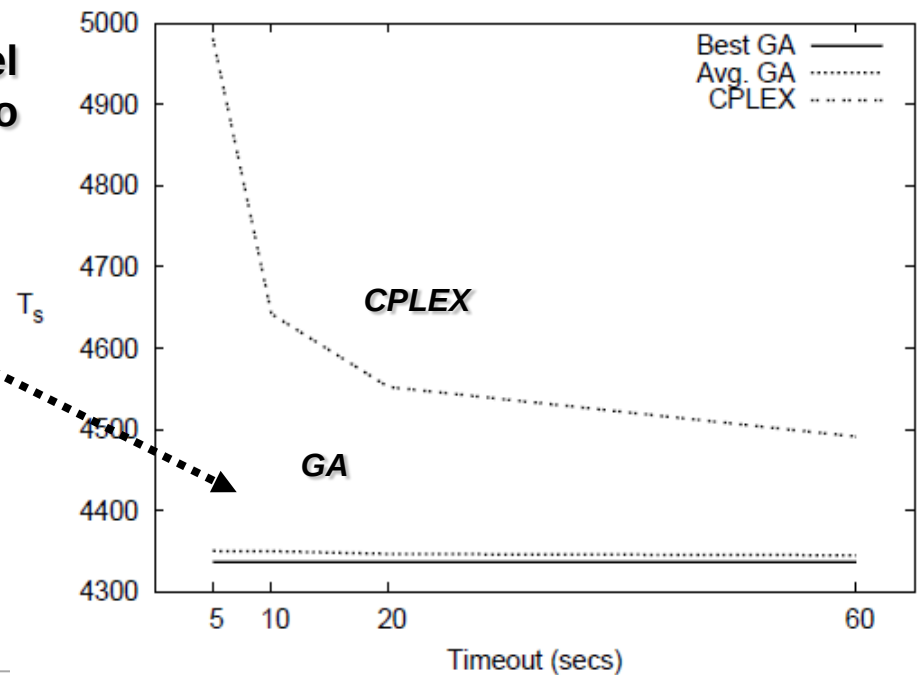
Condición de Terminación: número de generaciones, tiempo máximo.

Gráfico de Convergencia el AG



Mejora de la optimización en función del tiempo de cálculo

El AG no requiere un gran tiempo de cálculo. (la mejora es $<1\%$ a partir de 5" de cómputo)



Algoritmos Meméticos

A **Memetic Algorithm** is a population of agents that alternate periods of

self-improvement (via local search)

with periods of *cooperation (via recombination)*,

and competition (via selection). P. Moscato, 1989



"On Evolution, Search, Optimization, Genetic Algorithms and Martial Arts: Towards Memetic Algorithms".

Caltech Concurrent Computation Program Report 826, Catech, Pasadena, California (1989).

Los AG combinan procesos de búsqueda global (exploración) con búsqueda local (explotación).

- La **exploración** está relacionada con la diversidad de la población, explorando un amplio espacio de estados (**cruce, mutación**).
- La **explotación** permite focalizar sobre el entorno de buenas soluciones (**selección alto fitness**).

La explotación / exploración en los AG está controlada por el proceso de selección, mutación y cruce.

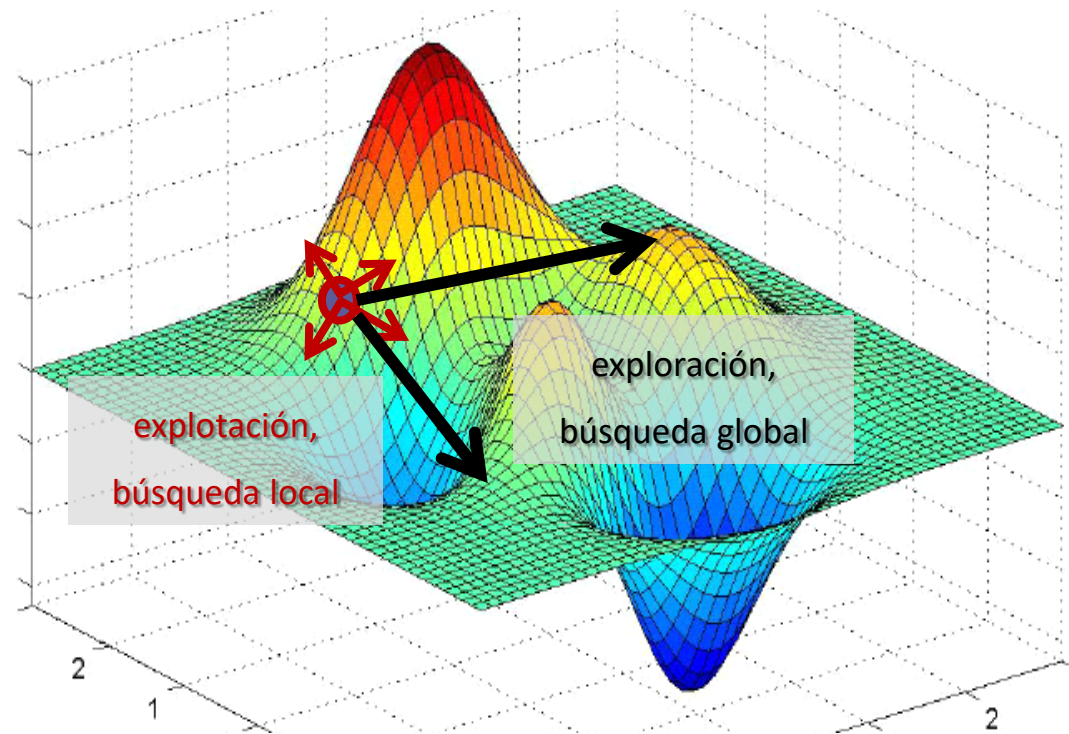
Pero no hay un proceso específico de mejora local.....

Los algoritmos evolutivos suelen resultar:

- ✓ Buenos exploradores (Búsq. global)
- Malos explotadores (Búsq. local vecinos)

Los algoritmos de mejora local (ES, Tabú, etc.) son:

- ✓ Buenos explotadores (Búsq. Local vecinos)
- Malos exploradores (Búsq. global)

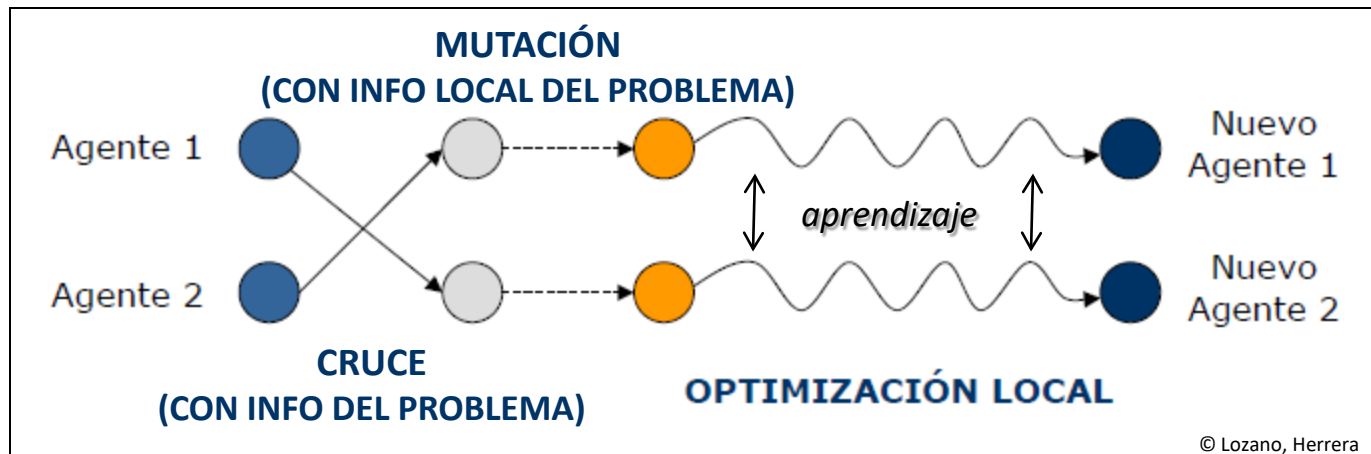


Los Algoritmos Meméticos aparecen como evolución de los *Algoritmos Genéticos Híbridos*.

También conocidos como Algoritmos lamarckianos, Algoritmos evolutivos híbridos, Algoritmos de búsqueda local genética, etc.

Un algoritmo memético es una población de agentes que alternan etapas de auto-mejora (**búsqueda local**) con etapas de cooperación (recombinación, mutación) y competición (selección).

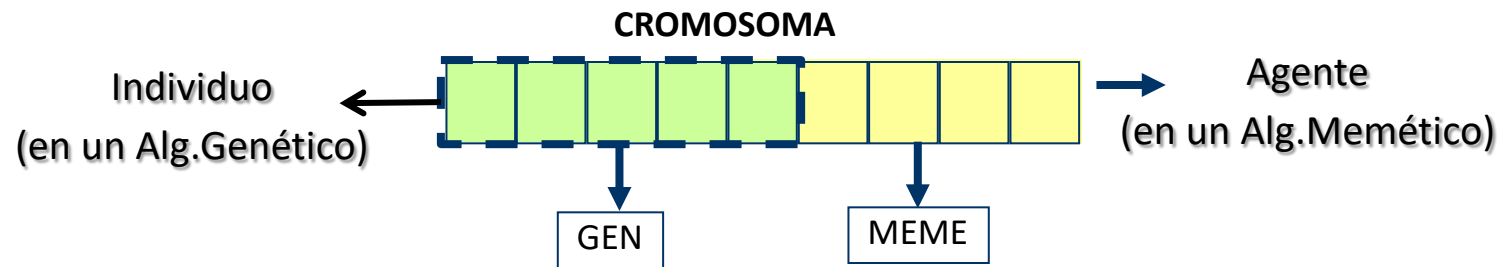
- **Algoritmos Genéticos Híbridos:** Incorporan un proceso de mejora local de la solución.
- **Algoritmos Meméticos:** Algoritmo poblacional, con los principios de selección, cruce y mutación, pero introduciendo *información local* del problema en los operadores, *procesos de mejora local* y *transmisión del conocimiento* no exclusivamente genética (colaboración y aprendizaje en los procesos de mejora local, mutación, etc.).



Similitud con la naturaleza:

Los individuos, además de la información genética de sus padres, *también heredan información (aprenden) sobre cómo mejorar* en el dominio del problema concreto a resolver.

- Gen: Unidad básica de información genética $\Rightarrow$ Genotipo: Características Solución
- **Meme**: unidad básica de transmisión cultural (imitación, aprendizaje) en el dominio del problema $\Rightarrow$ Aprendizaje sobre proceso mejora local.



MEME: Concepto análogo al gen (pero en la evolución cultural / aprendizaje).

Tratado por Lamarck, 1850: *“las experiencias de un organismo afectan directamente el marcaje genético de su prole, por lo que lo aprendido por los padres se traslada genéticamente a sus hijos”.*)

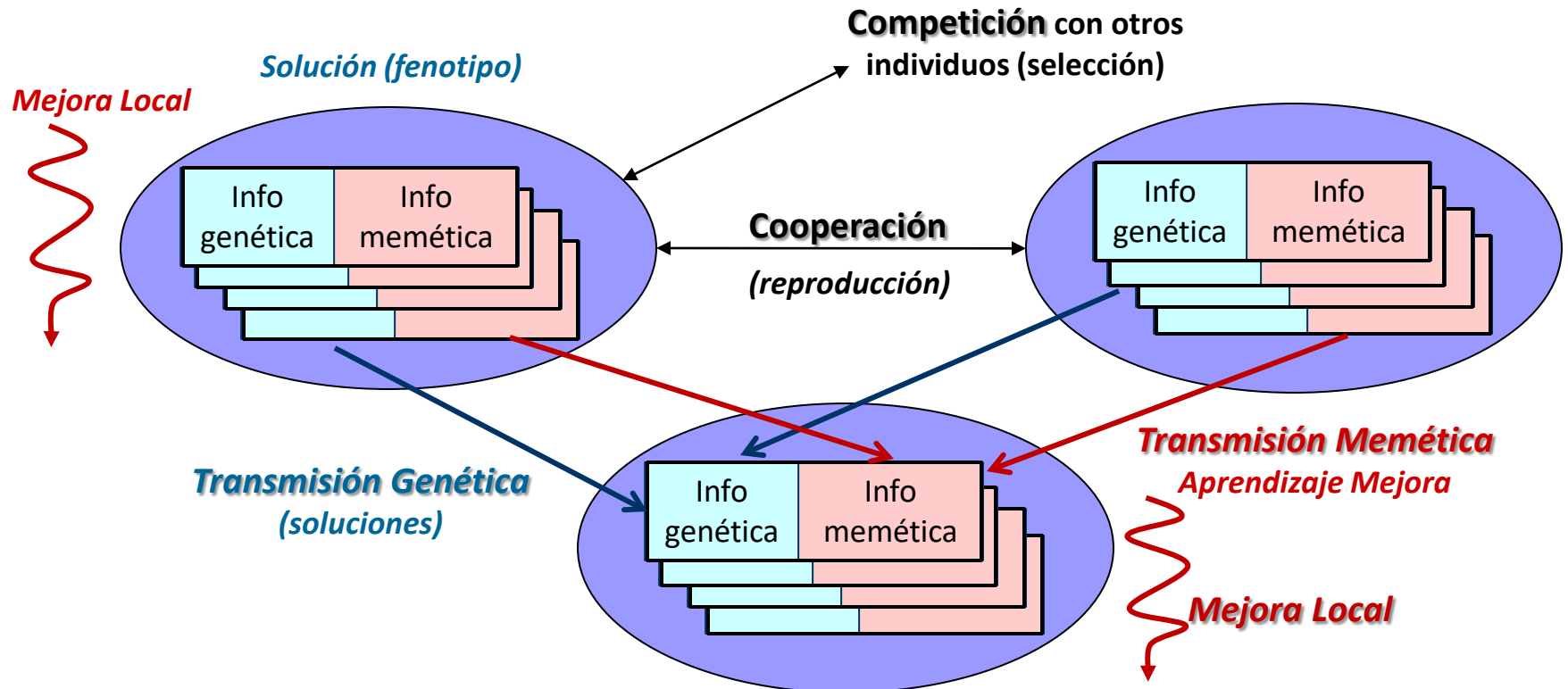
- Los ‘memes’ se propagan de individuo a individuo en un proceso de ‘imitación’ (aprendizaje, mejora)
- Representan información del comportamiento (heurísticas a emplear en un proceso de mejora local).

Algoritmos Mémeticos

Conjunto de individuos (agentes) compiten, cooperan y..... *mejoran y aprender a mejorar*

- AG $\Rightarrow$ Los agentes **interaccionan entre sí** en un marco de **competición** (selección) y **cooperación** (cruce) de manera análoga al comportamiento de los individuos en la naturaleza
- AG-HIBRIDO $\Rightarrow$ Los agentes tienen un **proceso interno de mejora local** de sus soluciones (guiado por heurísticas / metaheurísticas).
- AM $\Rightarrow$ Cada agente es capaz de aprender de sus padres y de **transmitir su conocimiento** (sobre su proceso de mejora local) a sus descendientes.

Cada agente puede **contener una o más soluciones**.

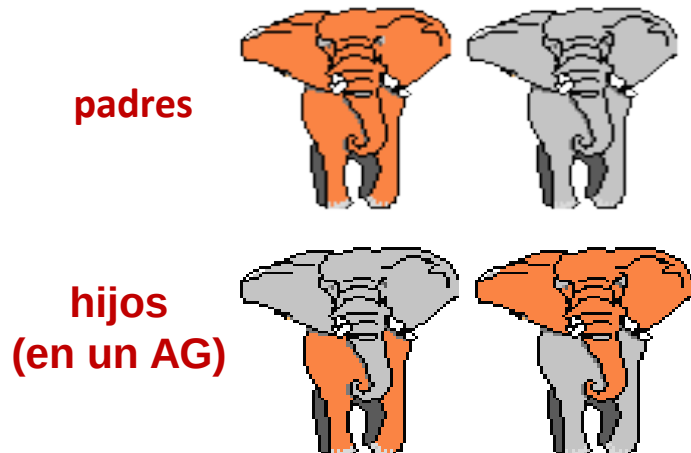


- Los **procesos de mejora local** de los agentes (individuos) se pueden realizar:
 - a) **Tras cada cruce:** Como un proceso de mutación, **tras el cruce** (proceso de mejora local, priorizando ciertas características de individuo).
 - b) **Al final de cada ciclo inferencial (generacional).**

Cada agente tiene su propio proceso de mejora local (y heurística). En función de su resultado, las características del proceso de mejora local es transmitido de padres a hijos (meme).

Para que un algoritmo híbrido sea considerado AM, la Búsqueda Local debe aplicarse integrada en el proceso evolutivo.

- Se requieren **Meta-operadores** (selección, cruce, mutación) para actuar con la información del individuo, de su entorno, de la transmisión del aprendizaje y del problema.



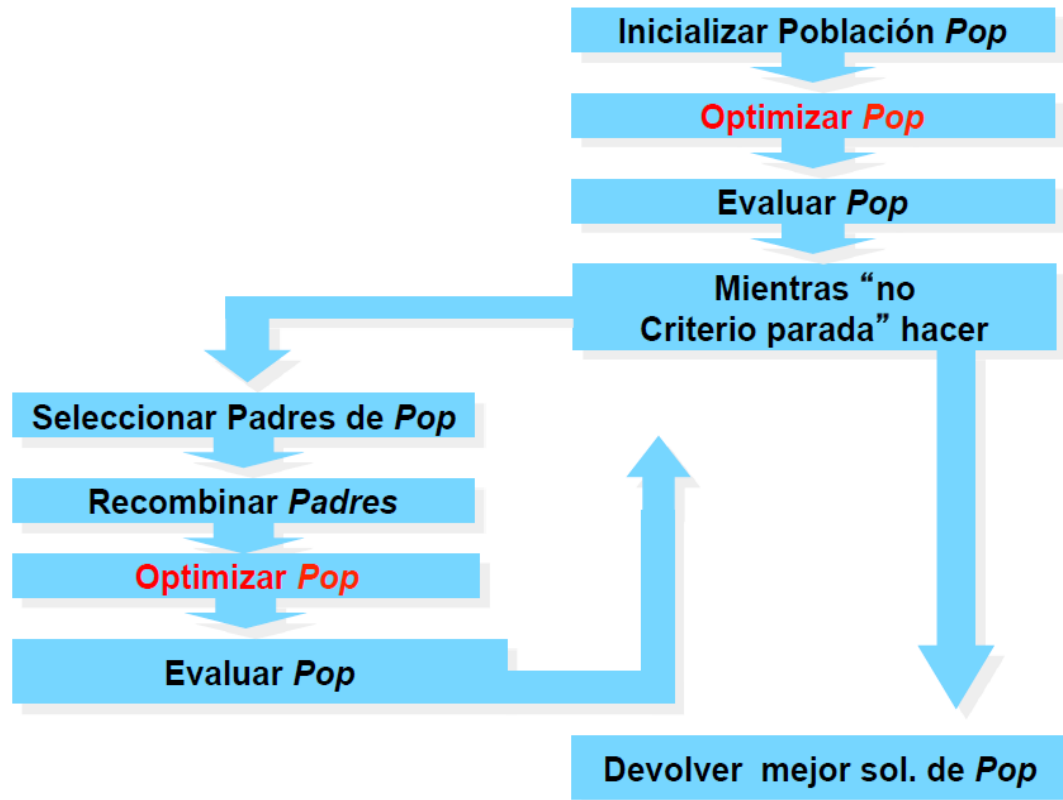
Información memética (que se transmite):

- Es importante ser de un solo color,
- Es mejor optimizar tamaño orejas,
- Grado curvatura colmillos,
- etc.

Características del proceso de Mejora Local a transmitir:

- ✓ *Esfuerzo en la búsqueda (competición recursos).*
- ✓ *Sobre qué individuos*
- ✓ *Definición de la vecindad*
- ✓ *Tipo metaheurística de mejora local, Parámetros*
- ✓ *Etc.*

GA	
<i>codificación</i>	esquemas, cadenas lineales, alfabetos predefinidos
<i>individuo</i>	solución al problema
<i>cruce</i>	intercambio no-guiado de información
<i>mutación</i>	introducción aleatoria de nueva información
MA	
<i>representación</i>	formas, no-linealidad, cercanía al problema
<i>agente</i>	solución/ones al problema + mecanismo mejora local
<i>recombinación</i>	intercambio guiado de información
<i>mutación</i>	introducción sensible de nueva información
<i>mejora local</i>	aprendizaje lamarckiano



UN AM puede caracterizarse como una colección de agentes que realizan **exploraciones autónomas del espacio de búsqueda**,

cooperando ocasionalmente a través de la recombinación, y compitiendo continuamente por los recursos computacionales a través de la selección y el reemplazo

Algoritmo Memético

(No existe una implementación única, sino variantes.

Un AM es un esquema a aplicar en la resolución del problema)

Inicialización: Generar una población inicial de individuos (agentes/soluciones: genotipos + memes);

Mientras No condición-de-Parada, hacer

Población inicial de Agentes:
{Soluciones}, {Características}, {Procesos mejora local}

1 **Evaluar** los individuos (fenotipos) de la población (fitness).

2 **Paso Generacional:** Obtener una nueva población

Padres := Selección (Población, **Fitness**);

Nueva_Población := Reproducir (Padres, **Cruce**);

Nueva_Población := Mutación (Población, **Mutación**);

Población := Actualizar (Población, Nueva_Población, **Reemplazo**, Fitness)

Información a heredar: Genes + Memes:
No exclusivamente genes, sino características
inferidas de buenas soluciones
Cómo aplicar buenos procesos de mejora?
Cruces basados en información del problema

Incluye mutación genes y memes
Posible Proceso de Mejora

Incluye genes
y memes

3 **Seleccionar** un Conjunto de Individuos (**W**) que deben someterse a mejora individual

4 Para cada individuo de W, hacer:

Nuevo Individuo W = Proceso_Mejora (W, **búsqueda local**)

En la Mejora Local, el individuo
hace uso de información
heredada del padre (meme).

Fin Mientras

Los meta-operadores de Selección, Cruce, Mutación y Reemplazo trabajan con información añadida del problema para hacer un proceso no exclusivamente sintáctico

Decisiones de Diseño de una AM

- **¿Cuándo se aplica el Algoritmo de Búsqueda Local?:** Tras cada cruce?, Al final de cada ciclo? Cada k-generaciones? Decisión estocástica?
- **¿Sobre qué agentes (W) se aplica?:** A toda la población? A un subconjunto (mejores, representantes, aleatorios,...?)
- **¿Qué uso se hace del agente optimizado?:** Sustituye al agente origen (aprendizaje Lamarkiano)? El agente origen se mantiene y solo recibe el fitness mejorado (aprendizaje Baldwiniano)
- **¿Qué Algoritmo de Búsqueda Local se utiliza?**
- **¿Cómo se aplica la optimizador local?** (Intensidad de la aplicación, frecuencia)

Por ejemplo:

En cada generación, aplicar la BL sobre los **N/10 mejores** cromosomas de la población (N es el tamaño de la población)

En cada generación, aplicar la BL sobre **todos** los cromosomas de la población

Cada 10 generaciones, aplicar la BL sobre los N/10 mejores cromosomas

Cada 10 generaciones, aplicar la BL a N/10 individuos seleccionados estocásticamente (ruleta, rango, etc.)

Otras Definiciones

Memetic algorithms contain a population of search agents, which is often implemented by GA or some other population based heuristic algorithm.

- The agents can produce or find good memes (*i.e. local search heuristics*) through evolution or innovation, and share these memes with other agents.
- If the memes are produced (e.g. evolved) and shared (imitated) properly, MA can obtain global optimum with high opportunity.

Memetic algorithm = memetic (culture, mental) evolution + genetic (agent, body) evolution.

- The basic evolution process of memes is the same as that of genes.
- Imitation or learning is the fundamental memetic operation.
 - Imitation is the process to learn or emulate the leader(s) or the parent(s);
 - Imitation is a crossover operation that recombines the original meme with others (social factor);
 - Imitation can also be a mutation operator to change one meme into another (personal factor).

Memetic algorithms: Hyper-heuristic algorithms (hybrid the heuristics in different ways, not limited by “levels”: GA + TS, GA + PSO, ...). Proper hybridization of different heuristics can improve the efficiency and the efficacy simultaneously.

Conclusiones

El AM puede verse como una colección de agentes que:

1. realizan exploraciones de mejora en su espacio de búsqueda (mutaciones),
2. cooperando ocasionalmente con el resto de agentes (soluciones) por medio de operaciones de cruce, y
3. compitiendo por los recursos computacionales (espacio) con procesos de selección y reemplazo.

La aportación fundamental frente a un AG es:

- La representación del individuo (solución) suele ser más compleja, estructurada, en vez de cadenas lineales (representado las soluciones de forma más estructurada)
- Un agente puede contener diversas soluciones e incorpora heurísticas de mejora local.
- Se incorpora un proceso de mejora local del agente, guiado por heurísticas propias. Esta mejora del padre (aprendizaje, meme) se traslada y hereda a los hijos.

Equilibrio en un AM:

- Los AM deben encontrar un equilibrio entre la diversidad de la población (AG) y la convergencia en vecindarios (Búsqueda Local).

*Los AM no son un paradigma ortodoxo". Muchos grados de libertad para el usuario.
No existe un procedimiento sistemático. Existen muchas variantes y alternativas de diseño.*

Múltiples aplicaciones. En general, más eficaces que los AG.

Búsqueda Dispersa

Búsqueda Dispersa sobre un Espacio de Soluciones

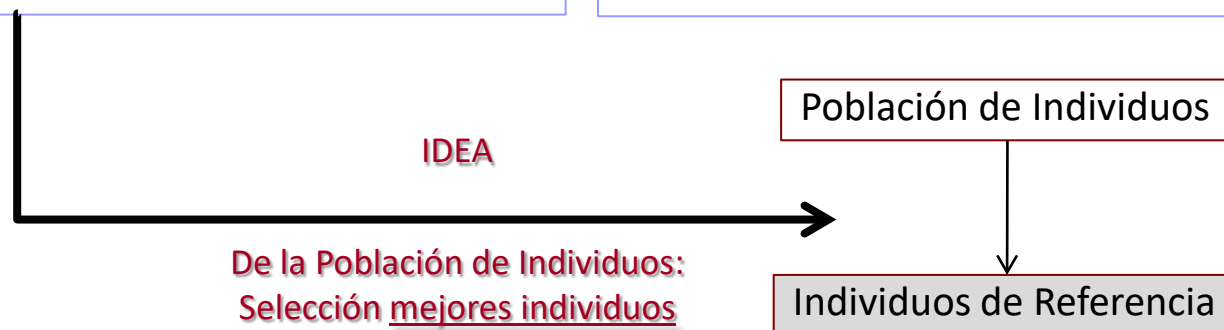
- Evolución de los algoritmos genéticos (como *algoritmo poblacional*, que construye soluciones mediante la combinación de otras soluciones).
- Integración de la Búsqueda en Haz (Beam Search) en un AG

Búsqueda por haz local (*beam search*)

- Comienza con k estados (soluciones) generados aleatoriamente.
- En cada paso se generan *todos los sucesores* de los k estados: $k \cdot b$ estados.
- Se seleccionan los k mejores sucesores de la lista completa, y se repite el proceso

Algoritmo Genético

- Comienza con n individuos (soluciones) generados aleatoriamente.
- En cada paso se cruzan y mutan, generando nuevos individuos.
- Los nuevos individuos se añaden (o reemplazan) a la población inicial, y se repite el proceso



Búsqueda Dispersa sobre un Espacio de Soluciones:

Algoritmo poblacional, que construye soluciones mediante la combinación de otras soluciones seleccionadas.

- Opera sobre un conjunto de soluciones, llamadas *soluciones de referencia*, que son una selección de “buenas” soluciones”, extraída del conjunto de soluciones.

Población: {Soluciones} $\Rightarrow$ **Soluciones de referencia:** {Mejores-Soluciones}

Reducida población. *Típicamente*, $|Sol_Ref| \leq |Población|/10$

- Las Soluciones de Referencia son combinadas para obtener nuevas soluciones.

Las combinaciones son “*formas*” de combinaciones lineales de (dos o más) soluciones:

$$\text{Solución-Nueva} = \alpha_1 S_1 + \alpha_2 S_2 + \dots + \alpha_n S_n,$$

Similar a Cruce y Selección

con procesos adaptativos para garantizar la factibilidad.

Este proceso (*combinación lineal ponderada de soluciones previas*) es una forma más estructurada de ‘*cruce*’ y ‘*selección*’.

- Las nuevas soluciones suelen tener un posterior proceso de mejora local.
- De las antiguas soluciones de referencia, y las nuevas soluciones obtenidas, se obtiene el nuevo conjunto de soluciones de referencia (proceso iterativo)

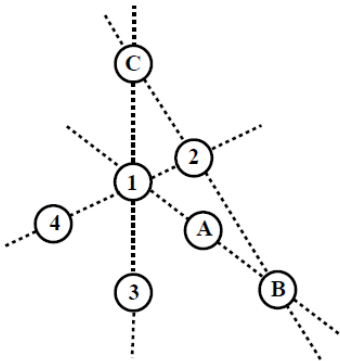
Ejemplo de Combinación-1:

Ejemplo: 8 reinas

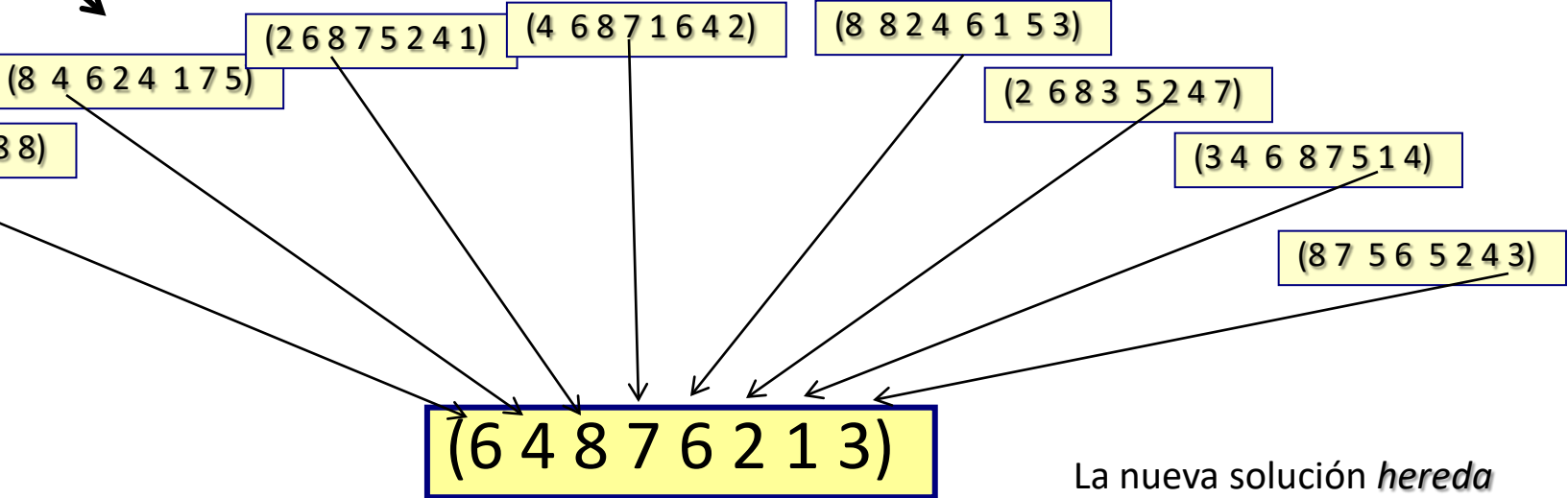


Soluciones de Referencia

- Nuevas soluciones se obtienen mediante combinación lineal de soluciones de referencia.
- El objetivo es transmitir las buenas propiedades que tienen las mejores soluciones (generando soluciones en el entorno de buenas soluciones)



Soluciones de Referencia



Nueva solución

La nueva solución *hereda* propiedades del conjunto de soluciones de referencia

$$\text{Solución-Nueva} = \alpha_1 S_1 + \alpha_2 S_2 + + \alpha_n S_n$$

Nueva solución: Obtiene fila_i de la solución_i

Ejemplo de Combinación-2: Generar rutas de dos autobuses, a través de 10 nodos, para recoger a estudiantes. (Corberán et al. Tech report TR08-2000, D.E.I.O., UV.2000)

Supongamos, dos soluciones: **Solución A:** $A1 = \{ 4, 2, 7, 1 \}$, $A2 = \{ 5, 10, 6, 9, 3, 8 \}$

Solución B: $B1 = \{ 2, 6, 8, 10, 9 \}$, $B2 = \{ 3, 4, 7, 1, 5 \}$

Combinando A y B, obtenemos dos nuevas soluciones: **N1** = Combina (A1 B2), **N2**= Combina (A2 B1)

La combinación es más compleja que un simple cruce.

COMBINACIÓN DE SOLUCIONES.

Casos:

- En N1 se añade {4, **3**}, porque '2' ya en N2 y '3' precede a '7' en B2.
- En N2 se añade {2, **5**}, porque 5 precede a 6 en A2.
- etc....

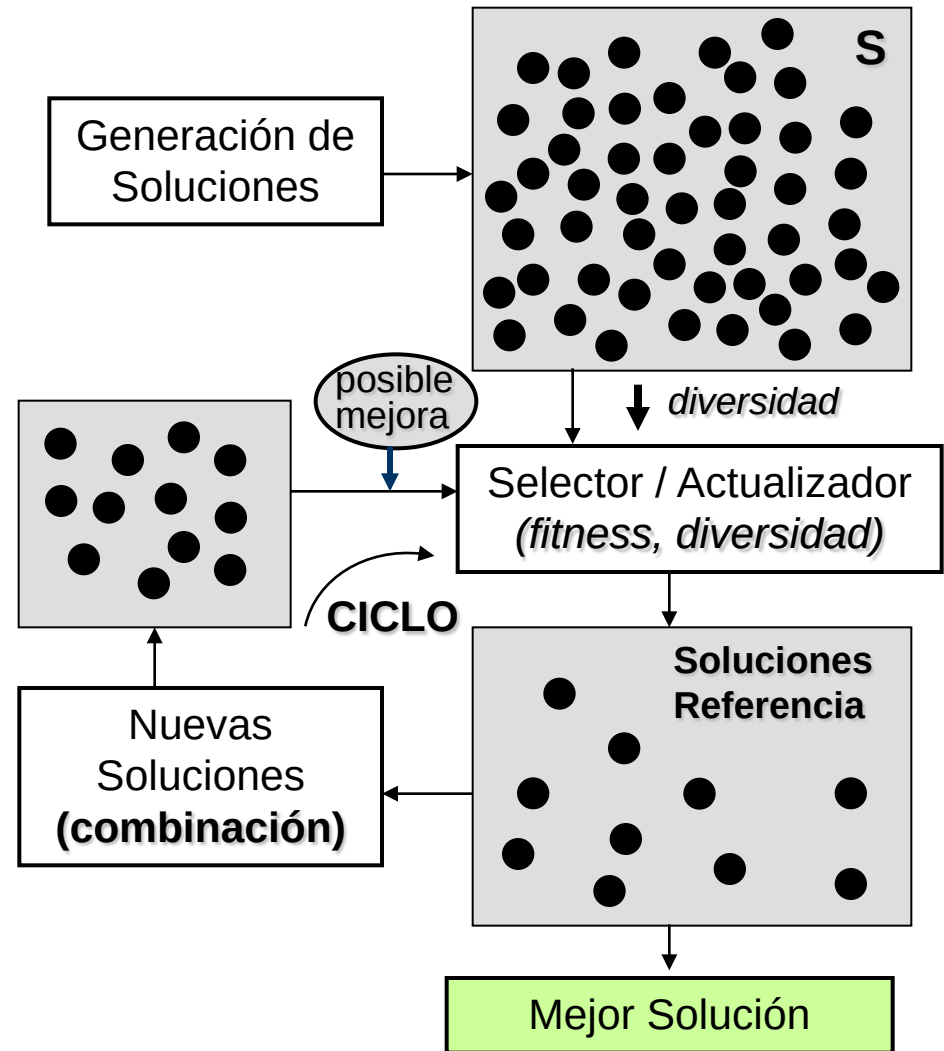
Paso	Par	Voto 1	Voto 2	Asignación	Regla de selección
1	(A ₁ ,B ₂)	4	3	N ₁ = { 4 }	azar
2	(A ₂ ,B ₁)	5	2	N ₂ = { 2 }	azar
3	(A ₁ ,B ₂)	7	3	N ₁ = { 4, 3 }	3 antes 7
4	(A ₂ ,B ₁)	5	6	N ₂ = { 2, 5 }	5 antes 6
5	(A ₁ ,B ₂)	7	7	N ₁ = { 4, 3, 7 }	igual
6	(A ₂ ,B ₁)	10	6	N ₂ = { 2, 5, 6 }	azar
7	(A ₁ ,B ₂)	1	1	N ₁ = { 4, 3, 7, 1 }	igual
8	(A ₂ ,B ₁)	10	8	N ₂ = { 2, 5, 6, 10 }	10 antes 8
9		9	8	N ₂ = { 2, 5, 6, 10, 8 }	8 antes 9
10		9	9	N ₂ = { 2, 5, 6, 10, 8, 9 }	igual

Obtenemos: **N1** = { 4, 3, 7, 1 } **N2** = { 2, 5, 6, 10, 8, 9 }

Esquema General Búsqueda Dispersa (Scatter Search):

1. Generar conjunto inicial de soluciones que garantice diversidad (*semillas*)
2. Seleccionar un subconjunto de las mejores soluciones (criterios: función objetivo y diversidad): *Soluciones de Referencia*.
3. Crear *nuevas soluciones* combinando soluciones de referencia. Se adaptan para hacerlas factibles.
4. Mejorar las nuevas soluciones (*búsqueda local*)
5. Extraer las *mejores soluciones* y se añaden al conjunto de soluciones de referencia.
6. Eliminar del conjunto de referencia las menos óptimizadas/diversas.
7. Tras varias iteraciones, se extrae la mejor solución.

Si en alguna iteración no se consigue mejorar el conjunto de soluciones de referencia, se añaden nuevas soluciones de S que añadan diversidad.

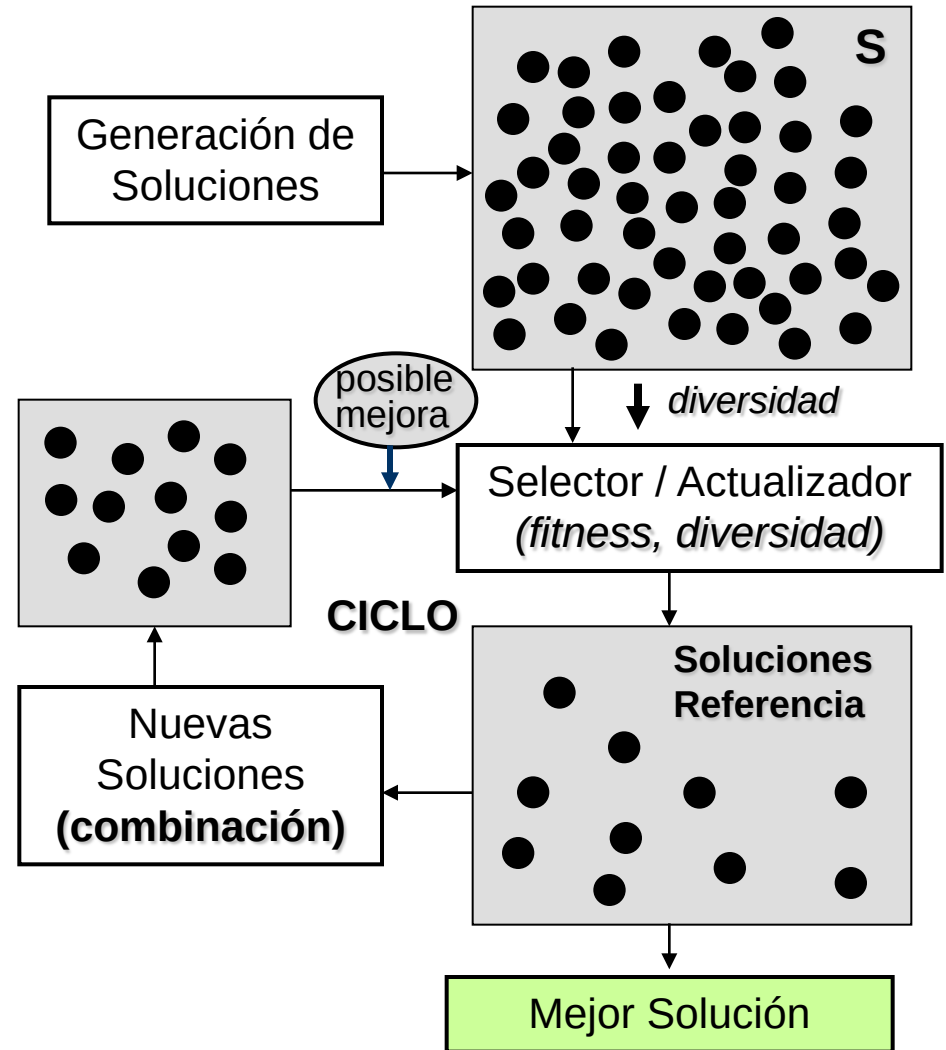


Conclusiones

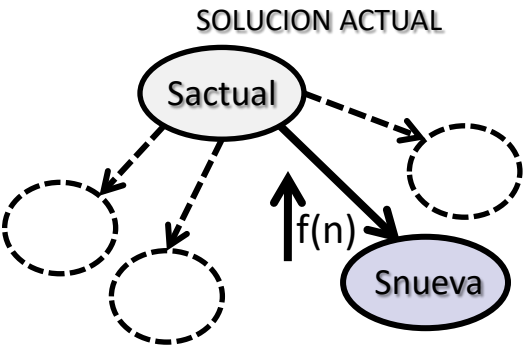
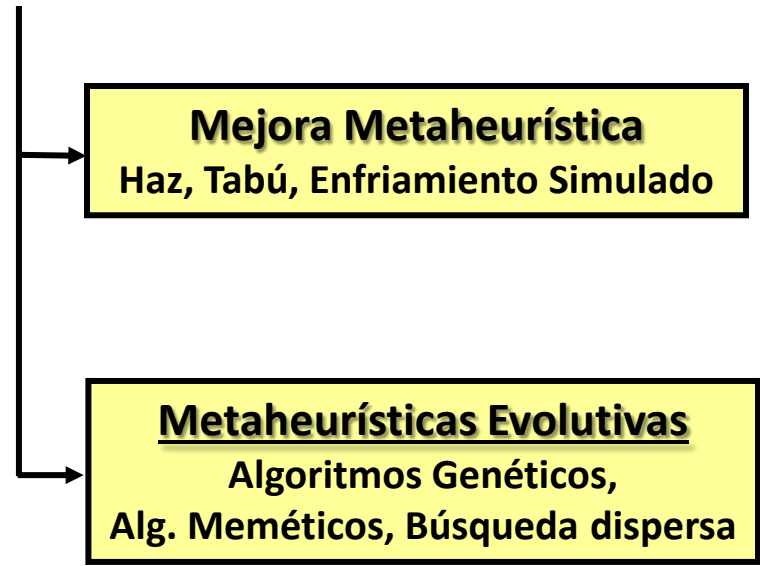
La búsqueda dispersa fundamentalmente introduce:

1. *Soluciones de referencia (haz)*, como selección de mejores soluciones en la población.
2. Método de **combinación** de soluciones de referencia.
3. **Mejora local** de nuevas soluciones.
4. **Revisión** del nuevo conjunto de soluciones de referencia (haz)

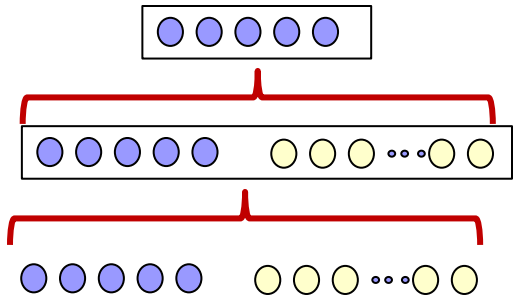
Adecuado para problemas de los que se tengan soluciones iniciales optimizadas.



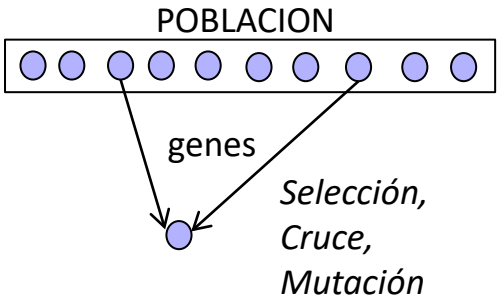
Evolución Metaheurísticas



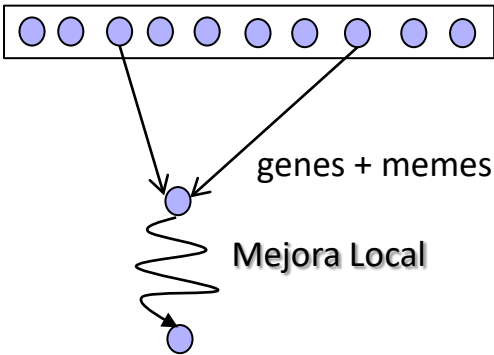
BÚSQUEDA EN EL VECINDARIO $N(s)$
(Enfriamiento Simulado, Tabú)



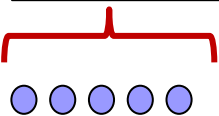
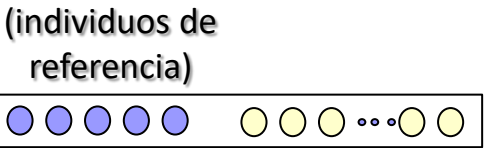
AG: Padres transmiten genes a hijos



AM: Padres transmiten genes/memes a hijos



Búsqueda Dispersa



Individuos de referencia
colaboran para generar
nueva población